
Syntax directed
Definitions

Well, hello there, bright minds! 

Step into the classroom, grab your favorite seat

and let's dive into the marvelous world of compiler design!

Today, we are tackling a topic:

Syntax-Directed Definitions (SDDs).

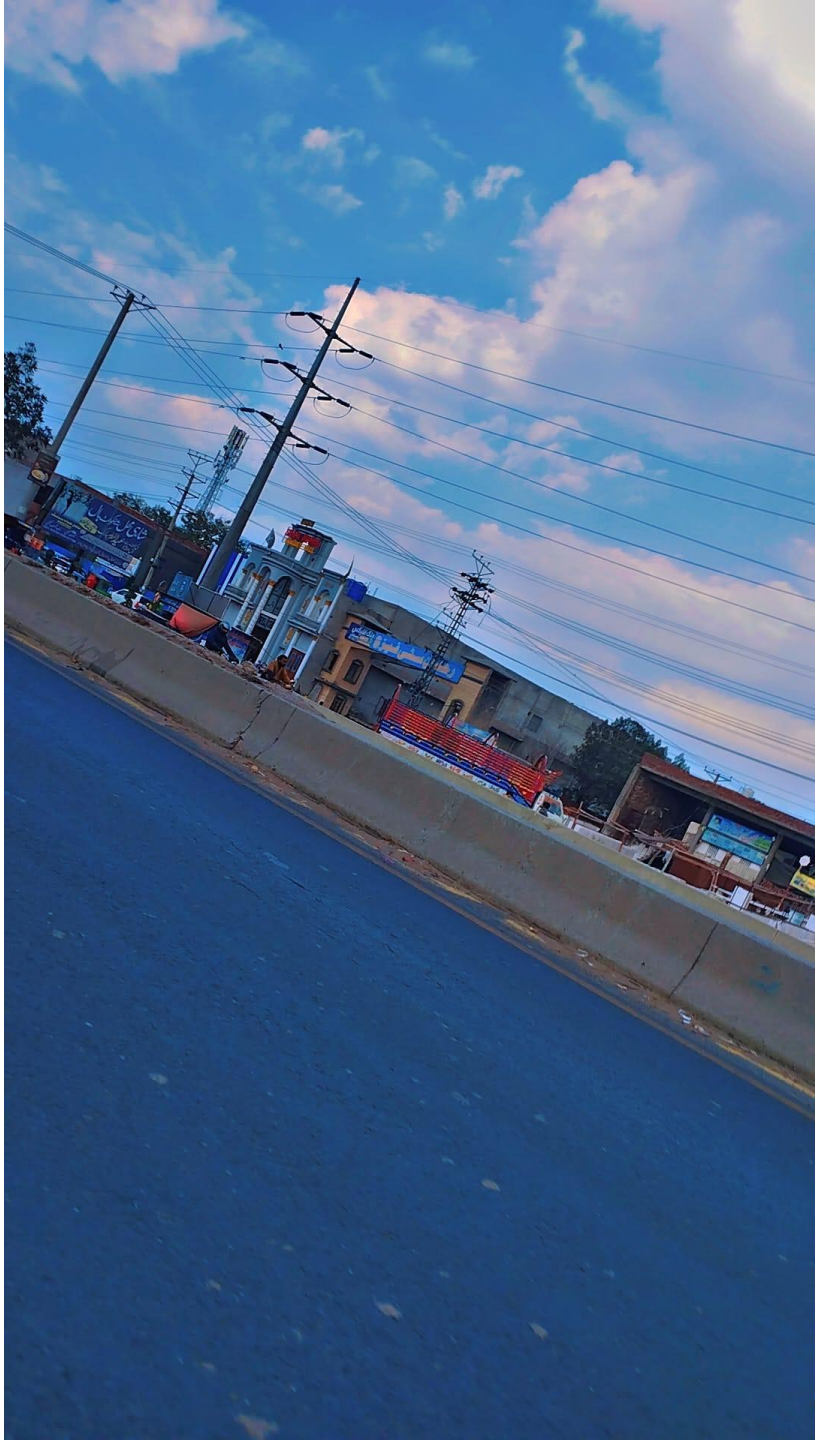
Kazy Noor e Alam Siddiquee, PhD

Professor, CSE, MIST






MIND
BOX







- Context-Free Grammar (CFG)
- Production Rule
- Parse Tree
- Syntax Tree
- Attribute
- Attribute Grammar
- Semantic Rule
- Semantic Action
- Semantic Analysis
- Static Semantics
- Type Checking
- Type Inference
- Symbol Table
- Scope Information
- Translation Scheme
- Semantic Information
- Compiler Front End
- Declaration Processing
- Expression Evaluation
- Data Type Propagation
- Semantic Error Detection



🎭 What in the World is an SDD?

In compiler land, a context-free grammar (CFG) tells us *how code is built* (the syntax/structure), but it has no idea *what the code actually means* (the semantics).

Enter **Syntax-Directed Definitions (SDDs)**!

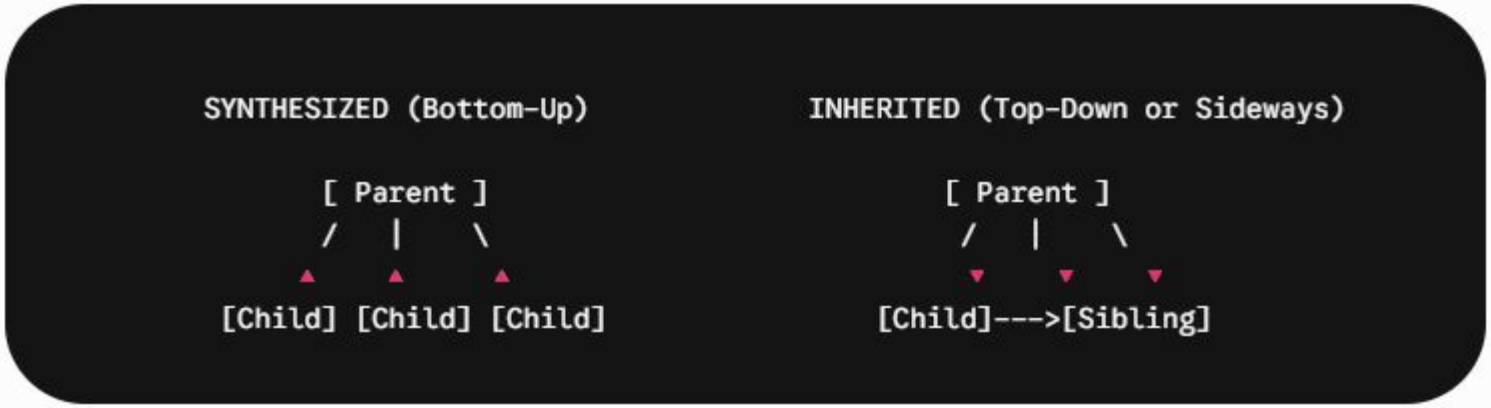
"Syntax without semantics is like a skeleton without muscle—it holds a pose, but it can't actually do anything useful!"

An **SDD** is simply a Context-Free Grammar combined with **attributes** and **semantic rules**:

- **Attributes:** Variables associated with grammar symbols (e.g., a number's `val`, a type's `type`, or a variable's `location`).
- **Semantic Rules:** Mathematical or logical functions attached to productions that calculate those attribute values.

🏷️ The Two Stars of the Show: Synthesized vs. Inherited Attributes

To understand SDDs, you need to meet the two dynamic siblings of semantic evaluation:



1. Synthesized Attributes (The "Bottom-Up" Overachievers)

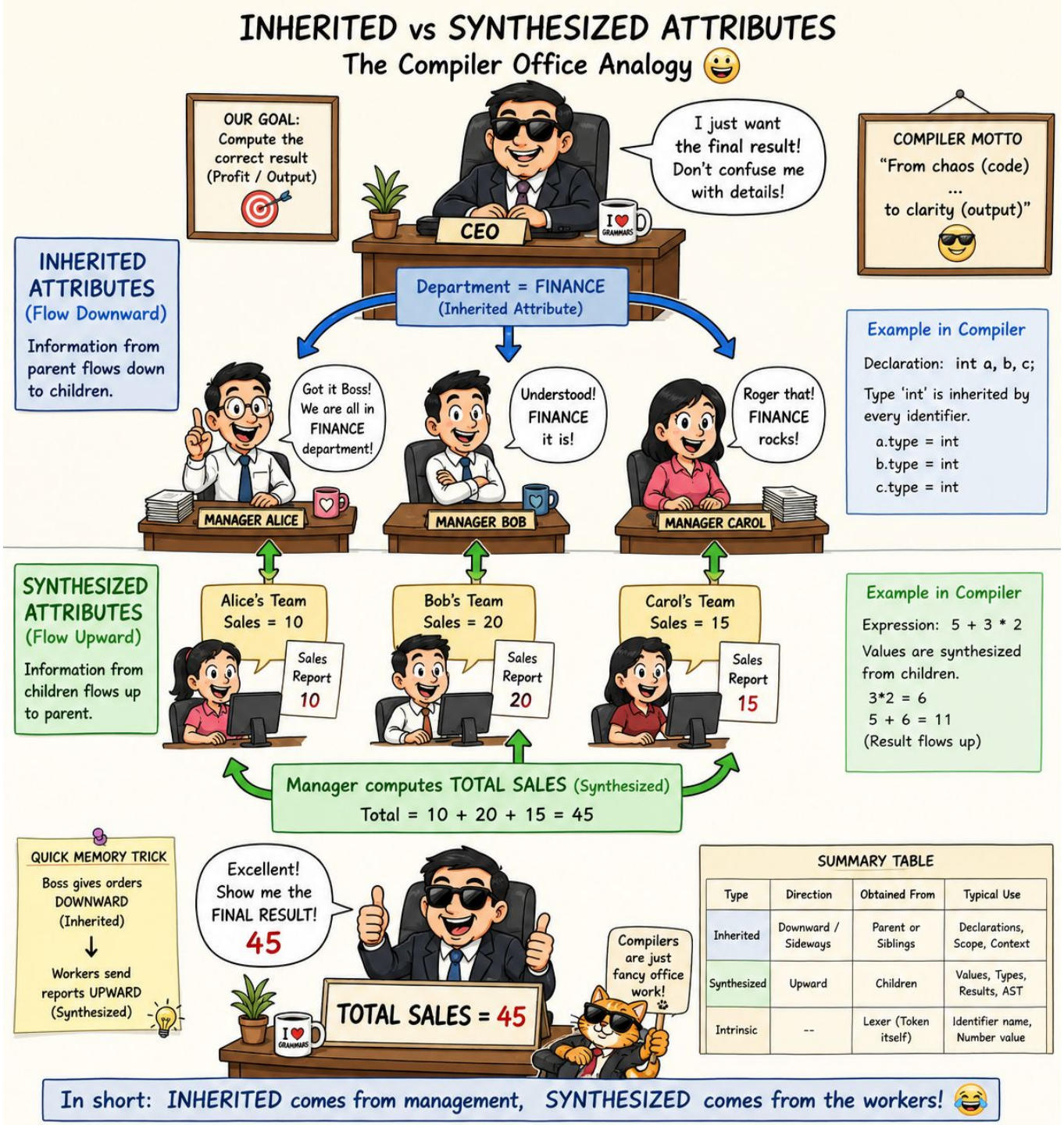
- **How they work:** The child nodes compute their values first and pass them **UP** to their parent node.
- **Real-world analogy:** Like a group project where every student does their part, hands it to the team lead, and the team lead compiles the final report.

2. Inherited Attributes (The "Top-Down" Micro-managers)

- **How they work:** The parent or sibling nodes pass values **DOWN** or **ACROSS** to a node.
- **Real-world analogy:** Like parent-teacher advice—you inherit guidance from above (or judgment from your older sibling beside you) before you compute your own life path!

INHERITED vs SYNTHESIZED ATTRIBUTES

The Compiler Office Analogy 😀



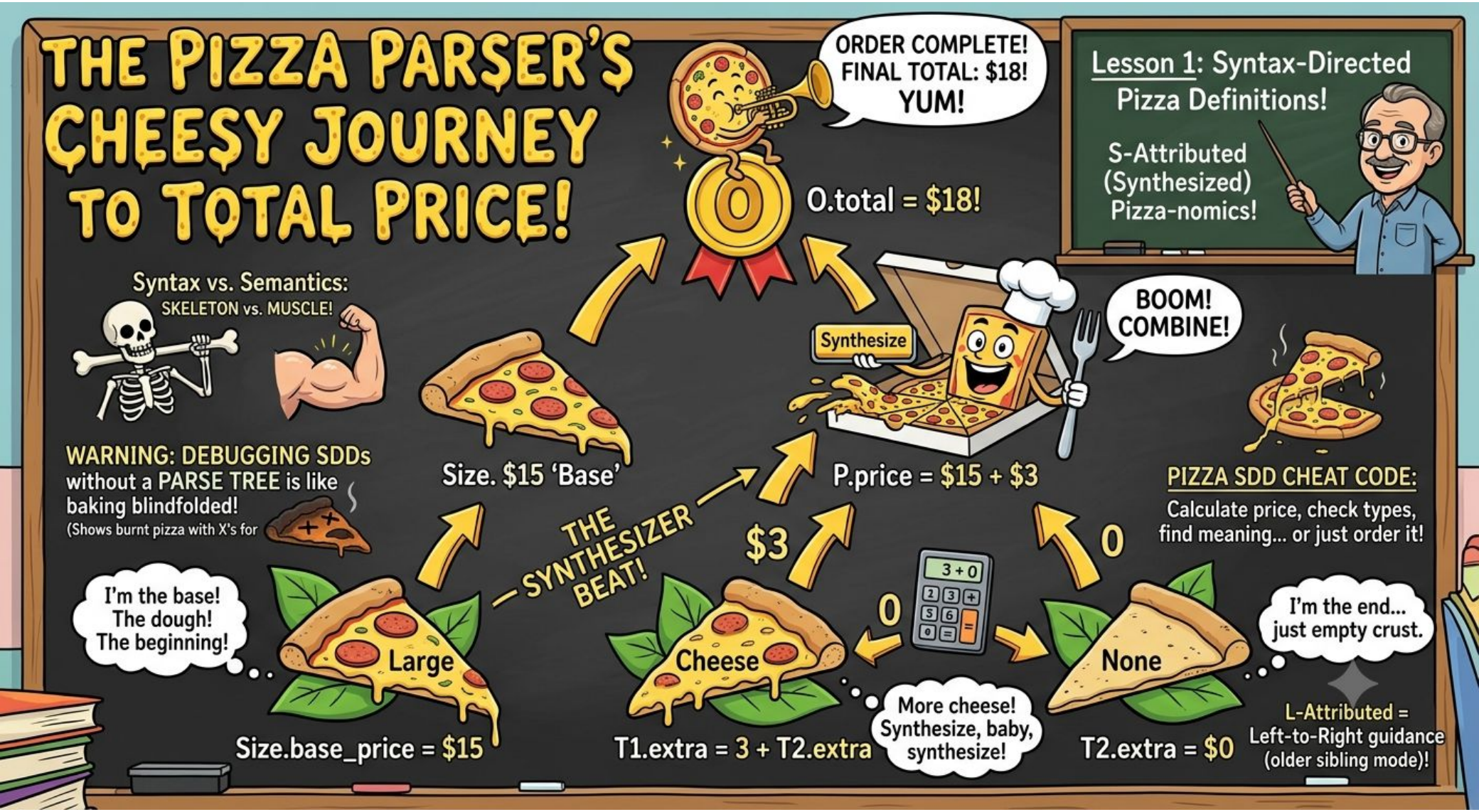
🍕 Real-Time Example: The Pizza Order Parser

Let's make this crystal clear with a real-time, appetizing example! Imagine we are building a compiler for a **Pizza Ordering App**.

We want our parser to calculate the total price of a pizza order based on the size and selected toppings.

1. The Syntax Rules (Grammar) & Semantic Rules (SDD)

Production Rule	Semantic Rule (SDD Action)	Attribute Type
$O \to P$	$O.\text{total} = P.\text{price}$	Synthesized
$P \to \text{Size } T$	$P.\text{price} = \text{Size.base_price} + T.\text{extra}$	Synthesized
$\text{Size} \to \text{'Large'}$	$\text{Size.base_price} = 15$	Synthesized
$T_1 \to \text{'Cheese'}\ T_2$	$T_1.\text{extra} = 3 + T_2.\text{extra}$	Synthesized
$T \to \text{'None'}$	$T.\text{extra} = 0$	Synthesized


THE PIZZA PARSER'S CHEESY JOURNEY TO TOTAL PRICE!
ORDER COMPLETE! FINAL TOTAL: $18! YUM!
Lesson 1: Syntax-Directed Pizza Definitions!
S-Attributed (Synthesized) Pizza-nomics!
O.total = $18!
Syntax vs. Semantics:
SKELETON vs. MUSCLE!
Synthesize
BOOM! COMBINE!
WARNING: DEBUGGING SDDs without a PARSE TREE is like baking blindfolded!
(Shows burnt pizza with X's for
Size. $15 'Base'
P.price = $15 + $3
PIZZA SDD CHEAT CODE:
Calculate price, check types, find meaning... or just order it!
THE SYNTHESIZER BEAT!
$3
0
0
I'm the base! The dough! The beginning!
Large
Cheese
None
I'm the end... just empty crust.
More cheese! Synthesize, baby, synthesize!
Size.base_price = $15
T1.extra = 3 + T2.extra
T2.extra = $0
L-Attributed = Left-to-Right guidance (older sibling mode)!

Quick Summary Checklist

- **Syntax Rules** create the structure (the parse tree skeleton).
- **Semantic Rules** compute the meaning (populating values on the nodes).

- **S-Attributed SDD:** Uses **only** synthesized attributes (evaluated easily in a bottom-up pass).
- **L-Attributed SDD:** Can use synthesized attributes **and** restricted inherited attributes (evaluated left-to-right)

THEORY

Syntax-Directed Translation

1. We associate information with the programming language constructs by attaching attributes to grammar symbols.

2. Values of these attributes are evaluated by the **semantic rules** associated with the production rules.

3. Evaluation of these semantic rules:
   - may generate intermediate codes
   - may put information into the symbol table
   - may perform type checking
   - may issue error messages
   - may perform some other activities
   - in fact, they may perform almost any activities.

4. An attribute may hold almost any thing.
   - a string, a number, a memory location, a complex record.

Syntax-Directed Definitions and Translation Schemes

1. When we associate semantic rules with productions, we use two notations:
   - **Syntax-Directed Definitions**
   - **Translation Schemes**

A. Syntax-Directed Definitions:

- give high-level specifications for translations
- hide many implementation details such as order of evaluation of semantic actions.
- We associate a production rule with a set of semantic actions, and we do not say when they will be evaluated.

B. Translation Schemes:

- indicate the order of evaluation of semantic actions associated with a production rule.
- In other words, translation schemes give a little bit information about implementation details.

Syntax-Directed Translation

- Conceptually with both the syntax directed translation and translation scheme we
  - Parse the input token stream
  - Build the parse tree
  - Traverse the tree to evaluate the semantic rules at the parse tree nodes.



Conceptual view of syntax directed translation

Syntax-Directed Definitions

1. A syntax-directed definition is a generalization of a context-free grammar in which:
   - Each grammar symbol is associated with a set of attributes.
   - This set of attributes for a grammar symbol is partitioned into two subsets called
     - **synthesized** and
     - **inherited** attributes of that grammar symbol.
   - Each production rule is associated with a set of semantic rules.

2. The value of an attribute at a parse tree node is defined by the semantic rule associated with a production at that node.
3. The value of a **synthesized attribute** at a node is computed from the values of attributes at the children in that node of the parse tree
4. The value of an **inherited attribute** at a node is computed from the values of attributes at the siblings and parent of that node of the parse tree

Syntax-Directed Definitions

Examples:

Synthesized attribute : E→E1+E2 { E.val =E1.val + E2.val}

Inherited attribute :A→XYZ {Y.val = 2 * A.val}

1. *Semantic rules* set up dependencies between attributes which can be represented by a *dependency graph*.

2. This *dependency graph* determines the evaluation order of these semantic rules.

3. Evaluation of a semantic rule defines the value of an attribute. But a semantic rule may also have some side effects such as printing a value.

Annotated Parse Tree

1. A parse tree showing the values of attributes at each node is called an **annotated parse tree**.
2. Values of Attributes in nodes of annotated parse-tree are either,
   - initialized to constant values or by the lexical analyzer.
   - determined by the semantic-rules.

3. The process of computing the attributes values at the nodes is called **annotating** (or **decorating**) of the parse tree.

4. Of course, the order of these computations depends on the dependency graph induced by the semantic rules.

Syntax-Directed Definition

In a syntax-directed definition, each production $A \rightarrow \alpha$ is associated with a set of semantic rules of the form:

$$b = f(c_1, c_2, \ldots, c_n)$$

where f is a function and b can be one of the followings:

- b is a synthesized attribute of A and $c_1, c_2, \ldots, c_n$ are attributes of the grammar symbols in the production ($A \rightarrow \alpha$).

OR

- b is an inherited attribute one of the grammar symbols in α (on the right side of the production), and $c_1, c_2, \ldots, c_n$ are attributes of the grammar symbols in the production ($A \rightarrow \alpha$).

Attribute Grammar

- So, a semantic rule $b=f(c_1,c_2,\ldots,c_n)$ indicates that the attribute b *depends* o*n* attributes $c_1,c_2,\ldots,c_n$.

- In a **syntax-directed definition**, a semantic rule may just evaluate a value of an attribute or it may have some side effects such as printing values.

- An **attribute grammar** is a syntax-directed definition in which the functions in the semantic rules cannot have side effects (they can only evaluate values of attributes).

Syntax-Directed Definition -- Example

Production	**Semantic Rules**
$L \rightarrow E$ **n**	print(E.val)
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	E.val = T.val
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	T.val = F.val
$F \rightarrow (E)$	F.val = E.val
$F \rightarrow$ **digit**	F.val = **digit**.lexval

1. Symbols E, T, and F are associated with a synthesized attribute *val*.
2. The token **digit** has a synthesized attribute *lexval* (it is assumed that it is evaluated by the lexical analyzer).
3. Terminals are assumed to have synthesized attributes only. Values for attributes of terminals are usually supplied by the lexical analyzer.
4. The start symbol does not have any inherited attribute unless otherwise stated.

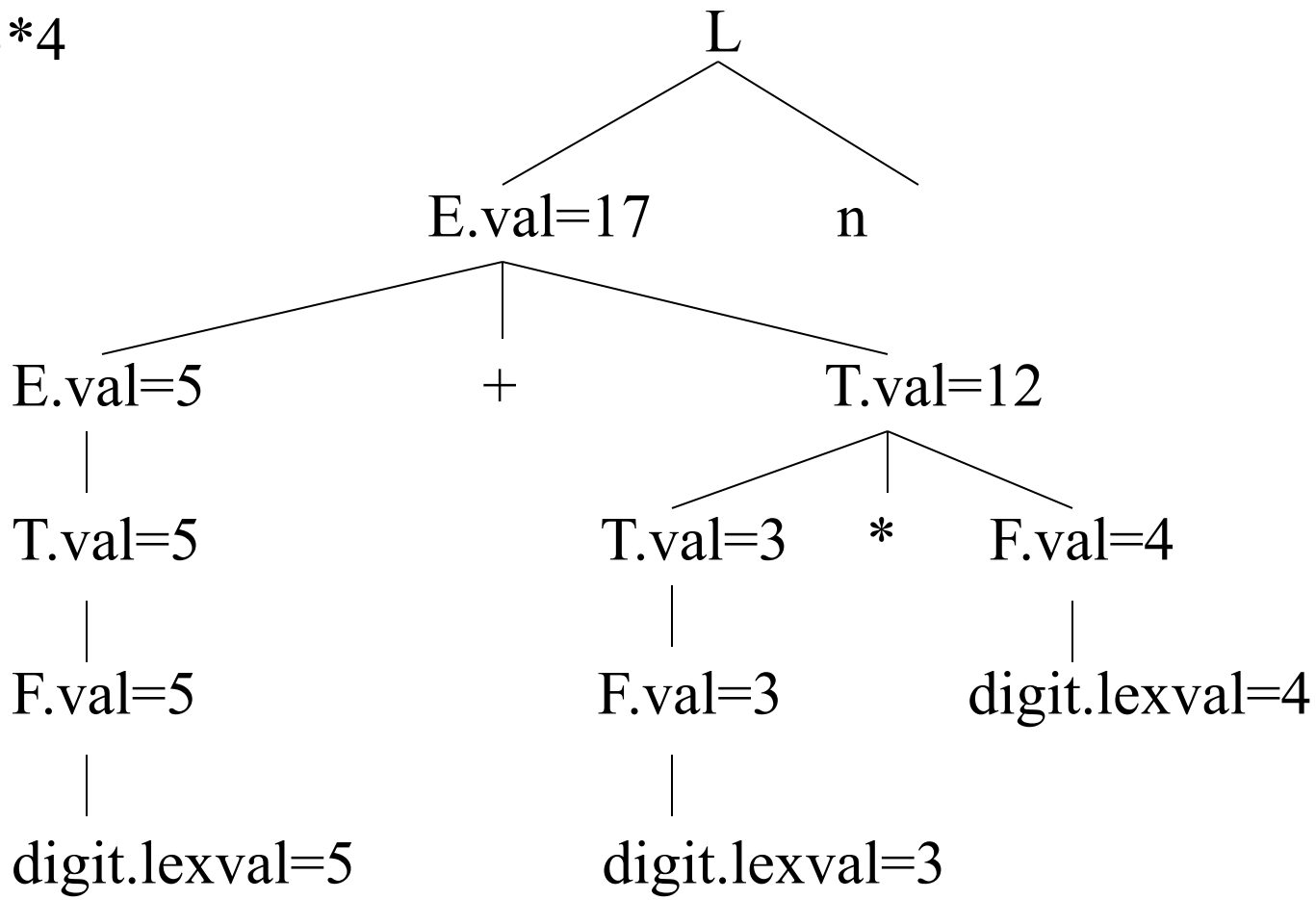
S-attributed definition

- A syntax directed translation that uses synthesized attributes exclusively is said to be a S-attributed definition.

- A parse tree for a S-attributed definition can be annotated by evaluating the semantic rules for the attributes at each node, bottom up from leaves to the root.

Annotated Parse Tree -- Example

Input: 5+3*4



Inherited attributes

- An inherited value at a node in a parse tree is defined in terms of attributes at the parent and/or siblings of the node.
- Convenient way for expressing the dependency of a programming language construct on the context in which it appears.
- We can use inherited attributes to keep track of whether an identifier appears on the left or right side of an assignment to decide whether the address or value of the assignment is needed.
- Example: The inherited attribute distributes type information to the various identifiers in a declaration.

Syntax-Directed Definition – Inherited Attributes

Production	**Semantic Rules**
$D \rightarrow T\ L$	L.in = T.type
$T \rightarrow$ **int**	T.type = integer
$T \rightarrow$ **real**	T.type = real
$L \rightarrow L_1$ **id**	L_1.in = L.in, addtype(**id**.entry,L.in)
$L \rightarrow$ **id**	addtype(**id**.entry,L.in)

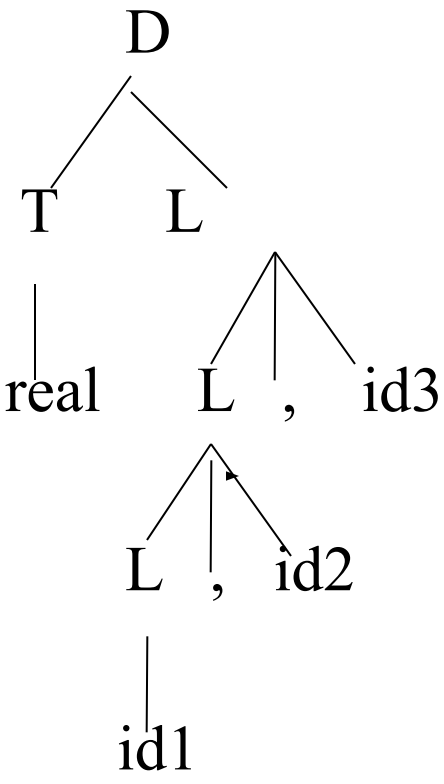
1. Symbol T is associated with a synthesized attribute *type*.

2. Symbol L is associated with an inherited attribute *in*.

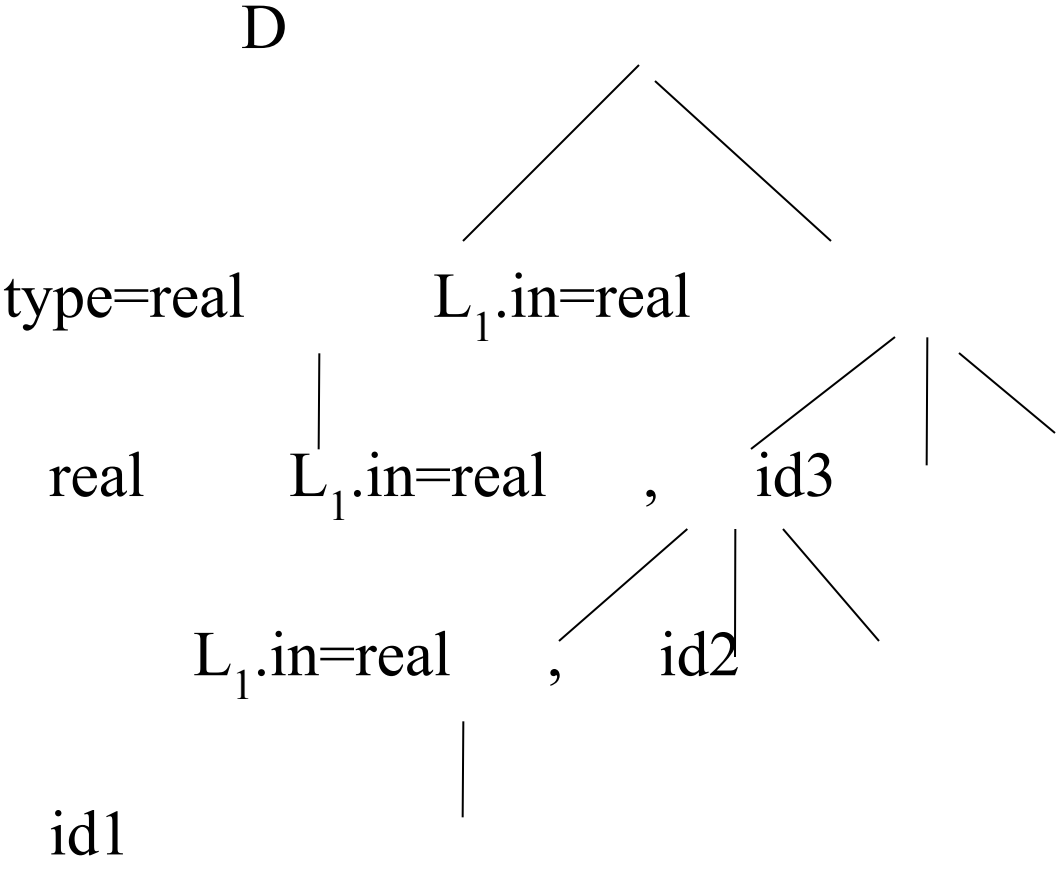
Annotated parse tree

Input: `real p,q,r`

parse tree



annotated parse tree



S vs L ATTRIBUTES

Compiler University Classroom 😀

In Compiler University, We pass information in two main ways!

Remember kids: Information has direction! 😎

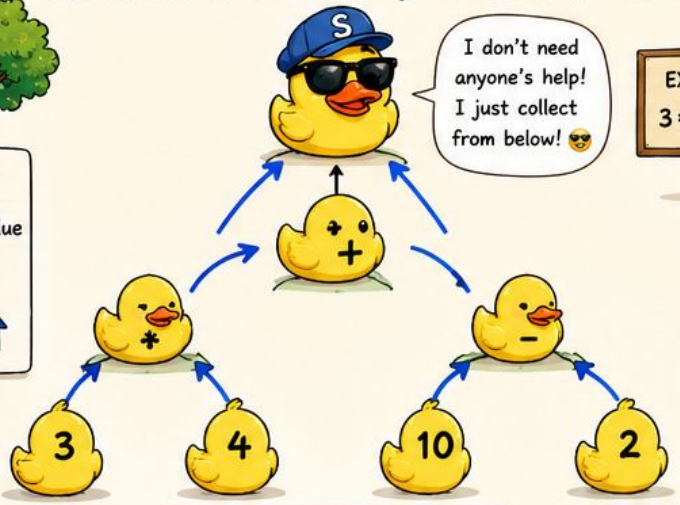
S-ATTRIBUTED

(Only Synthesized)

"Information flows UP like my exam marks (sometimes)!"



Rule:
Parent gets value from Children (Synthesized)
UPWARD ⬆



I don't need anyone's help! I just collect from below! 😎

EXPRESSION: 3 * 4 + 10 - 2

Boss, I just send my value up. That's all!

COMPUTATION (Flows UP)
3 * 4 = 12
10 - 2 = 8
12 + 8 = 20

Look! Final answer reaches the root! I'm S-uper! 😎

S-ATTRIBUTED = Children do the work, Parent just adds it up and takes the credit! 😉

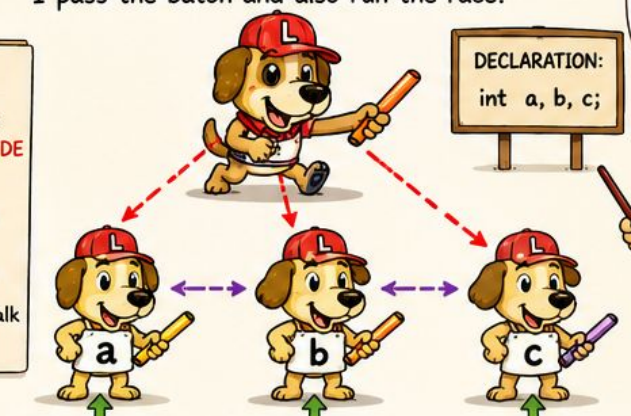
L-ATTRIBUTED

(Inherited + Synthesized)

"Information flows DOWN, SIDEWAYS and UP. I pass the baton and also run the race!"

RULES:
- Parent passes info DOWN/SIDE (Inherited)
- Children send results UP (Synthesized)
- Siblings can talk SIDEWAYS!

DECLARATION: int a, b, c;



a.type = int
b.type = int
c.type = int

RESULTS (Flow UP)
a.type = int, b.type = int, c.type = int
All gathered at the top! 😅

I give context down & sideways, you compute and send results up!

We got the context from above!

We compute our result and send it UP!

L-ATTRIBUTED = Pass the info DOWN & SIDEWAYS, Compute, and send results UP. Teamwork! 🐕🔥



In S, I work alone. In L, we work as a team. Either way... I want fish! 🐟

MORAL OF THE STORY:
Direction matters, Then computation gets smarter! 😄



QUICK COMPARISON

Type	Info Direction	Source	Example
S-Attributed	Bottom → Top	Children → Parent	Expression Evaluation
L-Attributed	Top → Down/Side & Up	Parent/Siblings → Child & Child → Parent	Declarations, Type info, Symbol Tables

Even in compilers, everyone passes something around! 😂



Dependency Graph

- Directed Graph
- Shows interdependencies between attributes.
- If an attribute b at a node depends on an attribute c, then the semantic rule for b at that node must be evaluated after the semantic rule that defines c.
- Construction:
  - Put each semantic rule into the form b=f(c_1,…,c_k) by introducing dummy synthesized attribute b for every semantic rule that consists of a procedure call.
  - E.g.,
    - **L → E n** ***print*(E.*val*)**
    - **Becomes:** ***dummy* = *print*(E.*val*)**
  - The graph has a node for each attribute and an edge to the node for b from the node for c if attribute b depends on attribute c.

Dependency Graph Construction

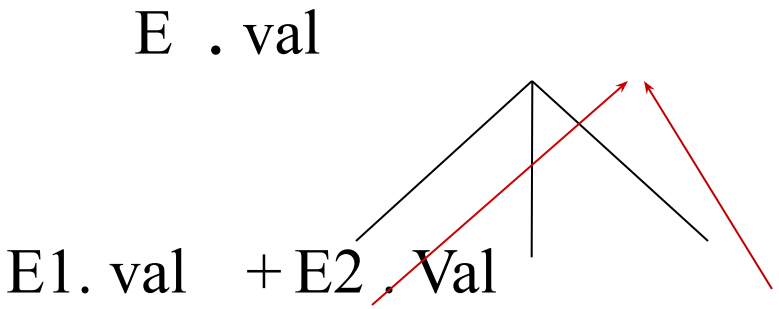
for each node *n* in the parse tree do
    for each attribute a of the grammar symbol at *n* do
     construct a node in the dependency graph for a

for each node *n* in the parse tree do
    for each semantic rule $b = f(c_1, ..., c_n)$
     associated with the production used at *n* do
          for i= 1 to n do
               construct an edge from
               the node for c_i to the node for *b*

Dependency Graph Construction

- Example
- Production Semantic Rule

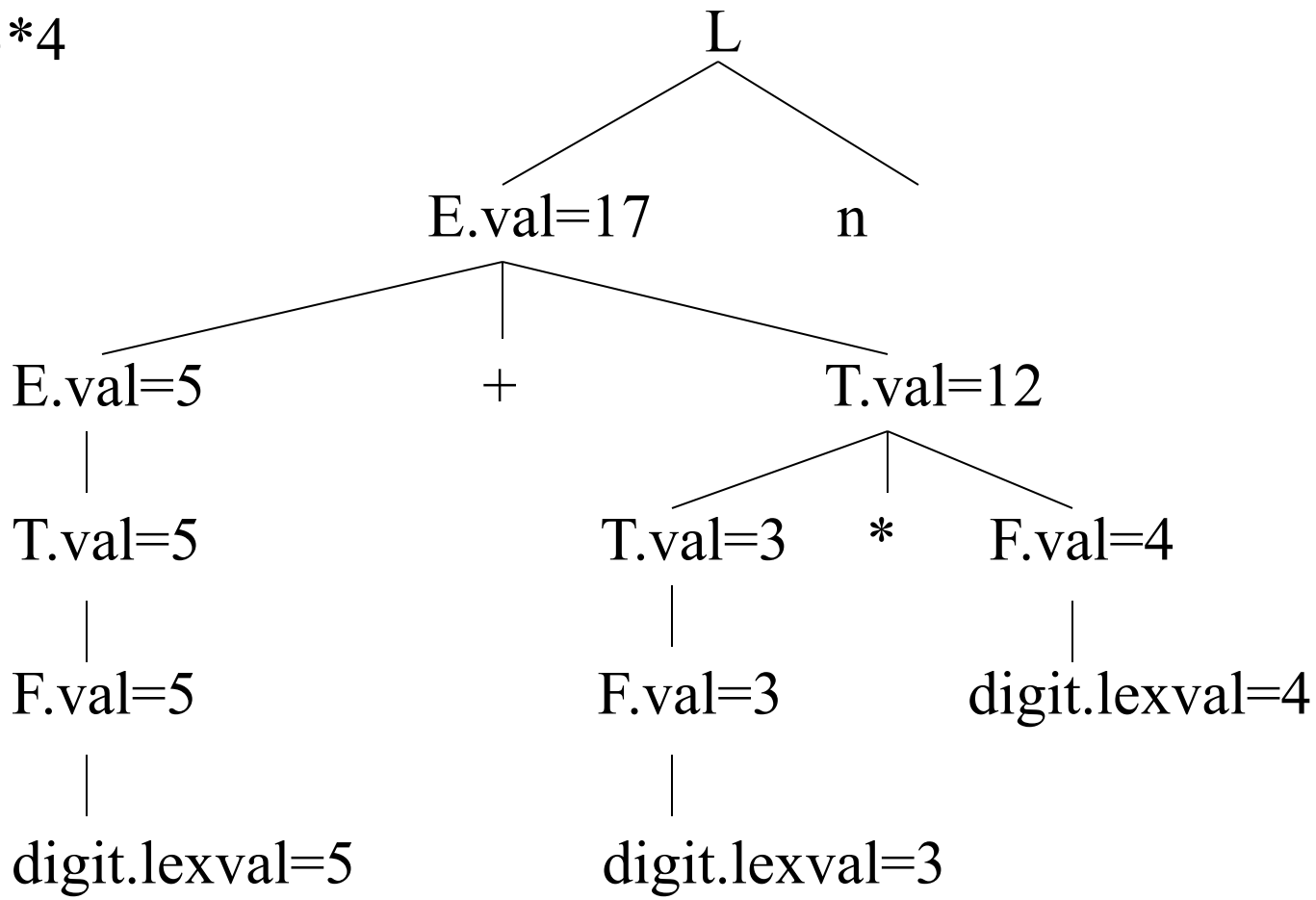
  E→E1 + E2 E.val = E1.val + E2.val



- E.val is synthesized from E1.val and E2.val
- The dotted lines represent the parse tree that is not part of the dependency graph.

Annotated Parse Tree – Example- S-attributes

Input: 5+3*4



Dependency Graph

D → T L L.in = T.type
T → **int** T.type = integer
T → **real** T.type = real
L → L_1 **id** L_1.in = L.in,
addtype(**id**.entry,L.in)

L → **id** addtype(**id**.entry,L.in)

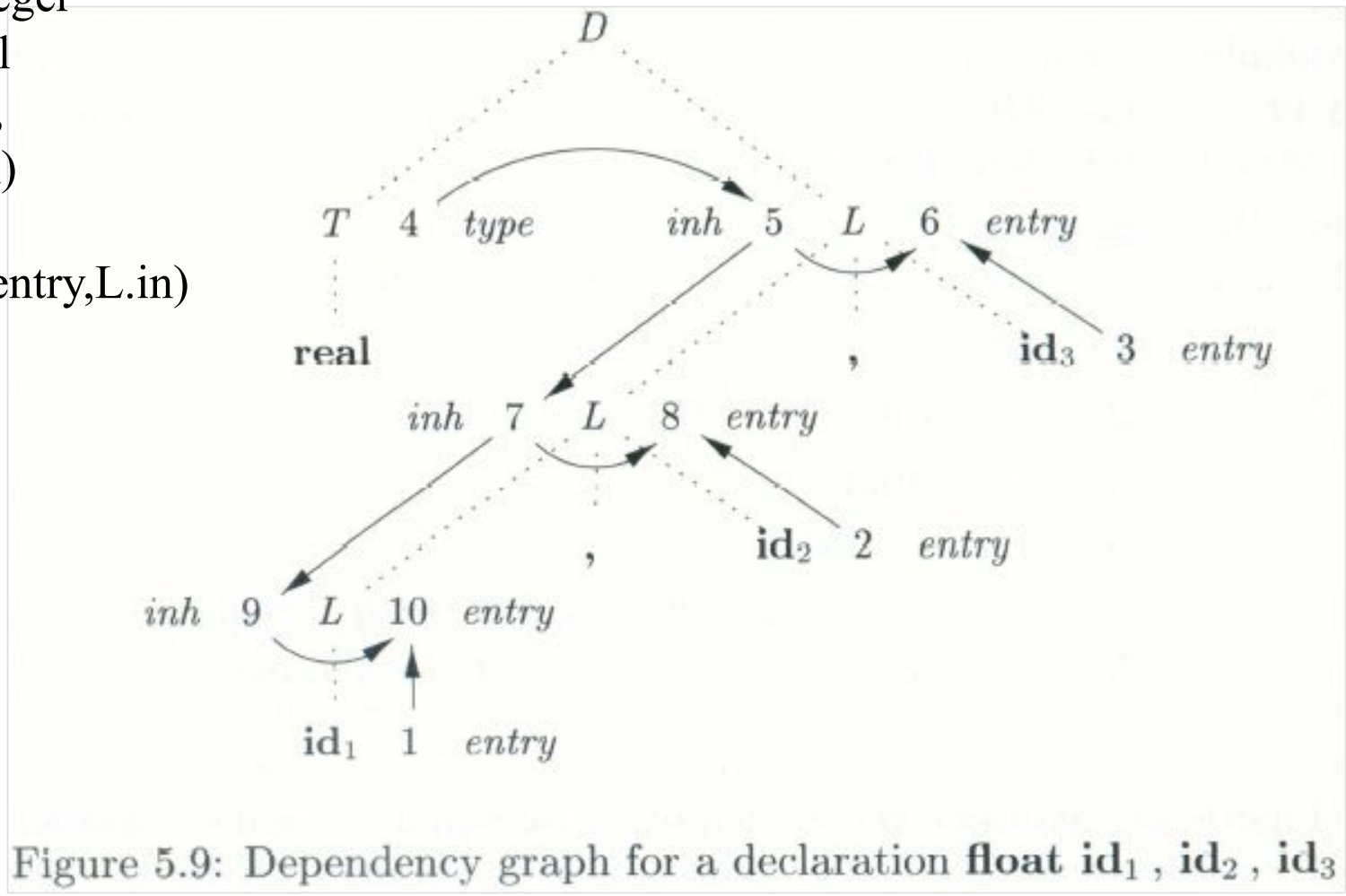


Figure 5.9: Dependency graph for a declaration **float** $\mathbf{id}_1$, $\mathbf{id}_2$, $\mathbf{id}_3$

PARSE TREE vs DEPENDENCY GRAPH

Same sentence... Different stories! 😂

I show STRUCTURE, not gossip!

I show WHO DEPENDS on WHO!

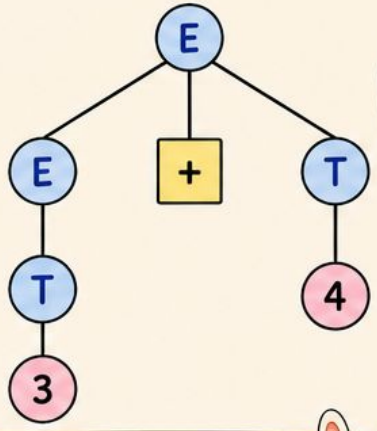
EXPRESSION: **3 + 4**

1. PARSE TREE

The "Family Tree" of the sentence

I only care about GRAMMAR RELATIONSHIPS. I don't care who talks to who!





We just follow the grammar rules. That's all! 😎

What it tells you:

- ✓ Who is parent of whom
- ✓ What rule was used
- ✓ Sentence structure



Yay! I know the structure!

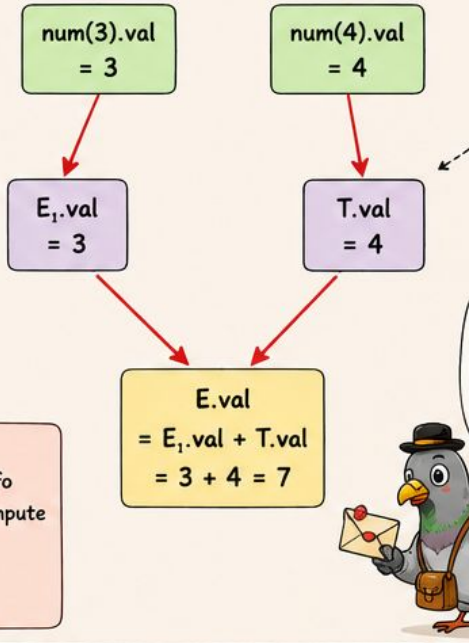
VS

2. DEPENDENCY GRAPH

The "Whisper Network" of attributes

I don't care about family. I care about INFORMATION FLOW!





Boss E.val can be computed ONLY after I get values from both sides!

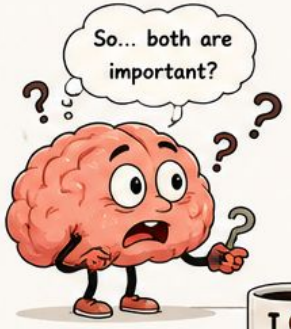


What it tells you:

- ✓ Who needs whose info
- ✓ In what order to compute
- ✓ Can detect circular dependencies
- ✓ Evaluation strategy

QUICK COMPARISON (Like comparing WhatsApp Group vs Email Chain 😅)

Feature	PARSE TREE	DEPENDENCY GRAPH	Funny Analogy
Purpose	Shows syntactic structure	Shows attribute dependencies	Family Photo vs WhatsApp Chat
Nodes	Grammar symbols	Attributes	People vs Messages
Edges	Parent-child (structure)	Information flow (who needs what)	Family relation vs Who sends gossip to whom
Built during	Parsing	Semantic analysis	Making family tree vs Spying on conversations
Detects cycles?	No ❌	Yes ✅	Family tree can't show drama, But gossip network reveals love triangles!





Exactly! One shows THE MAP, The other shows THE TRAFFIC!





Parse Tree says: "This is the structure." Dependency Graph says: "This is the plan."
Compiler says: "Thanks guys, now let's get the job done!" 😂


DO YOU THINK THAT A DEPENDENCY GRAPH SHOULD BE ACYCLIC?
PLEASE DO NOT ARGUE WITH THE CURATOR ABOUT COLOR PALETTES.

DEPENDENCY GRAPH IN COMPILER:

Acyclic is Good 😇, Cyclic is Bad 😈



I just want an order... not drama! 😂

No cycles, please. My head hurts!



1. ACYCLIC DEPENDENCY GRAPH ✅

Everyone knows whose report comes first!

```
A.x = B.y + 1
B.y = C.z + 1
C.z = 5
```



Dependency Graph



No cycles here! We can compute in this order: C.z → B.y → A.x

Life is beautiful 🌈

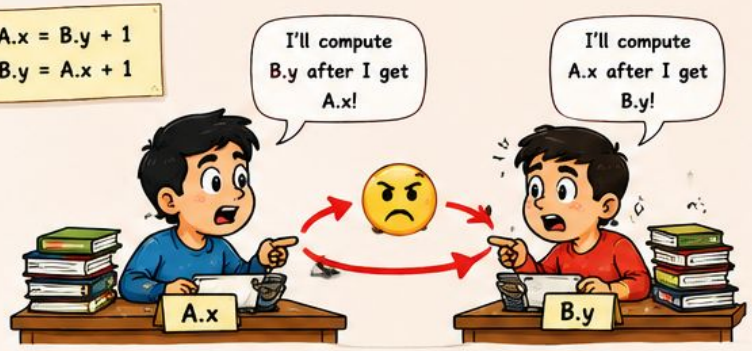
Compiler says:

- ✅ Evaluation order exists
- ✅ Everyone is happy
- ✅ Program runs
- ✅ Coffee break! ☕

2. CYCLIC DEPENDENCY GRAPH ❌

The famous "You first... No, you first!" problem!

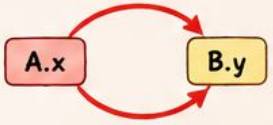
```
A.x = B.y + 1
B.y = A.x + 1
```



DEADLOCK DETECTED!

Mission Impossible!

Dependency Graph



Compiler says:

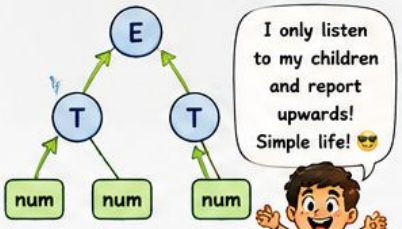
- ❌ No evaluation order
- ❌ Circular dependency
- ❌ Semantic Error
- ❌ Go and think again!



3. COMMON GRAMMAR TYPES (KEEP IT SIMPLE, STUPID! 😜)

S-ATTRIBUTED GRAMMAR

Only synthesized attributes
(Information flows UP)

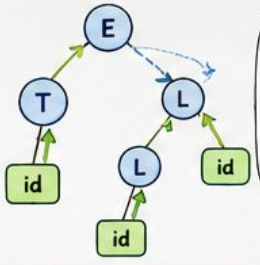


I only listen to my children and report upwards! Simple life! 😎



L-ATTRIBUTED GRAMMAR

Inherited (down/side) + Synthesized (up)
(Information flows in a disciplined way)



I pass info down & sideways like a good team player! Discipline is my superpower! 💪



Moral of the story: Acyclic = Smooth Execution 🚀 Cyclic = Infinite Waiting ⏳

Compiler wants peace, not a never-ending "Who-first" debate! 😂

4. REAL LIFE ANALOGY 😂

ACYCLIC (GOOD)

Orderly project meeting



Finish → Finish → Finish
Project Success! 🏆

CYCLIC (BAD)

Group project from hell



Result: Nothing! Zip! Zero! 😭

Deadline Missed. Grade Missed. Life Missed. 💀

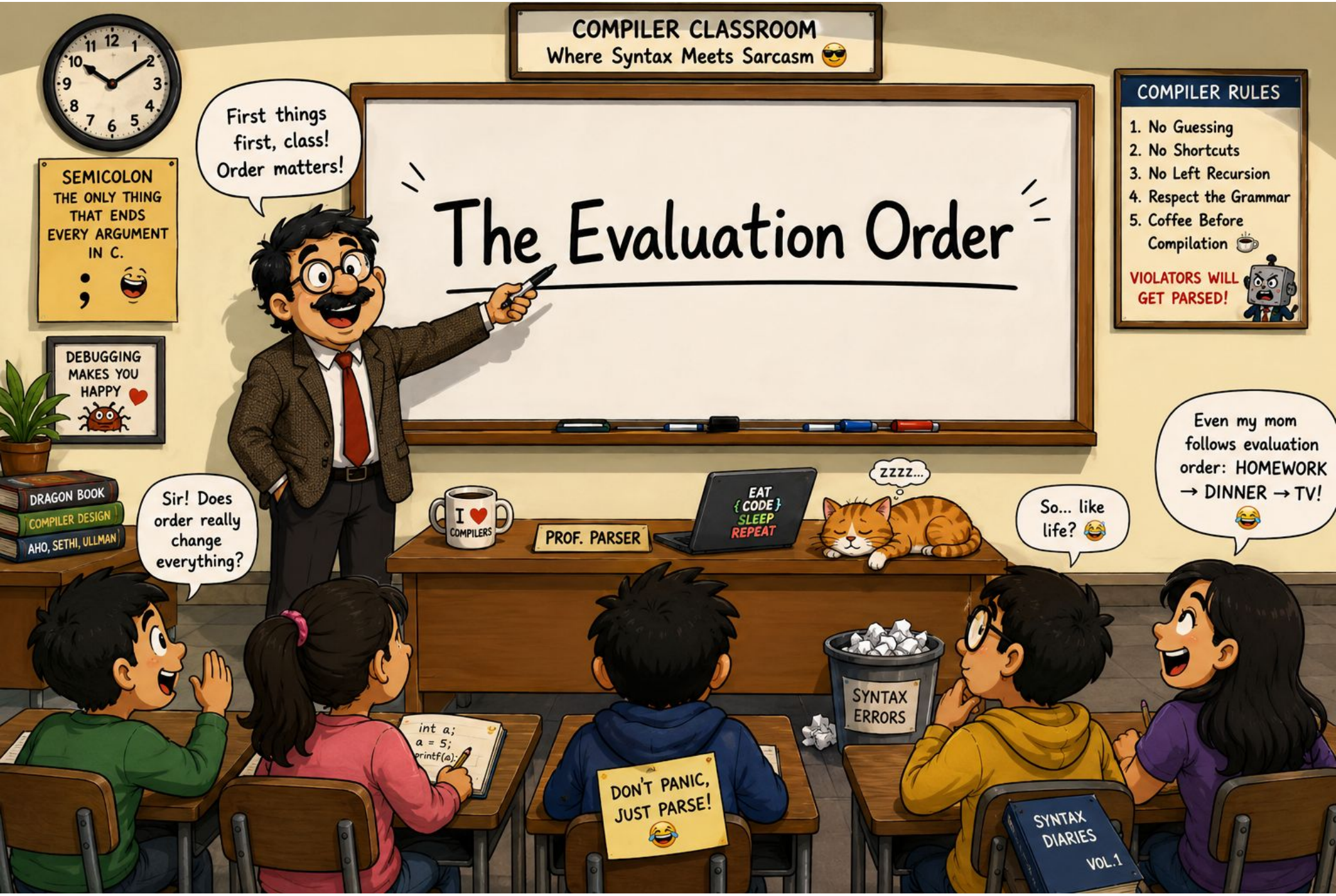


Remember:

If you see a cycle in a dependency graph, don't panic... Don't fight... Don't blame others... Just fix it! 😎

Compilers are strict, but fair. 😊




COMPILER CLASSROOM
Where Syntax Meets Sarcasm
The Evaluation Order
First things first, class! Order matters!
COMPILER RULES
1. No Guessing
2. No Shortcuts
3. No Left Recursion
4. Respect the Grammar
5. Coffee Before Compilation
VIOLATORS WILL GET PARSED!
SEMICOLON
THE ONLY THING THAT ENDS EVERY ARGUMENT IN C.
;
DEBUGGING MAKES YOU HAPPY
DRAGON BOOK
COMPILER DESIGN
AHO, SETHI, ULLMAN
I ♥ COMPILERS
PROF. PARSER
EAT { CODE } SLEEP REPEAT
zzzz...
Sir! Does order really change everything?
So... like life? 😂
Even my mom follows evaluation order: HOMEWORK → DINNER → TV! 😂
int a;
a = 5;
printf(a);
SYNTAX ERRORS
DON'T PANIC, JUST PARSE! 😂
SYNTAX DIARIES VOL.1



WHY EVALUATION ORDER IS MANDATORY!



In a compiler, attributes depend on other attributes.
If we don't follow the right order, we get **CONFUSION**, **WRONG ANSWERS** or **DISASTER!**

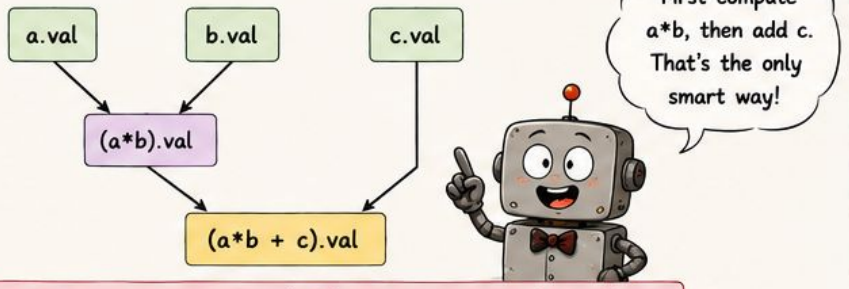
EXAMPLE

Expression: a * b + c

Let's say,

a.val = 5
b.val = 3
c.val = 2

Dependency (Information Flow)



Correct Evaluation Order: a.val, b.val → (a*b).val → c.val → (a*b+c).val

WHAT HAPPENS IF WE IGNORE ORDER? 😱

Suppose we evaluate (a*b+c).val first!



Result: Wrong answer, garbage value, or even compiler crash!

REAL LIFE ANALOGY

1. Making Tea ☕

Boil water → Add tea leaves → Add milk → Add sugar



2. Wrong Order

Add sugar first → then water → then tea leaves → then milk



3. Cooking 🍲

Wash rice → Soak → Cook → Add spices



4. Wrong Order

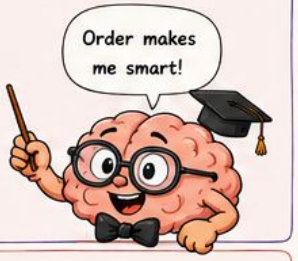
Cook first → Then wash rice → Then add spices



Moral: Right order = Success Wrong order = Stress! 😎

IN COMPILER TERMS

- ✅ Attributes depend on other attributes.
- ✅ Evaluation order shows which one to compute first.
- ✅ Eliminates ambiguity.
- ✅ Prevents use of undefined values.
- ✅ Ensures correct and consistent results.
- ✅ Helps in detecting circular dependencies.



WHAT IF NO EVALUATION ORDER?

- ❌ Undefined or garbage values
- ❌ Incorrect program behavior
- ❌ Semantic errors
- ❌ Compiler may reject the program
- ❌ Life becomes harder 😅



KEY TAKEAWAY

Evaluation order is not a choice, it is a MUST!

It turns dependencies into correct results! 😄

SUMMARY TABLE

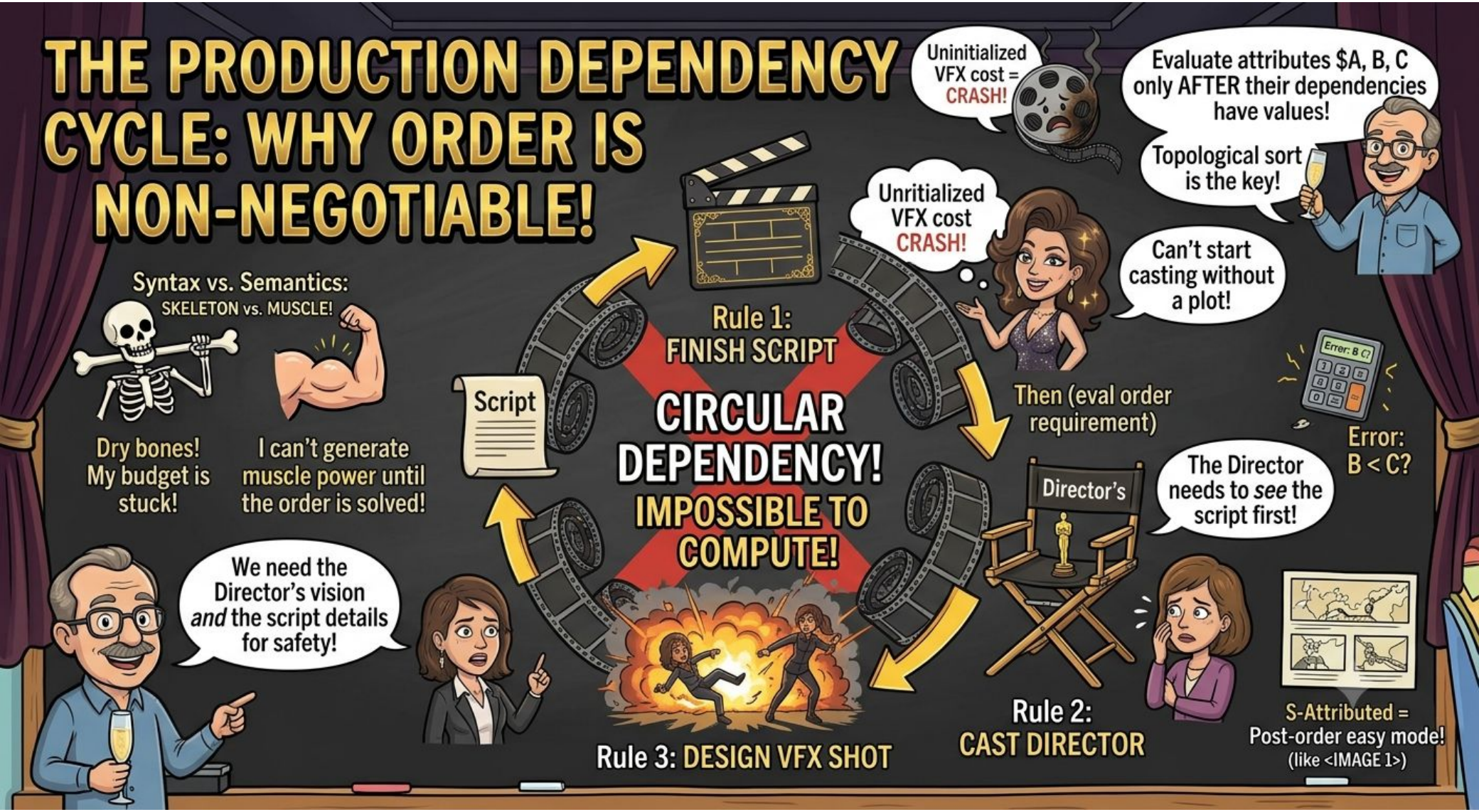
Scenario	Order Followed?	Result	Mood
Correct evaluation order	✅	Correct value	😄 Happy Compiler
Wrong evaluation order	❌	Wrong/garbage value	😵 Confused Compiler
Cyclic dependency	❌	No order possible	💀 Dead Compiler



BONUS JOKE 😹

Compiler's favorite quote:
"First things first, or first things worst!" 😂




THE PRODUCTION DEPENDENCY CYCLE: WHY ORDER IS NON-NEGOTIABLE!
Uninitialized VFX cost = CRASH!
Evaluate attributes $A, B, C only AFTER their dependencies have values!
Topological sort is the key!
Unritialized VFX cost CRASH!
Can't start casting without a plot!
Syntax vs. Semantics:
SKELETON vs. MUSCLE!
Rule 1: FINISH SCRIPT
Script
Then (eval order requirement)
Error: B < C?
Dry bones! My budget is stuck!
I can't generate muscle power until the order is solved!
CIRCULAR DEPENDENCY!
IMPOSSIBLE TO COMPUTE!
Director's
The Director needs to see the script first!
We need the Director's vision and the script details for safety!
Rule 2: CAST DIRECTOR
S-Attributed = Post-order easy mode! (like <IMAGE 1>)
Rule 3: DESIGN VFX SHOT

Evaluation Order

- A topological sort of a directed acyclic graph is any ordering m1,m2…mk of the nodes of the graph such that edges go from nodes earlier in the ordering to later nodes.

  . i.e if there is an edge from m_i to m_j them m_i appears before m_j in the ordering
- Any topological sort of dependency graph gives a valid order for evaluation of semantic rules associated with the nodes of the parse tree.
  - The dependent attributes c1,c2….ck in b=f(c1,c2….ck) must be available before f is evaluated.

- Translation specified by Syntax Directed Definition

- Input string ⟶ parse tree ⟶ dependency graph ⟶ evaluation order for semantic rules

Evaluation Order

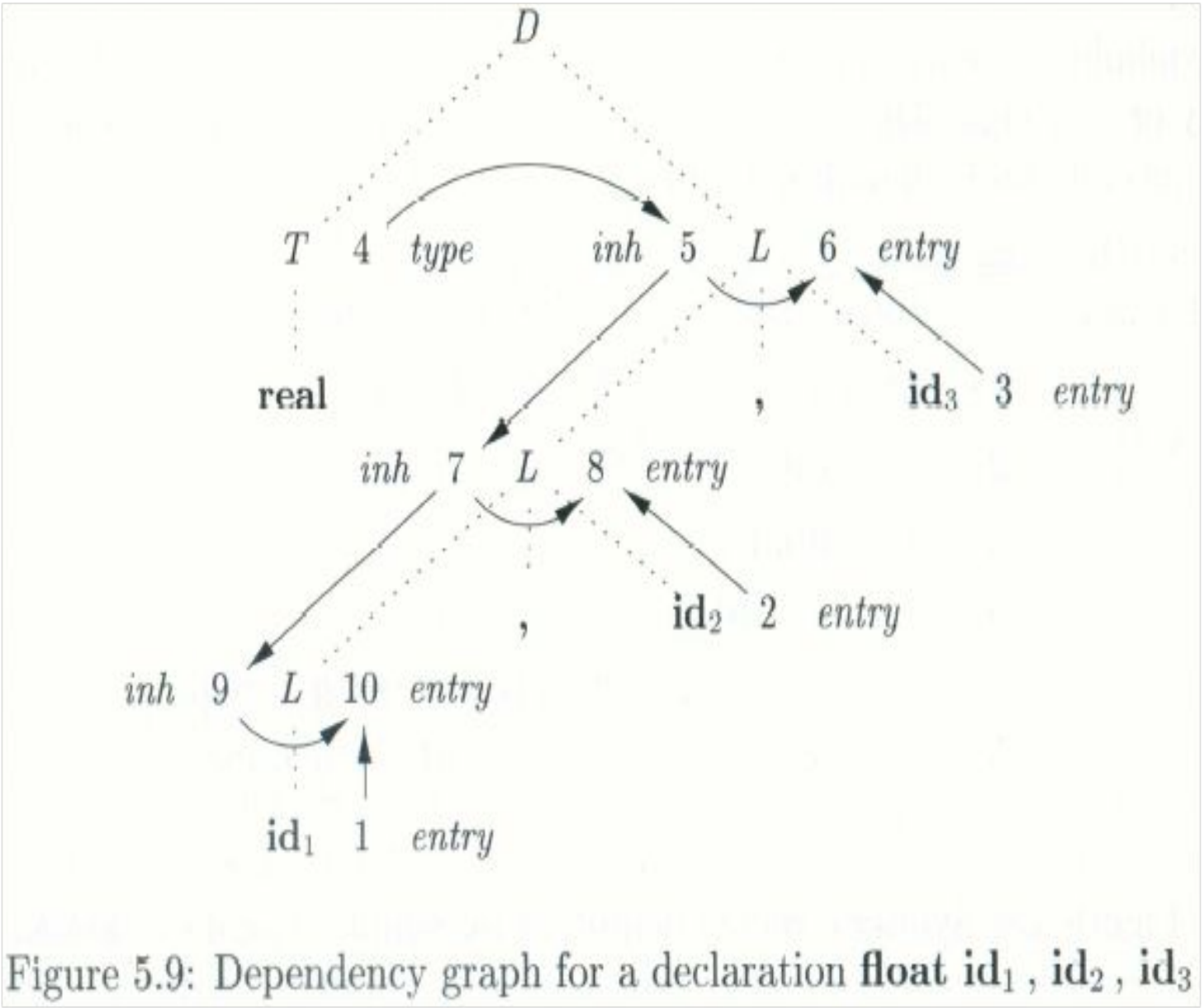


Figure 5.9: Dependency graph for a declaration **float** $\mathbf{id}_1$, $\mathbf{id}_2$, $\mathbf{id}_3$

- a4=real;
- a5=a4;
- addtype(id3.entry,a5);
- a7=a5;
- addtype(id2.entry,a7);
- a9=a7;
- addtype(id1.entry,a5);

Evaluating Semantic Rules

- Parse Tree methods
  - At compile time evaluation order obtained from the topological sort of dependency graph.
  - Fails if dependency graph has a cycle
- Rule Based Methods
  - Semantic rules analyzed by hand or specialized tools at compiler construction time
  - Order of evaluation of attributes associated with a production is pre-determined at compiler construction time
- Oblivious Methods
  - Evaluation order is chosen without considering the semantic rules.
  - Restricts the class of syntax directed definitions that can be implemented.
  - If translation takes place during parsing order of evaluation is forced by parsing method.

METHODS OF EVALUATING SEMANTIC RULES: BEYOND THE PARSE TREE!

METHOD: PURE SYNTAX-DIRECTED TRAVERSAL

1. PARSE TRAVERSAL METHODS

POST-ORDER PASS

O.total

Base

Cheese

None

Large

Cheese

None

Evaluates attributes $A, B, C only AFTER their dependencies have values!

I feel... structured!

Ready for muscle power!

Ideal for S-Attributed & L-Attributed (Left-to-Right older sibling guidance) SDDs!

METHOD: DEMAND-DRIVEN RULE EVALUATION

DEMAND-DRIVEN METHODS

Attributes *only* computed until explicitly "demanded"

O.total

Demand

Needs Base Price?

Funny label

Needs liness:

Base

Cheese

Price

Total Depends on Price?

Needs Toppings Extra?

Needs Depends Price?

Extra?

Large

Cheese

None

We can't start casting without a plot! Give me the specific dependency values!

Computes *only* what is necessary on demand. Very efficient!

METHOD: OBLIVIOUS (GRAPH-BASED) METHODS

IMPOSSIBLE TO COMPUTE!

Builds *full* dependency graph, finds dependencty graph, finds TOPOLOGICAL SORT. Guarantees no cycles!

TOPOLOGICAL SORT! THE KEY!

O.total

A, B

B, C

A, C

None

Base

Cheese

Synthesizer

None

I can't generate muscle power until the general order is solved!

Handles arbitrary dependency structures. Can solve *any* valid SDD!

Synthesizer Beat!

CRASH!

Warning: Avoid general methods on simple L-Attributed SDDs!

Syntax Trees

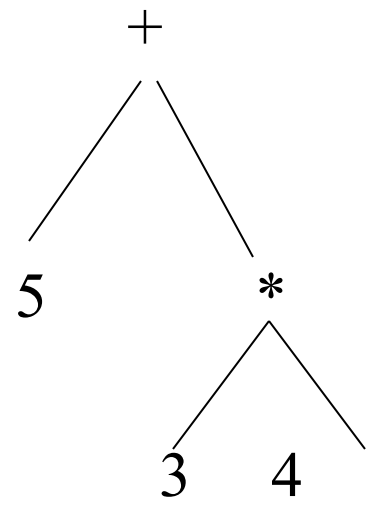
Syntax-Tree

- an intermediate representation of the compiler's input.
- A condensed form of the parse tree.
- Syntax tree shows the syntactic structure of the program while omitting irrelevant details.
- Operators and keywords are associated with the interior nodes.
- Chains of simple productions are collapsed.

Syntax directed translation can be based on syntax tree as well as parse tree.

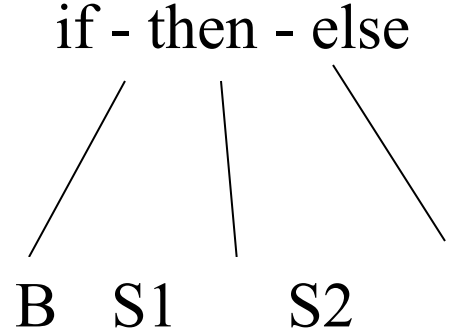
Syntax Tree-Examples

Expression:



- Leaves: identifiers or constants
- Internal nodes: labelled with operations
- Children: of a node are its operands

if B then S1 else S2



Statement:

- Node's label indicates what kind of a statement it is
- Children of a node correspond to the components of the statement

Constructing Syntax Tree for Expressions

- Each node can be implemented as a record with several fields.
- Operator node: one field identifies the operator (called *label of the node*) and remaining fields contain pointers to operands.
- The nodes may also contain fields to hold the values (pointers to values) of attributes attached to the nodes.

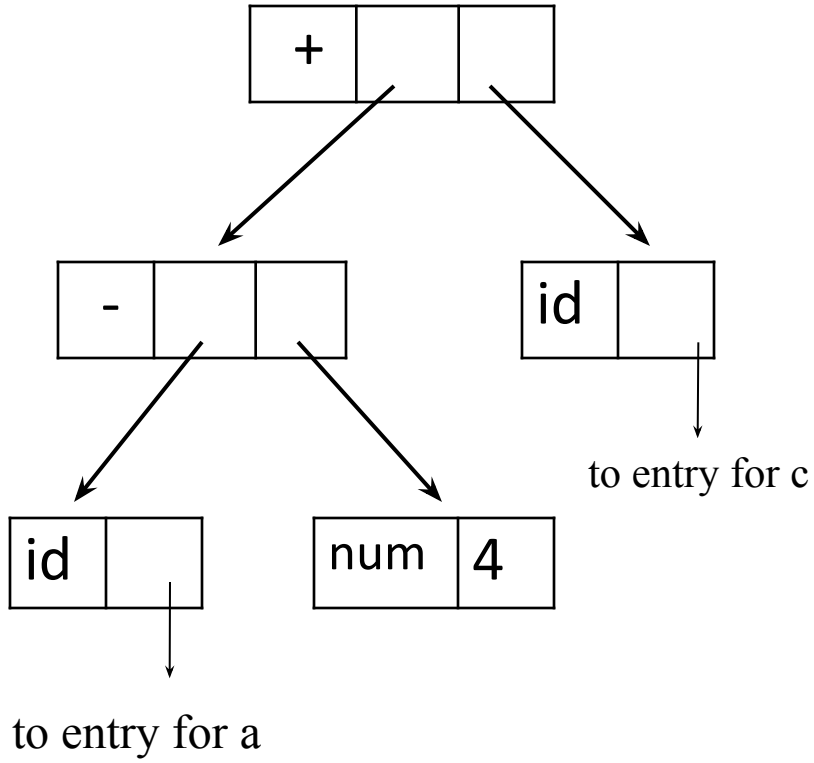
- Functions used to create nodes of syntax tree for expressions with binary operator are given below.
  - mknode(op,left,right)
  - mkleaf(id,entry)
  - mkleaf(num,val)

Each function returns a pointer to a newly created node.

Constructing Syntax Tree for Expressions-

Example: a-4+c

1. p1:=mkleaf(id,entrya);
2. p2:=mkleaf(num,4);
3. p3:=mknode(-,p1,p2)
4. p4:=mkleaf(id,entryc);
5. p5:= mknode(+,p3,p4);



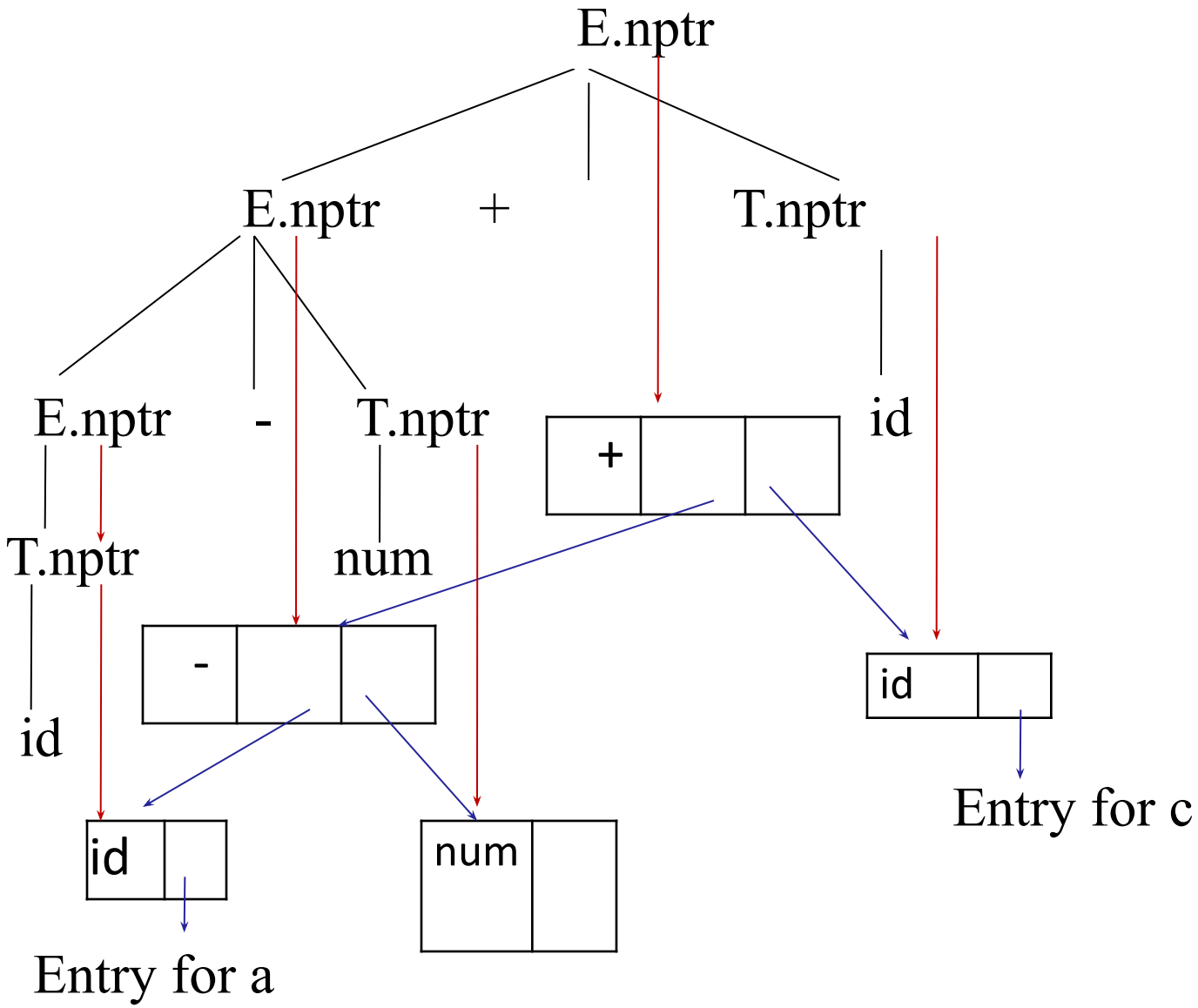
- **The tree is constructed bottom up.**

A syntax Directed Definition for Constructing Syntax Tree

1. It uses underlying productions of the grammar to schedule the calls of the functions *mkleaf* and *mknode* to construct the syntax tree
2. Employment of the synthesized attribute *nptr* (pointer) for E and T to keep track of the pointers returned by the function calls.

PRODUCTION	**SEMANTIC RULE**
$E \rightarrow E_1 + T$	E.*nptr* = *mknode*("+",E_1.*nptr* ,T.*nptr*)
$E \rightarrow E_1 - T$	E.*nptr* = *mknode*("-",E_1.*nptr* ,T.*nptr*)
$E \rightarrow T$	E.*nptr* = T.*nptr*
$T \rightarrow (E)$	T.*nptr* = E.*nptr*
$T \rightarrow id$	T.*nptr* = *mkleaf*(id, id.*lexval*)
$T \rightarrow num$	T.*nptr* = *mkleaf*(num, num.*val*)

Annotated parse tree depicting construction of syntax tree for the expression a-4+c



S-Attributed Definitions

1. Syntax-directed definitions are used to specify syntax-directed translations.

2. To create a translator for an arbitrary syntax-directed definition can be difficult.

3. We would like to evaluate the semantic rules during parsing (i.e. in a single pass, we will parse and we will also evaluate semantic rules during the parsing).

4. We will look at two sub-classes of the syntax-directed definitions:
   - **S-Attributed Definitions**: only synthesized attributes used in the syntax-directed definitions.
   - All actions occur on the right hand side of the production.
   - **L-Attributed Definitions**: in addition to synthesized attributes, we may also use inherited attributes in a restricted fashion.

5. To implement S-Attributed Definitions and L-Attributed Definitions we can evaluate semantic rules in a single pass during the parsing.

6. Implementations of S-attributed Definitions are a little bit easier than implementations of L-Attributed Definitions

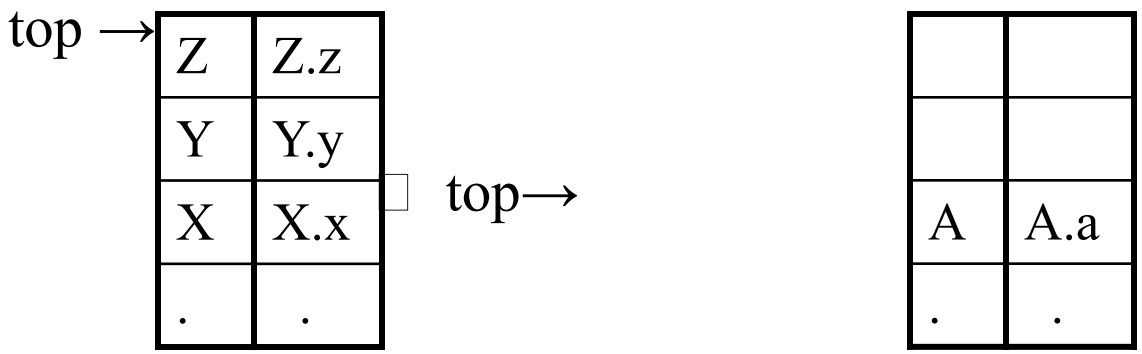
Bottom-Up Evaluation of S-Attributed Definitions

- A translator for an S-attributed definition can often be implemented with the help of an LR parser.
- From an S-attributed definition the parser generator can construct a translator that evaluates attributes as it parses the input.
- We put the values of the synthesized attributes of the grammar symbols a stack that has extra fields to hold the values of attributes.
  - The stack is implemented by a pair of arrays *val & state*
  - If the ith state symbol is A the val[i] will hold the value of the attribute associated with the parse tree node corresponding to this A.

Bottom-Up Evaluation of S-Attributed Definitions

- We evaluate the values of the attributes during reductions.

$A \rightarrow XYZ$ $\quad A.a=f(X.x,Y.y,Z.z)$ where all attributes are synthesized.



- Synthesized attributes are evaluated before each reduction.
- Before XYZ is reduced to A, the value of Z.z is in val[top], that of Y.y in val[top-1] and that of X.x in val[top-2].
- After reduction top is decremented by 2.
- If a symbol has no attribute the corresponding entry in the array is undefined.

Bottom-Up Evaluation of S-Attributed Definitions

Production	**Semantic Rules**
$L \rightarrow E$ **n**	print(val[top-1])
$E \rightarrow E_1 + T$	val[ntop] = val[top-2] + val[top]
$E \rightarrow T$	
$T \rightarrow T_1 * F$	val[ntop] = val[top-2] * val[top]
$T \rightarrow F$	
$F \rightarrow (E)$	val[ntop] = val[top-1]
$F \rightarrow$ **digit**	

1. At each shift of **digit**, we also push **digit.lexval** into *val-stack*.
2. At all other shifts, we do not put anything into *val-stack* because other terminals do not have attributes (but we increment the stack pointer for *val-stack*).

Bottom-Up Evaluation -- Example

- At each shift of **digit**, we also push **digit.lexval** into *val-stack*.

Input	state	val	semantic rule
5+3*4n	-	-	
+3*4n	5	5	
+3*4n	F	5	F → digit
+3*4n	T	5	T → F
+3*4 n	E	5	E → T
3*4n	E+	5-	
*4 n	E+3	5-3	
*4n	E+F	5-3	F → digit
*4n	E+T	5-3	T → F
4n	E+T*	5-3-	
n	E+T*4	5-3-4	
n	E+T*F	5-3-4	F → digit
n	E+T	5-12	$T \rightarrow T_1 * F$
n	E	17	$E \rightarrow E_1 + T$
	En	17-	L → E n
	L	17	

L-Attributed Definitions

- When translation takes place during parsing, order of evaluation is linked to the order in which the nodes of a parse tree are created by parsing method.
- A natural order can be obtained by applying the procedure *dfvisit* to the root of a parse tree.
- We call this evaluation order *depth first order*.
- L-attributed definition is a class of syntax directed definition whose attributes can always be evaluated in depth first order(L stands for left since attribute information flows from left to right).

```
dfvisit(node n)
{
  for each child m of n, from left to right
 {
 evaluate inherited attributes of m
  dfvisit(m)
}
evaluate synthesized attributes of n
}
```

L-Attributed Definitions

A syntax-directed definition is **L-attributed** if each inherited attribute of X_j, where $1 \leq j \leq n$, on the right side of $A \rightarrow X_1 X_2 ... X_n$ depends only on

1. The attributes of the symbols $X_1,...,X_{j-1}$ to the left of X_j in the production
2. The inherited attribute of A

Every S-attributed definition is L-attributed, since the restrictions apply only to the inherited attributes (not to synthesized attributes).

A Definition which is *not* L-Attributed

Productions Semantic Rules

$A \rightarrow L\ M$ L.in=l(A.i)

M.in=m(L.s)

A.s=f(M.s)

$A \rightarrow Q\ R$ R.in=r(A.in)

Q.in=q(R.s)

A.s=f(Q.s)

This syntax-directed definition is not L-attributed because the semantic rule Q.in=q(R.s) violates the restrictions of L-attributed definitions.

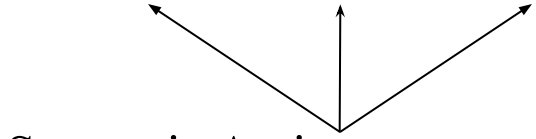
- When Q.in must be evaluated before we enter to Q because it is an inherited attribute.
- But the value of Q.in depends on R.s which will be available after we return from R. So, we are not be able to evaluate the value of Q.in before we enter to Q.

Translation Schemes

- In a syntax-directed definition, we do not say anything about the evaluation times of the semantic rules (when the semantic rules associated with a production should be evaluated).
- Translation schemes describe the order and timing of attribute computation.
- A **translation scheme** is a context-free grammar in which:
  - attributes are associated with the grammar symbols and
  - semantic actions enclosed between braces {} are inserted within the right sides of productions.

Each semantic rule can only use the information compute by already executed semantic rules.

- *Ex:* $A \rightarrow \{ \ldots \}\ X\ \{ \ldots \}\ Y\ \{ \ldots \}$



Semantic Actions

Translation Schemes for S-attributed Definitions

- useful notation for specifying translation during parsing.
- Can have both synthesized and inherited attributes.
- If our syntax-directed definition is S-attributed, the construction of the corresponding translation scheme will be simple.
- Each associated semantic rule in a S-attributed syntax-directed definition will be inserted as a semantic action into the end of the right side of the associated production.

Production Semantic Rule

$E \rightarrow E1 + T$ $E.val = E1.val + T.val$ a production of a syntax directed definition

$\Downarrow$

$E \rightarrow E1 + T \{ E.val = E1.val + T.val \}$ the production of the corresponding translation scheme

A Translation Scheme Example

• A simple translation scheme that converts infix expressions to the corresponding postfix expressions.

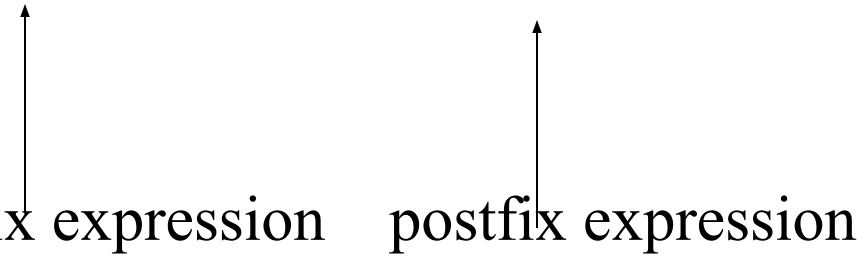
$E \rightarrow T\ R$

$R \rightarrow +\ T\ \{\ print(\text{“+”})\ \}\ R1$

$R \rightarrow \varepsilon$

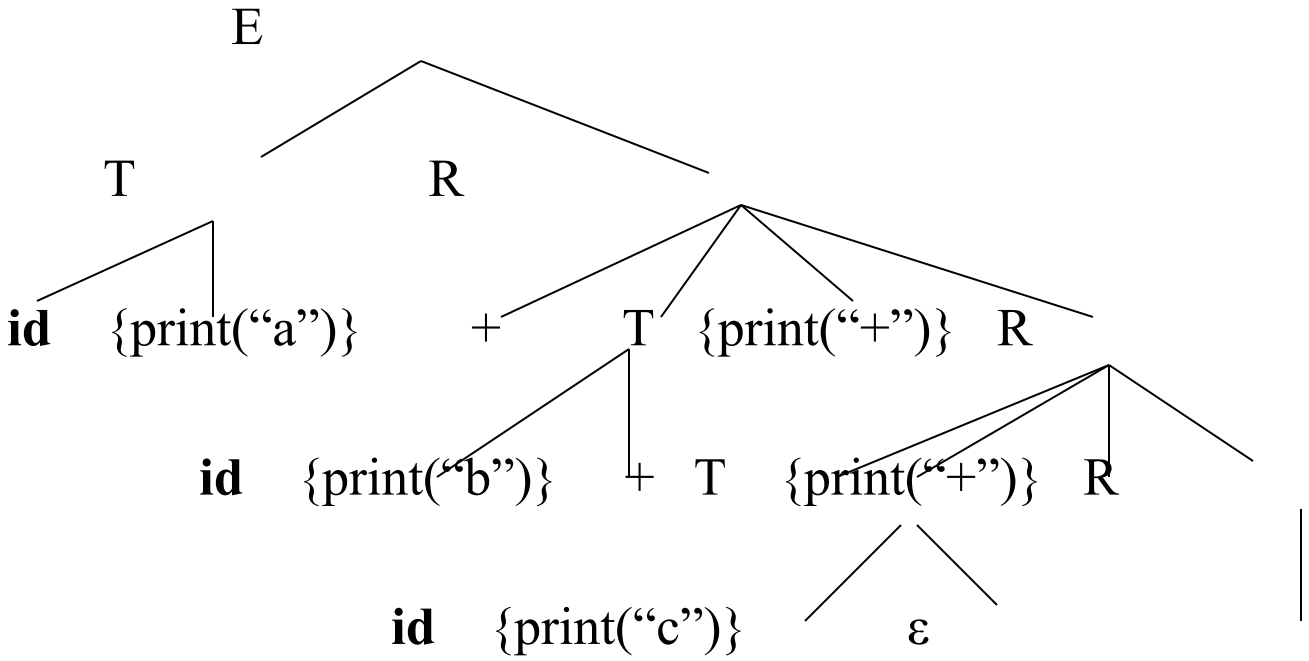
$T \rightarrow \mathbf{id}\ \{\ print(\mathbf{id}.name)\ \}$

a+b+c ab+c+



infix expression postfix expression

A Translation Scheme Example (cont.)



The depth first traversal of the parse tree (executing the semantic actions in that order) will produce the postfix representation of the infix expression.

Inherited Attributes in Translation Schemes

- If a translation scheme has to contain both synthesized and inherited attributes, we have to observe the following rules to ensure that the attribute value is available when an action refers to it.

  1. An inherited attribute of a symbol on the right side of a production must be computed in a semantic action before that symbol.

  2. A semantic action must not refer to a synthesized attribute of a symbol to the right of that semantic action.

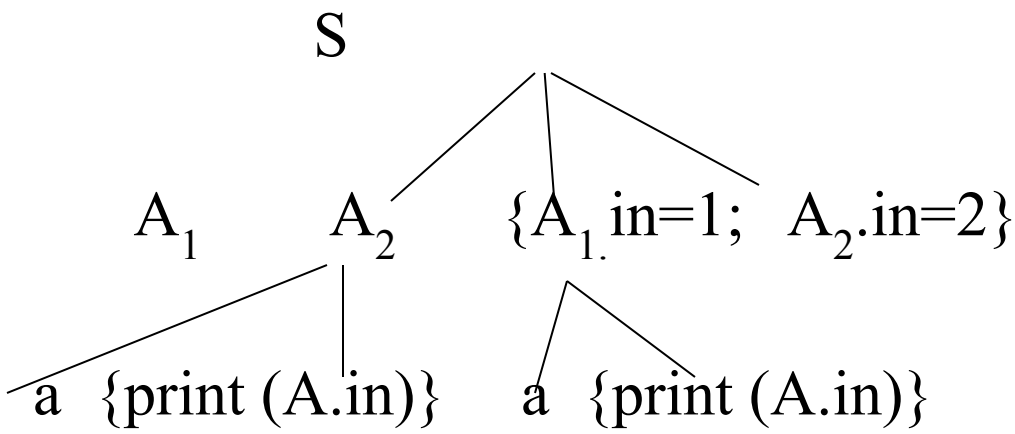
  3. A synthesized attribute for the non-terminal on the left can only be computed after all attributes it references have been computed (we normally put this semantic action at the end of the right side of the production).

- With a L-attributed syntax-directed definition, it is always possible to construct a corresponding translation scheme which satisfies these three conditions (This may not be possible for a general syntax-directed translation).

Inherited Attributes in Translation Schemes: Example

$S \rightarrow A_1 A_2 \quad \{A_1.in=1; \; A_2.in=2\}$

$A \rightarrow a \quad \{ \text{print } (A.in)\}$



A Translation Scheme with Inherited Attributes

D → T {L.in = T.type } L

T → **int** { T.type = integer }

T → **real** { T.type = real }

L → {L1.in = L.in } L1, **id** {addtype(**id**.entry,L.in)}

L → id {addtype(**id**.entry,L.in)}

• This is a translation scheme for an L-attributed definitions

Bottom Up evaluation of Inherited Attributes

- **Removing Embedding Semantic Actions**

In bottom-up evaluation scheme, the semantic actions are evaluated during reductions.

- During the bottom-up evaluation of S-attributed definitions, we have a parallel stack to hold synthesized attributes.
- *Problem*: where are we going to hold inherited attributes?
- *A Solution*:

– We will convert our grammar to an equivalent grammar to guarantee to the followings.

– All embedding semantic actions in our translation scheme will be moved into the end of the production rules.

– All inherited attributes will be copied into the synthesized attributes (most of the time synthesized attributes of new non-terminals).

– Thus we will be evaluate all semantic actions during reductions, and we find a place to store an inherited attribute.

Removing Embedding Semantic Actions

• To transform our translation scheme into an equivalent translation scheme:

1. Remove an embedding semantic action Si, put new a non-terminal Mi instead of that semantic action.
2. Put that semantic action Si into the end of a new production rule Mi→ε for that non-terminal Mi.
3. That semantic action Si will be evaluated when this new production rule is reduced.

Removing Embedding Semantic Actions

A→ {S1} X1 {S2} X2 ... {Sn} Xn

⇓

remove embedding semantic actions

A→ M1 X1 M2 X2 ... Mn Xn

M1→ε {S1}

M2→ε {S2}

.

.

Mn→ε {Sn}

Removing Embedding Semantic Actions

E → T R

R → + T { print(“+”) } R

R → ε

T → **id** { print(**id**.name) }

⇓

remove embedding semantic actions

E → T R

R → + T M R

R → ε

T → **id** { print(**id**.name) }

M → ε { print(“+”)

print(+) }

Inheriting attributes on parser stack

- A bottom up parser reduces the RHS of a production A→XY by removing X and Y from the top of the stack and replacing them by A.
- Suppose X has a synthesized attribute X.s which is already in the stack.
- If the inherited attrtibute Y.i is defined by the copy rule X.s=Y.i, then the value of X.s can where Y.i is called for.
- Copy rule plays an important role in the evaluation of inherited attributes during bottom up parsing.

Productions	**Semantic Rules**
D → T L	
T → **int**	val[ntop]=integer
T → **real**	val[ntop]=real
L → L1, **id**	addtype(val[top],val[top-3])
L → **id**	addtype(val[top],val[top-1])

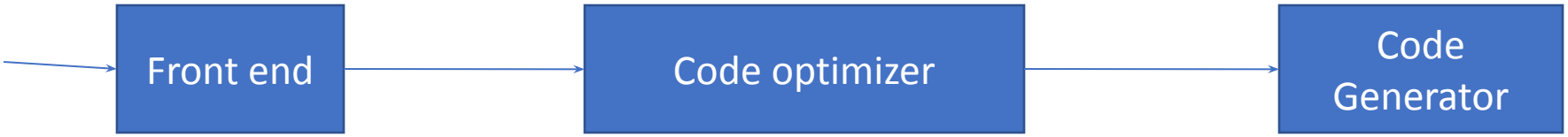
Intermediate Code Generation

Kazy Noor e Alam Siddiquee, PhD

Professor, CSE, MIST

Introduction

- The final phase of a compiler is code generator
- It receives an intermediate representation (IR) with supplementary information in symbol table
- Produces a semantically equivalent target program
- Code generator main tasks:
  - Instruction selection
  - Register allocation and assignment
  - Insrtuction ordering





THE HISTORY OF INTERMEDIATE CODE GENERATION

The Journey from Machine Language to Modern Intermediate Representations



A story of innovation that made compilers portable, optimizable, and powerful.

1940s–1950s — EARLY ERA

Programming was done in Machine Language and Assembly Language.

No intermediate code existed.



1957–1959 — BIRTH OF HIGH-LEVEL LANGUAGES

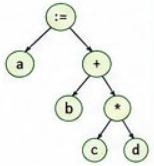
FORTRAN (1957), ALGOL (1958), COBOL (1959) introduced high-level programming.

The same language needed to run on different machines.

FORTRAN | ALGOL | COBOL

LATE 1950s — ALGOL & SYNTAX TREES

ALGOL compilers used formal grammars and parse (syntax) trees. First step toward internal program representations.



1960s — THEORETICAL FOUNDATION

Compiler theory matured. CFGs, parsing algorithms, and syntax-directed translation established the multi-phase compiler model.

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Intermediate Code
- Optimization
- Code Generation

1960s–1970s — THREE-ADDRESS CODE (TAC)

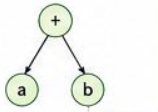
Three-address code became the first widely used intermediate representation.

```
t1 = c * d
t2 = b + t1
a  = t2
```

Easy to optimize and independent of machine.

1970s — DAGS & OPTIMIZATION

Directed Acyclic Graphs (DAGs), Value Numbering, and Common Subexpression Elimination introduced.

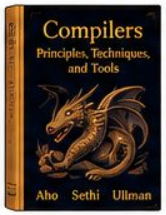


Shared Subexpression (a+b)

Powerful optimizations became possible.

1977–1986 — THE DRAGON BOOK ERA

The famous "Dragon Book" (Aho, Sethi, Ullman) formalized the modern compiler architecture used worldwide.



1980s — P-CODE REVOLUTION

Pascal introduced P-Code. The idea of a machine-independent code executed by a virtual machine became practical.



P-Code Interpreter

1995 — JAVA BYTECODE REVOLUTION

Java bytecode and JVM made "Write Once, Run Anywhere" a reality.



2000s–PRESENT — MODERN COMPILER ERA

Advanced IRs power modern compilers and tools.

- LLVM IR
- GIMPLE (GCC)
- Java Bytecode
- .NET IL (CIL)
- Rust MIR
- Swift SIL

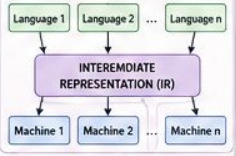
THE CORE MOTIVATION

Without intermediate code, the number of compilers needed grows explosively!

WITHOUT IR — Languages × Machines

	M1	M2	…	Mn
L1	×	×		×
L2	×	×		×
⋮	×	×		×
Ln	×	×		×

WITH IR — One Front-End, Many Back-Ends



> *Separate the understanding of the program from the details of a particular computer architecture.*

BENEFITS OF INTERMEDIATE CODE

- Machine independence and portability
- Enables powerful optimizations
- Simplifies code generation
- Reduces compiler complexity
- Supports retargeting to new architectures



FORMS OF INTERMEDIATE REPRESENTATION (IR)

1. THREE-ADDRESS CODE (TAC)

```
t1 = c * d
t2 = b + t1
a  = t2
```

Simple, linear, optimizable.

2. QUADRUPLES

Op	Arg1	Arg2	Result
*	c	d	t1
+	b	t1	t2
=	t2	–	a

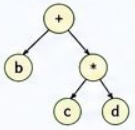
Explicit representation of operations.

3. TRIPLES

Index	Op	Arg1	Arg2
0	*	c	d
1	+	b	(0)
2	=	(1)	a

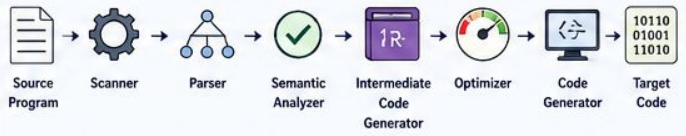
Compact form, uses indices.

4. SYNTAX TREES / DAGS



Shows structure and enables optimizations.

EVOLUTION OF THE COMPILER



Machine Independent → | ← Machine Dependent

TIMELINE SUMMARY



1940s–50s	1957–1959	Late 1950s	1960s	1960s–70s	1970s	1977–1986	1980s	1995	2000s+
Machine & Assembly Only	High-Level Languages	Syntax Trees Emerge	Theory & Compiler Phases	Three-Address Code	DAGs & Optimization	Dragon Book Formalization	P-Code Era	Java Bytecode	Modern IRs & Advanced Compilers

IMPACT & LEGACY

- ★ Enables "Write Once, Run Anywhere"
- ★ Foundation of modern compilers and language tools
- ★ Powers optimization, verification, and analysis tools
- ★ At the heart of software engineering today



> *From machine code to modern IRs, intermediate code generation is the bridge that connects ideas to machines.* — The Legacy Continues

INTERMEDIATE CODE GENERATION

The Bridge Between Source Code and Machine Code

THE CONCEPT

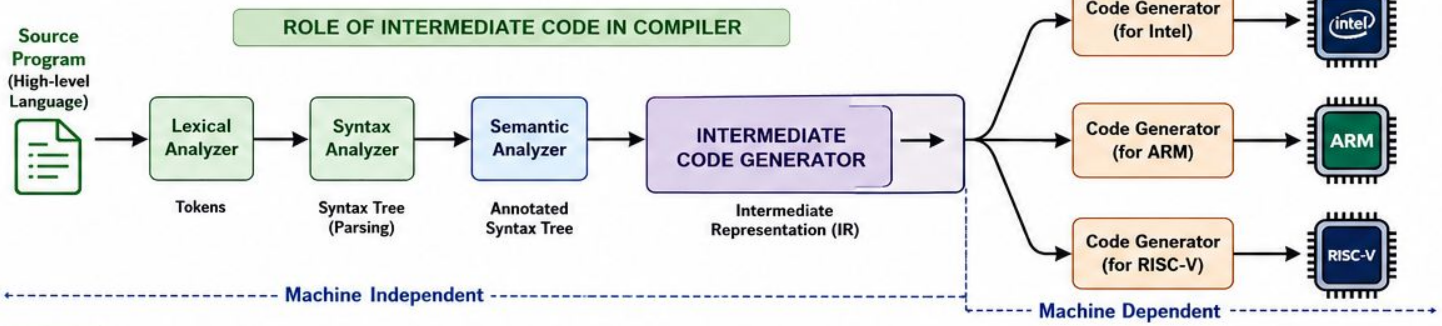
Intermediate Code Generation is the compiler phase that converts the source program into a machine-independent intermediate representation (IR) before generating target machine code.

> *"Separate the understanding of the program from the details of a particular computer architecture."*

WHY DO WE NEED INTERMEDIATE CODE?

1. **Machine Independence**
   One front-end can support many processors.
2. **Easier Optimization**
   Optimizations are performed on IR rather than machine code.
3. **Simplifies Code Generation**
   Complex statements are broken into simple operations.
4. **Better Portability**
   The same IR can be used for different machines and operating systems.
5. **Reduced Compiler Complexity**
   Separates language processing from machine-specific details.

ROLE OF INTERMEDIATE CODE IN COMPILER

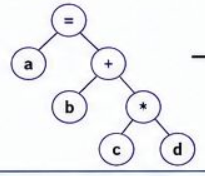


EXAMPLE

Source Statement

```
a = b + c * d;
```

Syntax Tree



Three-Address Code (TAC)

```
t1 = c * d
t2 = b + t1
a  = t2
```

After Optimization (Constant Folding)

```
t1 = c * d
t2 = b + t1
a  = t2
```

Target Code (Example) (for Intel)

```
MOV   EAX, c
IMUL  EAX, d
ADD   EAX, b
MOV   a, EAX
```

FORMS OF INTERMEDIATE CODE

1. THREE-ADDRESS CODE (TAC)

Each instruction has at most three addresses (two operands and one result).

```
t1 = c * d
t2 = b + t1
a  = t2
```

2. QUADRUPLES

Each instruction represented as (op, arg1, arg2, result).

Op	Arg1	Arg2	Result
*	c	d	t1
+	b	t1	t2
=	t2	-	a

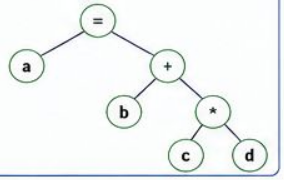
3. TRIPLES

Each instruction represented as (op, arg1, arg2). Results are referred by their index.

Index	Op	Arg1	Arg2
0	*	c	d
1	+	b	(0)
2	=	(1)	a

4. SYNTAX TREE / DAG

Tree/DAG representation shows the structure of computation.



BENEFITS SUMMARY

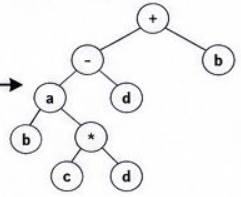
- ✓ Machine independent representation works across multiple architectures.
- ✓ Enables powerful machine-independent optimizations.
- ✓ Simplifies the design of code generator.
- ✓ Improves portability and maintainability of compilers.
- ✓ Reduces development time and cost.

STEP-BY-STEP: HOW INTERMEDIATE CODE IS GENERATED

Source Program

```
a = b + c * d;
e = a - d;
f = e + b;
```

1 Build Syntax Tree (or DAG)



2 Generate TAC

```
t1 = c * d
t2 = b + t1
a  = t2
t3 = a - d
e  = t3
t4 = e + b
f  = t4
```

3 Optimize IR

```
t1 = c * d
t2 = b + t1
a  = t2
t3 = a - d
e  = t3
t4 = e + b
f  = t4
```

- Common subexpr. eliminated
- Copy Propagation
- Dead code elim. (if any)

4 Generate Target Code

```
MOV   EAX, c
IMUL  EAX, d
ADD   EAX, b
MOV   a, EAX
SUB   EAX, d
MOV   e, EAX
ADD   EAX, b
MOV   f, EAX
```

Output

Efficient Machine Code for Target Architecture

KEY TAKEAWAY

Intermediate code is the universal language inside the compiler that makes optimization possible and code generation easier for different target machines.

Issues in the Design of Code Generator

- The most important criterion is that it produces correct code
- Input to the code generator
  - IR + Symbol table
  - We assume front end produces low-level IR, i.e. values of names in it can be directly manipulated by the machine instructions.
  - Syntactic and semantic errors have been already detected
- The target program
  - Common target architectures are: RISC, CISC and Stack based machines
  - In this chapter we use a very simple RISC-like computer with addition of some CISC-like addressing modes

complexity of mapping

- the level of the IR
- the nature of the instruction-set architecture
- the desired quality of the generated code.

x=y+z

```
LD   R0, y
ADD      R0, R0, z
ST   x, R0
```

a=b+c
d=a+e

```
LD   R0, b
ADD      R0, R0, c
ST   a, R0
LD   R0, a
ADD      R0, R0, e
ST   d, R0
```

Register allocation

- Two subproblems
  - Register allocation: selecting the set of variables that will reside in registers at each point in the program
  - Resister assignment: selecting specific register that a variable reside in
- Complications imposed by the hardware architecture
  - Example: register pairs for multiplication and division

```
t=a+b
t=t*c
T=t/d

L     R1, a
A     R1, b
M     R0, c
D     R0, d
ST    R1, t
```

```
t=a+b
t=t+c
T=t/d

L     R0, a
A     R0, b
M     R0, c
SRDA  R0, 32
D     R0, d
ST    R1, t
```

A simple target machine model

- Load operations: LD r,x and LD r1, r2
- Store operations: ST x,r
- Computation operations: OP dst, src1, src2
- Unconditional jumps: BR L
- Conditional jumps: Bcond r, L like BLTZ r, L

Target program for a sample call and return

```
                                                    // code for c
                                100:  ACTION1       // code for action1
          // code for c         120:  ST 364, #140   // save return address 140 in location 364
action1                         132:  BR 200        // call p
call p                          140:  ACTION2
action2                         160:  HALT          // return to operating system
halt                                  ...
                                                     // code for p
          // code for p         200:  ACTION3
action3                         220:  BR *364       // return to address saved in location 364
return                                ...
                                                    // 300-363 hold activation record for c
                                300:                // return address
                                304:                // local data for c
                                      ...
                                                    // 364-451 hold activation record for p
                                364:                // return address
                                368:                // local data for p
```

Stack Allocation

```
LD    SP, #stackStart                    // initialize the stack
code for the first procedure
HALT                                     // terminate execution

ADD   SP, SP, #caller.recordSize         // increment stack pointer
ST    *SP, #here + 16                    // save return address
BR    callee.codeArea                    // return to caller
```

Branch to called procedure

Return to caller

in Callee: BR *0(SP)

in caller: SUB SP, SP, #caller.recordsize

Target code for stack allocation

```
                    // code for m
action1
call q
action2
halt

                    // code for p
action3
return

                    // code for q
action4
call p
action5
call q
action6
call q
return
```

```
                                    // code for m
100:  LD SP, #600                   // initialize the stack
108:  ACTION1                       // code for action1
128:  ADD SP, SP, #msize            // call sequence begins
136:  ST *SP, #152                  // push return address
144:  BR 300                        // call q
152:  SUB SP, SP, #msize            // restore SP
160:  ACTION1 2
180:  HALT
      ...

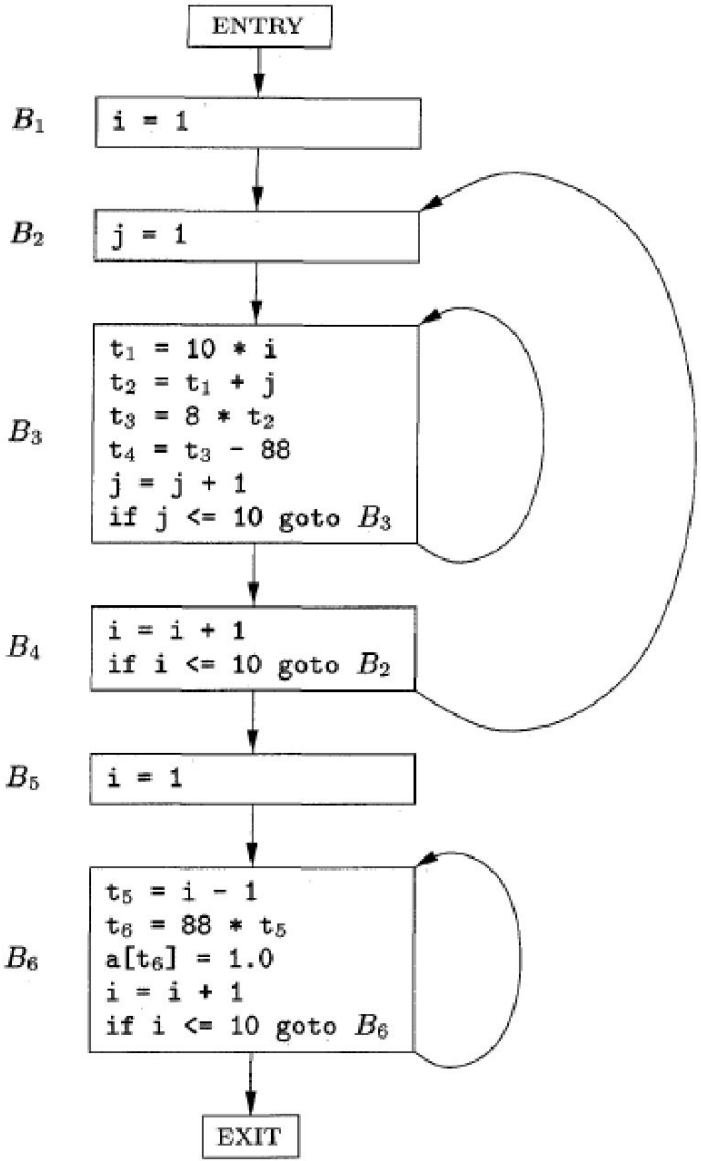
                                    // code for p
200:  ACTION3
220:  BR *0(SP)                     // return
      ...

                                    // code for q
300:  ACTION4                       // contains a conditional jump to 456
320:  ADD SP, SP, #qsize
328:  ST *SP, #344                  // push return address
336:  BR 200                        // call p
344:  SUB SP, SP, #qsize
352:  ACTION5
372:  ADD SP, SP, #qsize
380:  BR *SP, #396                  // push return address
388:  BR 300                        // call q
396:  SUB SP, SP, #qsize
404:  ACTION6
424:  ADD SP, SP, #qsize
432:  ST *SP, #440                  // push return address
440:  BR 300                        // call q
448:  SUB SP, SP, #qsize
456:  BR *0(SP)                     // return
      ...
600:                                // stack starts here
```

Basic blocks and flow graphs

- Partition the intermediate code into basic blocks
  - The flow of control can only enter the basic block through the first instruction in the block. That is, there are no jumps into the middle of the block.
  - Control will leave the block without halting or branching, except possibly at the last instruction in the block.
- The basic blocks become the nodes of a flow graph

Flow graph based on Basic Blocks



DAG representation of basic blocks

- There is a node in the DAG for each of the initial values of the variables appearing in the basic block.
- There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s, of the operands used by s.
- Node N is labeled by the operator applied at s, and also attached to N is the list of variables for which it is the last definition within the block.
- Certain nodes are designated *output nodes.* These are the nodes whose variables are *live on exit* from the block.

VALUE-NUMBER METHOD FOR CONSTRUCTING DAGs

Value-number method assigns a unique number to every variable and expression (value). If the same expression is encountered again (same operator and same operand value numbers), the existing DAG node is reused. This exposes common subexpressions and saves computations.

BASIC IDEA

- Assign value numbers to variables and expressions.
- Maintain a table of expressions with their value numbers.
- For a new expression:
  - If an identical entry exists, reuse that node.
  - Otherwise create a new node, assign a new value number.
- Attach variable names to the value number of the node representing their value.

EXAMPLE BASIC BLOCK

1. a = b + c;
2. d = b + c;
3. e = a * d;
4. f = b + c;
5. g = e - f;

STEP–BY–STEP CONSTRUCTION USING VALUE NUMBERS

Step	Statement	Expression Processed	Lookup in Table	Action	New VN	Attach (Variables)	DAG Node / Comment
1	*a = b + c*	b + c	(+, VN(b), VN(c)) = (+,1,2) Not found	Create new node	3	*a*	Node 3 = +(1,2)
2	*d = b + c*	*b + c*	(+,1,2) Found (VN=3)	Reuse node 3	–	*d*	Attach *d* to VN 3
3	*e = a * d*	*a * d*	(*, VN(a), VN(d)) = (*,3,3) Not found	Create new node	4	*e*	Node 4 = *(3,3)
4	*f = b + c*	*b + c*	(+,1,2) Found (VN=3)	Reuse node 3	–	*f*	Attach *f* to VN 3
5	*g = e - f*	e - f	(-, VN(e), VN(f)) = (-,4,3) Not found	Create new node	5	*g*	Node 5 = -(4,3)

DATA STRUCTURES

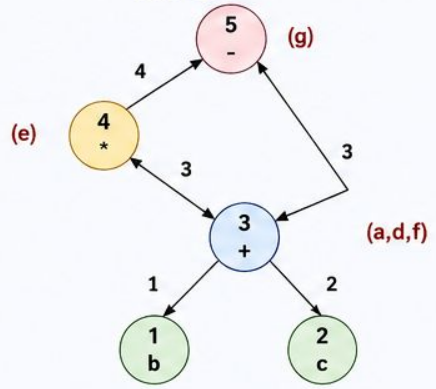
1. VALUE NUMBER TABLE
(Expression Table)
Stores: (Operator, Left VN, Right VN) → VN

VN	Operator	Left VN	Right VN	Expression
1	–	–	–	b
2	–	–	–	c
3	+	1	2	b + c
4	*	3	3	(b + c) * (b + c)
5	-	4	3	(b + c) * (b + c) - (b + c)

2. VARIABLE TABLE (Name Table)
Stores: Variable → Value Number

Variable	Value Number
a	3
d	3
f	3
e	4
g	5

FINAL DAG



EXPLANATION

- Variables *b* and *c* are leaves with value numbers 1 and 2.
- *b + c* is computed once → VN 3. Variables *a*, *d*, and *f* share this node.
- *(b + c) * (b + c)* becomes VN 4. Variable *e* attaches to it.
- *(b + c) * (b + c) - (b + c)* becomes VN 5. Variable *g* attaches to it.
- Only 5 DAG nodes are created instead of 10 separate computations.

VALUE NUMBER ASSIGNMENT RULES

- Each leaf (variable/constant) gets a unique value number.
- For operator *op* with operands having value numbers (*x*, *y*):
  - If *op* is commutative (+, *): store operands in sorted order (min, max) to detect *a+b* and *b+a* as same.
  - Look up (*op*, *x*, *y*) in the expression table.
  - If found → reuse existing node and its value number.
  - Otherwise → create new node and assign new value number.

ADVANTAGES

- ✓ Detects common subexpressions efficiently.
- ✓ Reduces number of computations.
- ✓ Speeds up DAG construction (using hash table).
- ✓ Helps in further optimizations: constant folding, copy propagation, dead code elimination, etc.

APPLICATIONS

Common Subexpression Elimination · Intermediate Code Generation · Code Optimization · Efficient Code Generation

COMPARISON

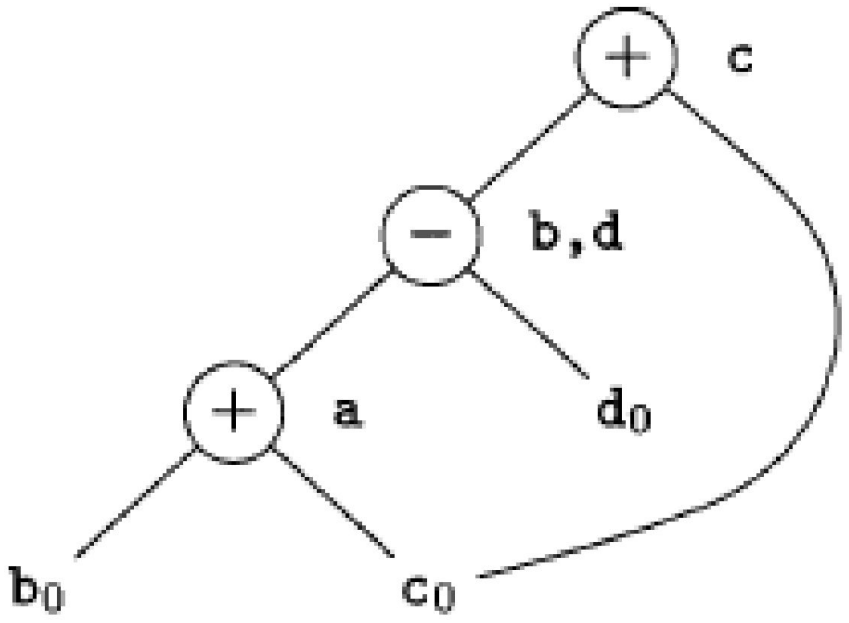
Aspect	Without Value Numbering	With Value Numbering
Search	Traverse whole DAG	Table lookup (hash)
Time Complexity	O(n^2) (approx.)	O(n) average
Nodes Created	More (duplicates)	Fewer (shared)
Memory Usage	Higher	Lower
Optimization	Limited	Better and easier

NOTE

Value numbering is performed within a basic block. New basic block → value numbers restart.

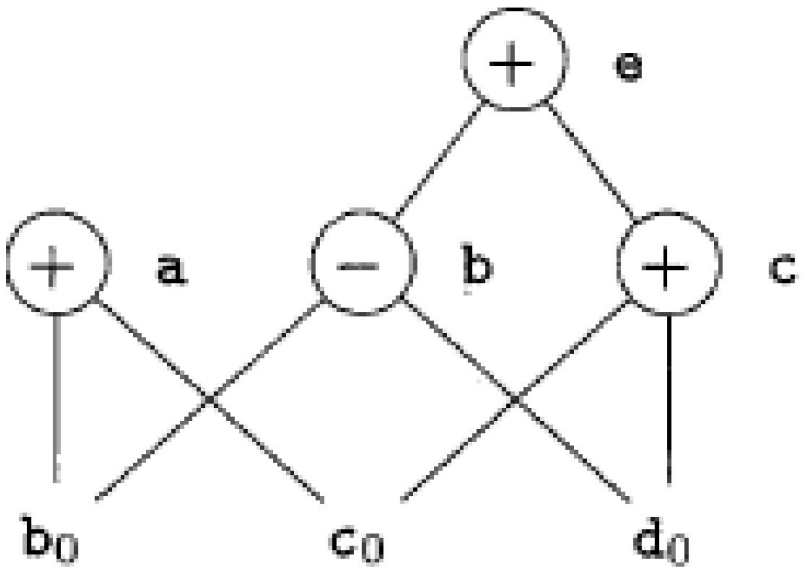
DAG for basic block

```
a = b + c
b = a - d
c = b + c
d = a - d
```



DAG for basic block

```
a = b + c;
b = b - d
c = c + d
e = b + c
```



Rules for reconstructing the basic block from a DAG

- The order of instructions must respect the order of nodes in the DAG. That is, we cannot compute a node's value until we have computed a value for each of its children.
- Assignments to an array must follow all previous assignments to, or evaluations from, the same array, according to the order of these instructions in the original basic block.
- Evaluations of array elements must follow any previous (according to the original block) assignments to the same array. The only permutation allowed is that two evaluations from the same array may be done in either order, as long as neither crosses over an assignment to that array.
- Any use of a variable must follow all previous (according to the original block) procedure calls or indirect assignments through a pointer.
- Any procedure call or indirect assignment through a pointer must follow all previous (according to the original block) evaluations of any variable.

INTERMEDIATE CODE REPRESENTATIONS
THREE-ADDRESS CODE vs QUADRAPOLES vs TRIPLES

Different Ways to Represent the Same Computation

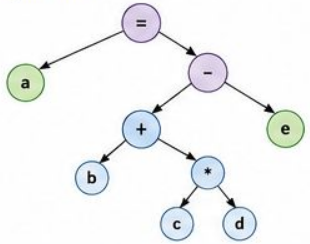
EXAMPLE EXPRESSION

Infix Expression:

```
a = b + c * d - e
```

Assume all variables are in memory.

Its Syntax Tree (for reference):



KEY DIFFERENCES AT A GLANCE

Feature	Three-Address Code	Quadruples	Triples
Representation	Human-readable statements	Record with (op, arg1, arg2, result)	Record with (op, arg1, arg2)
Result Storage	Uses names (temporaries)	Result stored in separate field	Result identified by the position (index)
Reference to Results	By temporary names	By names (temporaries)	By instruction index
Ease for Humans	★★★★★	★★★☆☆	★★☆☆☆
Ease for Machines	★★★★☆	★★★★☆	★★★★☆
Size	More (uses names)	Largest (4 fields per record)	Compact (3 fields per record)
When Used	For readability, understanding, debugging	Easy to implement and modify	Space efficient, used in some optimizers

LEGEND

- `op` : operator
- `arg1` : first argument
- `arg2` : second argument
- `result` : result of the operation

Evaluation Order
(From Syntax Tree)

```
1. t1 = c * d
2. t2 = b + t1
3. t3 = t2 - e
4. a  = t3
```

Note
All three forms represent the same computation and the same evaluation order.

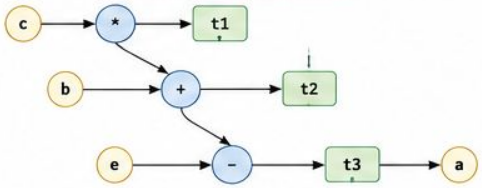
1) THREE-ADDRESS CODE (TAC)

Each statement has at most three addresses.

Statement No.	Three-Address Code
1	`t1 = c * d`
2	`t2 = b + t1`
3	`t3 = t2 - e`
4	`a = t3`

Characteristics
- ✓ Human readable.
- ✓ Uses temporary names (t1, t2, t3).
- ✓ Easy to write, understand and optimize.

Visual Flow (Using Temporaries)



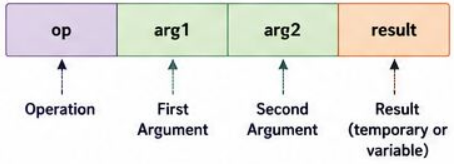
2) QUADRAPOLES

Each instruction is a 4-tuple: (op, arg1, arg2, result).

Index	op	arg1	arg2	result
1	*	c	d	t1
2	+	b	t1	t2
3	-	t2	e	t3
4	=	t3	-	a

Characteristics
- ✓ Explicit result field.
- ✓ Easy to reorder and modify.
- ✓ Widely used in classic compilers.

Record Format



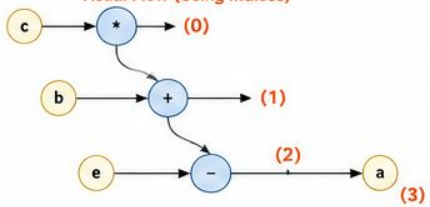
3) TRIPLES

Each instruction is a 3-tuple: (op, arg1, arg2).
The result is the index (position) of the instruction.

Index	op	arg1	arg2
0	*	c	d
1	+	b	(0)
2	-	(1)	e
3	=	(2)	a

Characteristics
- ✓ No explicit result field.
- ✓ Result is referred by the index.
- ✓ More compact.
- ✓ Harder for humans.

Visual Flow (Using Indices)



How it Works?
The result of instruction at index i is referred by (i) in other instructions.

Example:
(0) means "result of instruction at index 0".

SUMMARY

Three-Address Code is best for readability and understanding.
It uses temporary variables.

Quadruples are easy to implement and modify.
They store results explicitly.

Triples are compact and efficient in space.
They refer to results by index.

Common Use
All three forms are used in different compiler phases like optimization and code generation.

Worked example 1: TAC, quadruples, triples, and indirect triples

Translate `a = b * -c + b * -c` from left to right. Unary minus is an operator, so it gets its own instruction each time.

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

Quadruples

This gives 6 instructions and 5 temporaries. A quadruple records `(op, arg1, arg2, result)` :

Row	op	arg1	arg2	result
(0)	minus	c	-	t1
(1)	*	b	t1	t2
(2)	minus	c	-	t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	=	t5	-	a

Triples

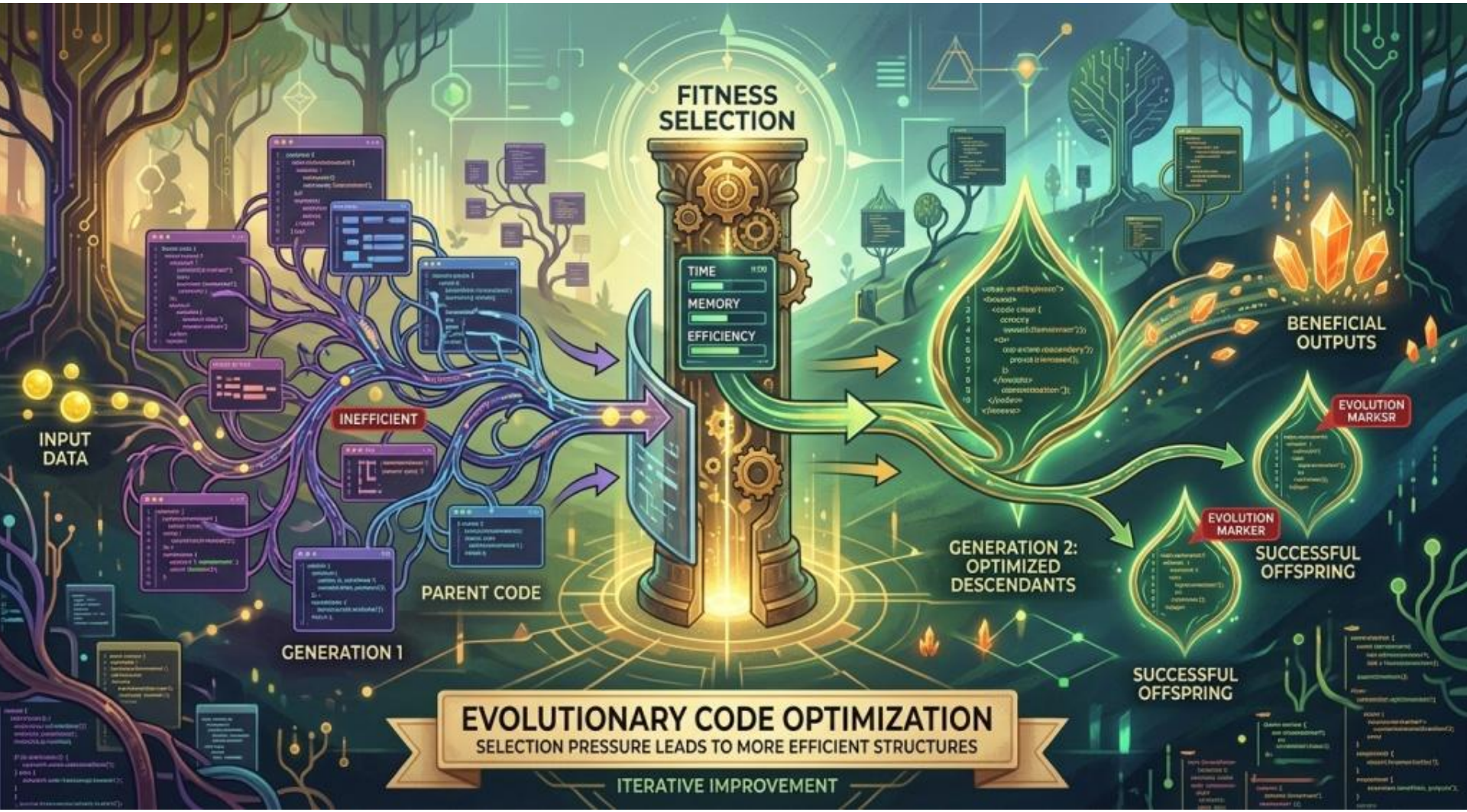
Triples refer to results by row number, so they do not need named temporaries:

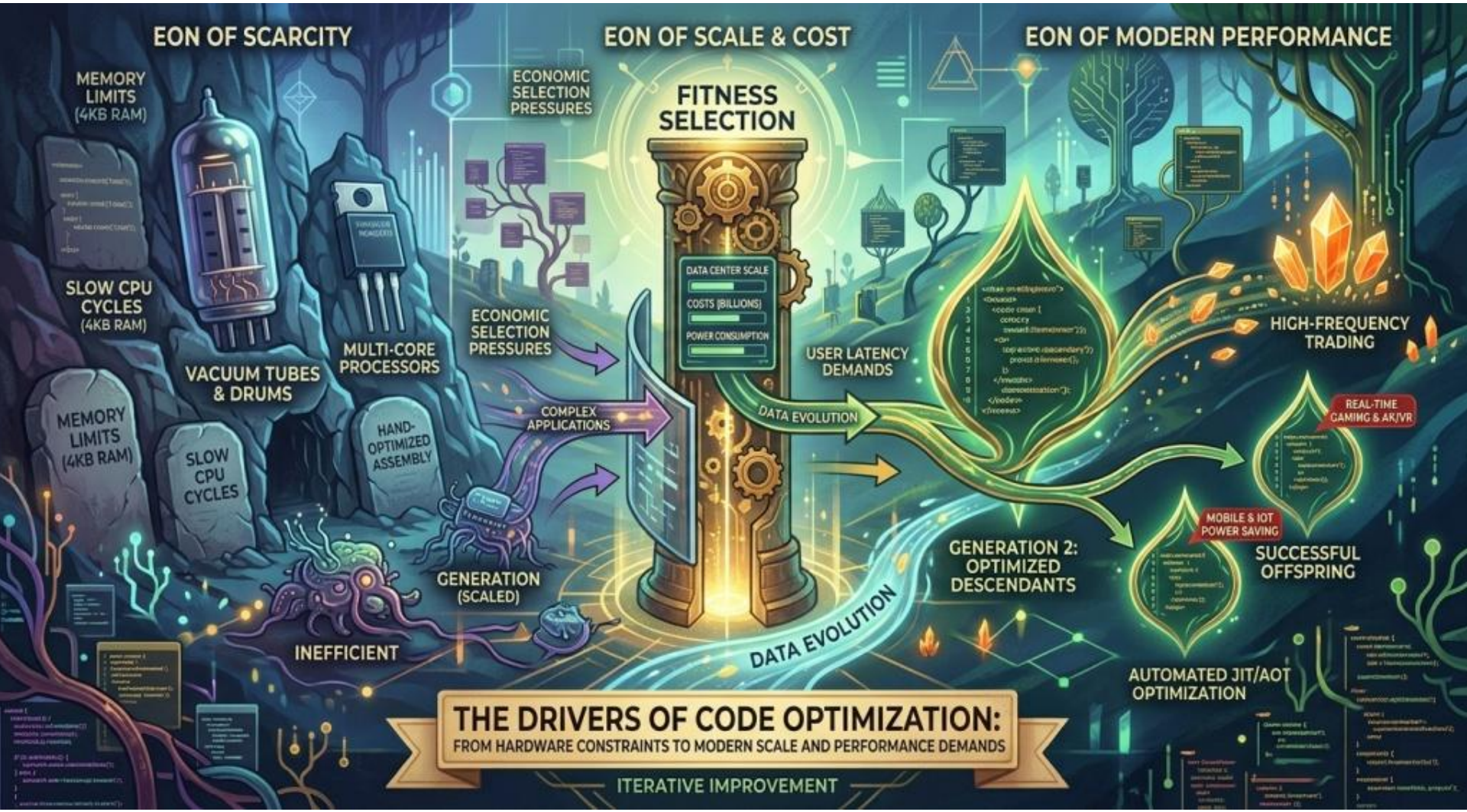
Row	op	arg1	arg2
(0)	minus	c	-
(1)	*	b	(0)
(2)	minus	c	-
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	=	a	(4)

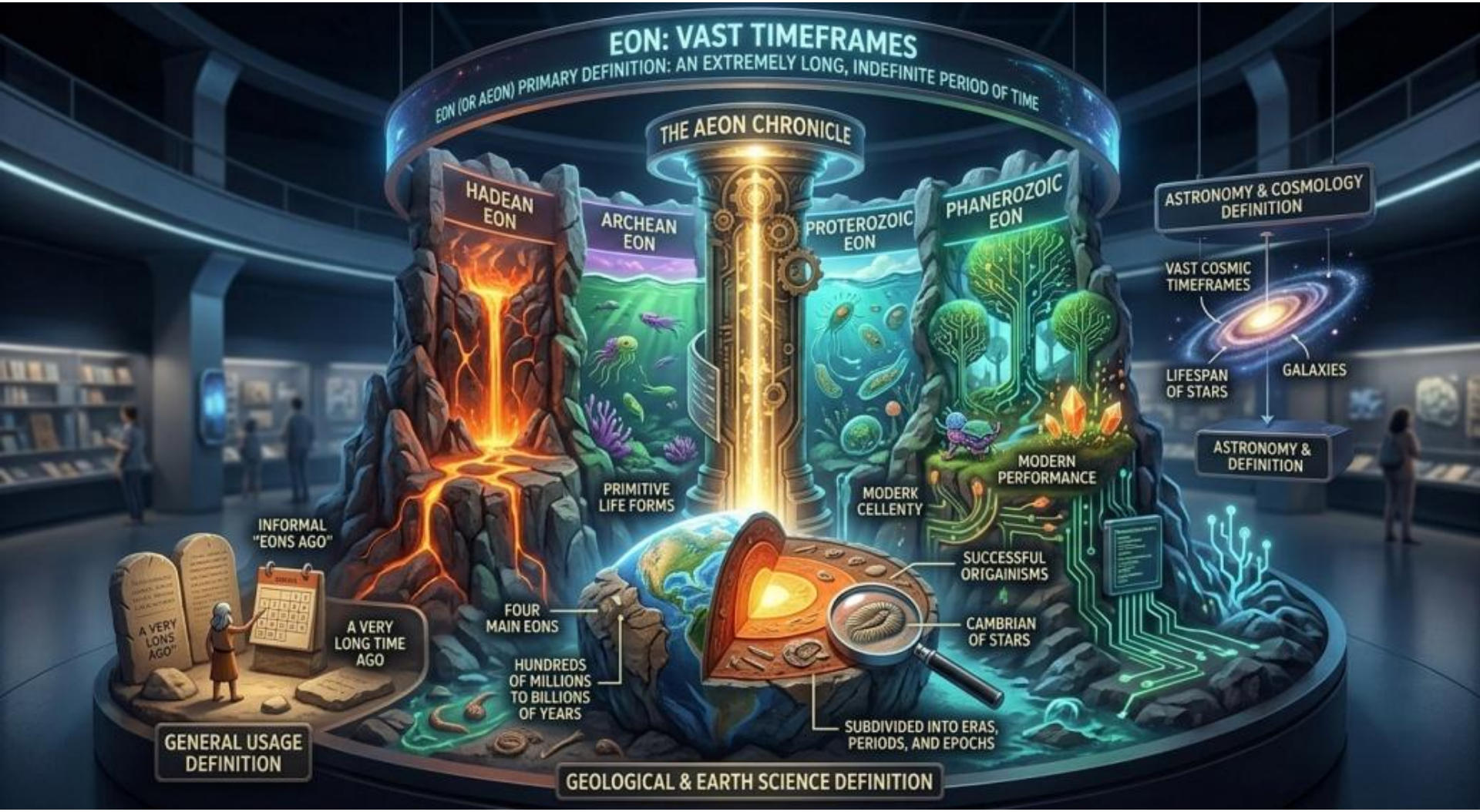
Thank You

Code Optimization

Kazy Noor e Alam Siddiquee, PhD
Professor, Department of CSE, MIST


FITNESS SELECTION
TIME
MEMORY
EFFICIENCY
INPUT DATA
INEFFICIENT
PARENT CODE
GENERATION 1
GENERATION 2: OPTIMIZED DESCENDANTS
BENEFICIAL OUTPUTS
EVOLUTION MARKSR
EVOLUTION MARKER
SUCCESSFUL OFFSPRING
SUCCESSFUL OFFSPRING
EVOLUTIONARY CODE OPTIMIZATION
SELECTION PRESSURE LEADS TO MORE EFFICIENT STRUCTURES
ITERATIVE IMPROVEMENT


EON OF SCARCITY
EON OF SCALE & COST
EON OF MODERN PERFORMANCE
MEMORY LIMITS (4KB RAM)
SLOW CPU CYCLES (4KB RAM)
VACUUM TUBES & DRUMS
MULTI-CORE PROCESSORS
MEMORY LIMITS (4KB RAM)
SLOW CPU CYCLES
HAND-OPTIMIZED ASSEMBLY
ECONOMIC SELECTION PRESSURES
FITNESS SELECTION
DATA CENTER SCALE
COSTS (BILLIONS)
POWER CONSUMPTION
ECONOMIC SELECTION PRESSURES
COMPLEX APPLICATIONS
USER LATENCY DEMANDS
DATA EVOLUTION
HIGH-FREQUENCY TRADING
REAL-TIME GAMING & AR/VR
MOBILE & IOT POWER SAVING
GENERATION (SCALED)
INEFFICIENT
GENERATION 2: OPTIMIZED DESCENDANTS
SUCCESSFUL OFFSPRING
DATA EVOLUTION
AUTOMATED JIT/AOT OPTIMIZATION
THE DRIVERS OF CODE OPTIMIZATION:
FROM HARDWARE CONSTRAINTS TO MODERN SCALE AND PERFORMANCE DEMANDS
ITERATIVE IMPROVEMENT


EON: VAST TIMEFRAMES
EON (OR AEON) PRIMARY DEFINITION: AN EXTREMELY LONG, INDEFINITE PERIOD OF TIME
THE AEON CHRONICLE
HADEAN EON
ARCHEAN EON
PROTEROZOIC EON
PHANEROZOIC EON
ASTRONOMY & COSMOLOGY DEFINITION
VAST COSMIC TIMEFRAMES
LIFESPAN OF STARS
GALAXIES
ASTRONOMY & DEFINITION
MODERN PERFORMANCE
PRIMITIVE LIFE FORMS
MODERK CELLENTY
INFORMAL "EONS AGO"
A VERY LONS AGO"
A VERY LONG TIME AGO
SUCCESSFUL ORIGAINISMS
FOUR MAIN EONS
CAMBRIAN OF STARS
HUNDREDS OF MILLIONS TO BILLIONS OF YEARS
SUBDIVIDED INTO ERAS, PERIODS, AND EPOCHS
GENERAL USAGE DEFINITION
GEOLOGICAL & EARTH SCIENCE DEFINITION

Code Optimization

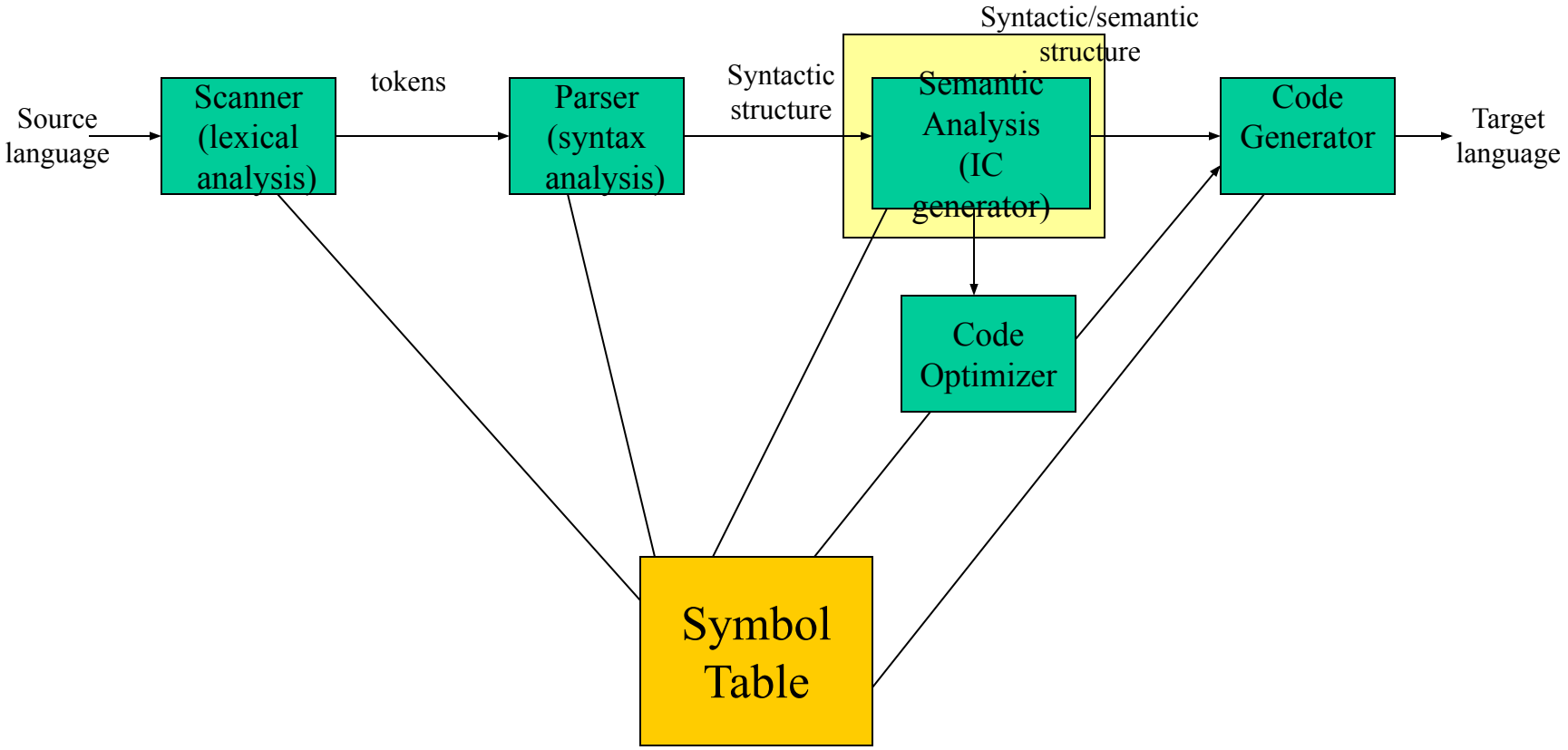
REQUIREMENTS:

- Meaning must be preserved (correctness)
- Speedup must occur on average.
- Work done must be worth the effort.

OPPORTUNITIES:

- Programmer (algorithm, directives)
- Intermediate code
- Target code

Code Optimization



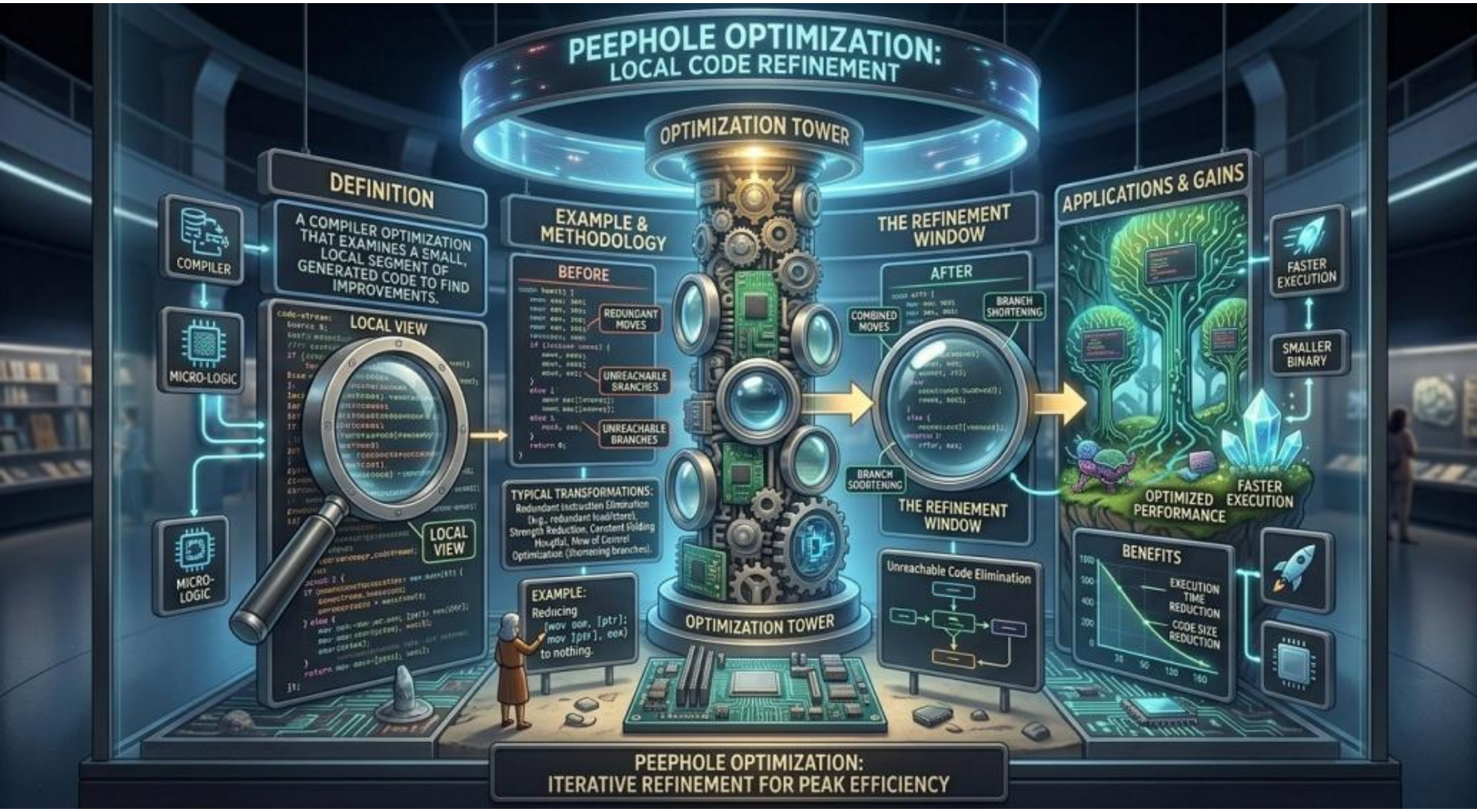
- Code optimization:
  - A transformation to a program to make it run faster and/or take up less space
  - Optimization should be safe, preserve the meaning of a program.
  - Code optimization is an important component in a compiler

  - Example: peephole optimization.
    - Peephole: a small moving window in the instruction sequence
    - Technique: examine a short sequence of target instructions (peephole) and try to replace it with a faster or shorter sequence

Levels

- Window – peephole optimization
- Basic block
- Procedural – global (control flow graph)
- Program level – intraprocedural (program dependence graph)

Peephole Optimization



- Peephole optimization:
  - Redundant instruction elimination
  - Flow of control optimization
  - Algebraic simplifications
  - Instruction selection

  - Examples:
    - Redundant loads and stores

      MOV R0, a
      MOV a, R0
    - Unreachable code

      If debug = 1 goto L1
      Goto L2
      L1: print debugging info
      L2:

Peephole Optimizations

- Constant Folding

  `x := 32` becomes `x := 64`

  `x := x + 32`

- Unreachable Code

  ```
  goto L2
  x := x + 1
  ```

  □ unneeded

- Flow of control optimizations

  `goto L1` becomes `goto L2`

  ```
  …
  L1: goto L2
  ```

Peephole Optimizations

- Algebraic Simplification

  `x := x + 0` □ unneeded

- Dead code

  `x := 32` □ where x not used after statement

  `y := x + y` □ `y := y + 32`

- Reduction in strength

  `x := x * 2` □ `x := x + x`

Peephole Optimizations

- Local in nature
- Pattern driven
- Limited by the size of the window

Basic Block Level

- Common Subexpression elimination
- Constant Propagation
- Dead code elimination
- Plus many others such as copy propagation, value numbering, partial redundancy elimination, …

Simple example: a[i+1] = b[i+1]

- t1 = i+1
- t2 = b[t1]
- t3 = i + 1
- a[t3] = t2

- t1 = i + 1
- t2 = b[t1]
- t3 = i + 1 □ *no longer live*
- a[t1] = t2

Common expression can be eliminated

Now, suppose i is a constant:

- i = 4
- t1 = i+1
- t2 = b[t1]
- a[t1] = t2

- i = 4
- t1 = 5
- t2 = b[t1]
- a[t1] = t2

- i = 4
- t1 = 5
- t2 = b[5]
- a[5] = t2

Final Code:

- i = 4
- t2 = b[5]
- a[5] = t2

Control Flow Graph - CFG

CFG = < V, E, Entry >, where

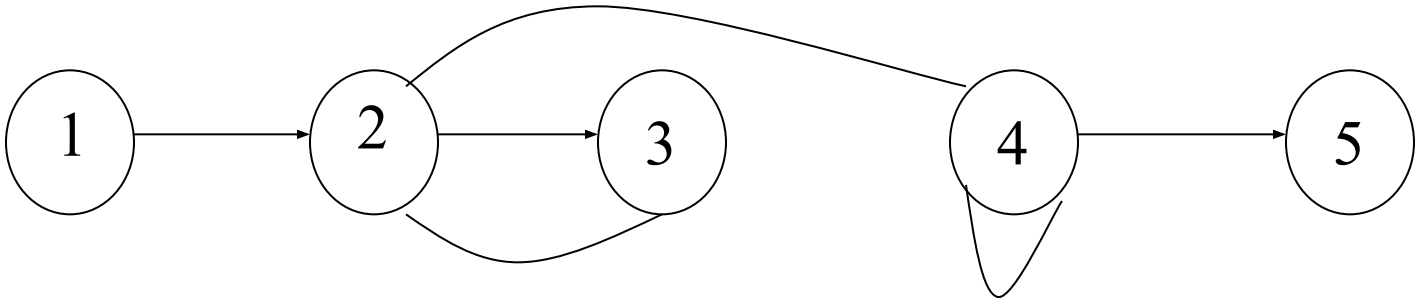
V = vertices or nodes, representing an instruction or basic block (group of statements).

E = (V x V) edges, potential flow of control

Entry is an element of V, the unique program entry

Two sets used in algorithms:

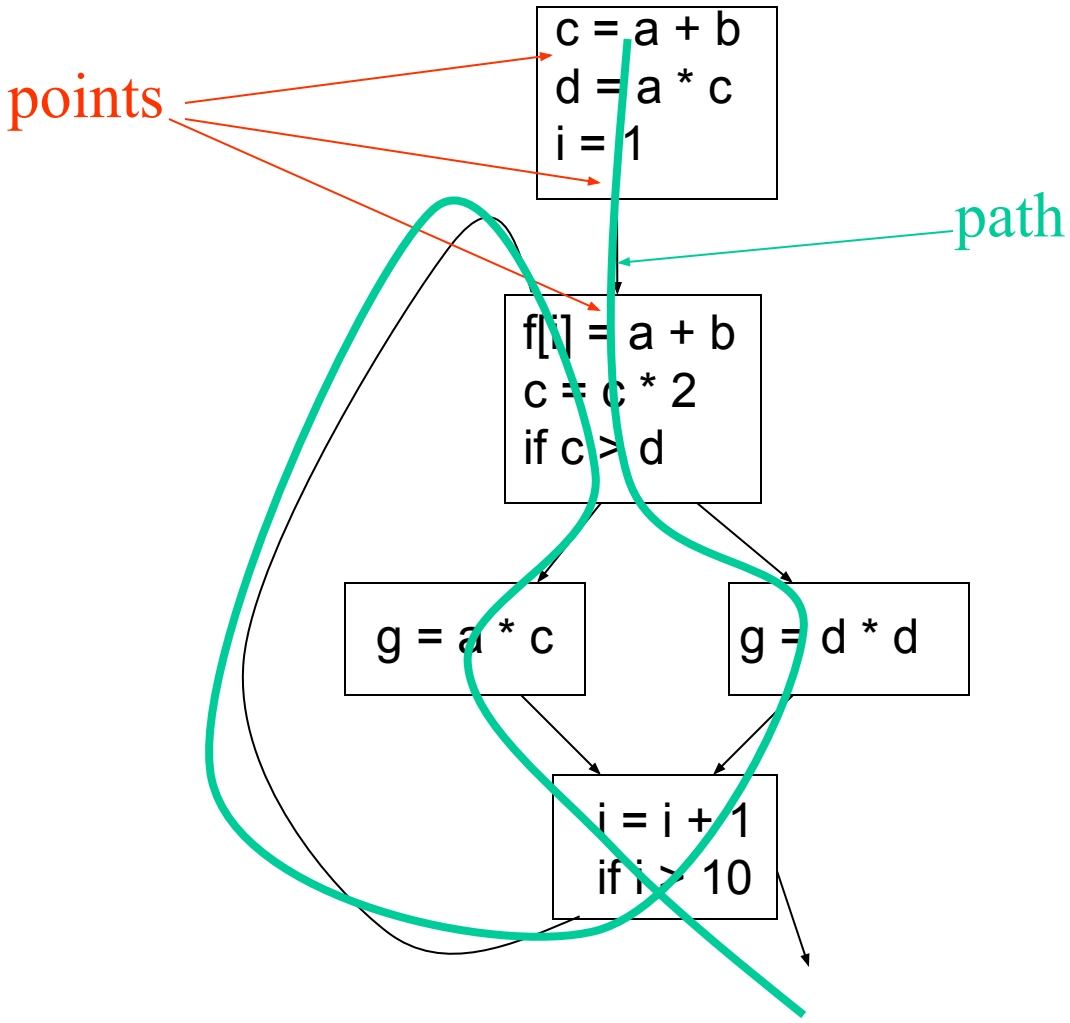
- Succ(v) = {x in V| exists e in E, e = v □x}
- Pred(v) = {x in V| exists e in E, e = x □v}



Definitions

- point - any location between adjacent statements and before and after a basic block.
- A path in a CFG from point p_1 to p_n is a sequence of points such that $\forall$ j, 1 <= j < n, either p_i is the point immediately preceding a statement and p_{i+1} is the point immediately following that statement in the same block, or p_i is the end of some block and p_{i+1} is the start of a successor block.

CFG



Optimizations on CFG

- Must take control flow into account
  - Common Sub-expression Elimination
  - Constant Propagation
  - Dead Code Elimination
  - Partial redundancy Elimination
  - …
- Applying one optimization may create opportunities for other optimizations.

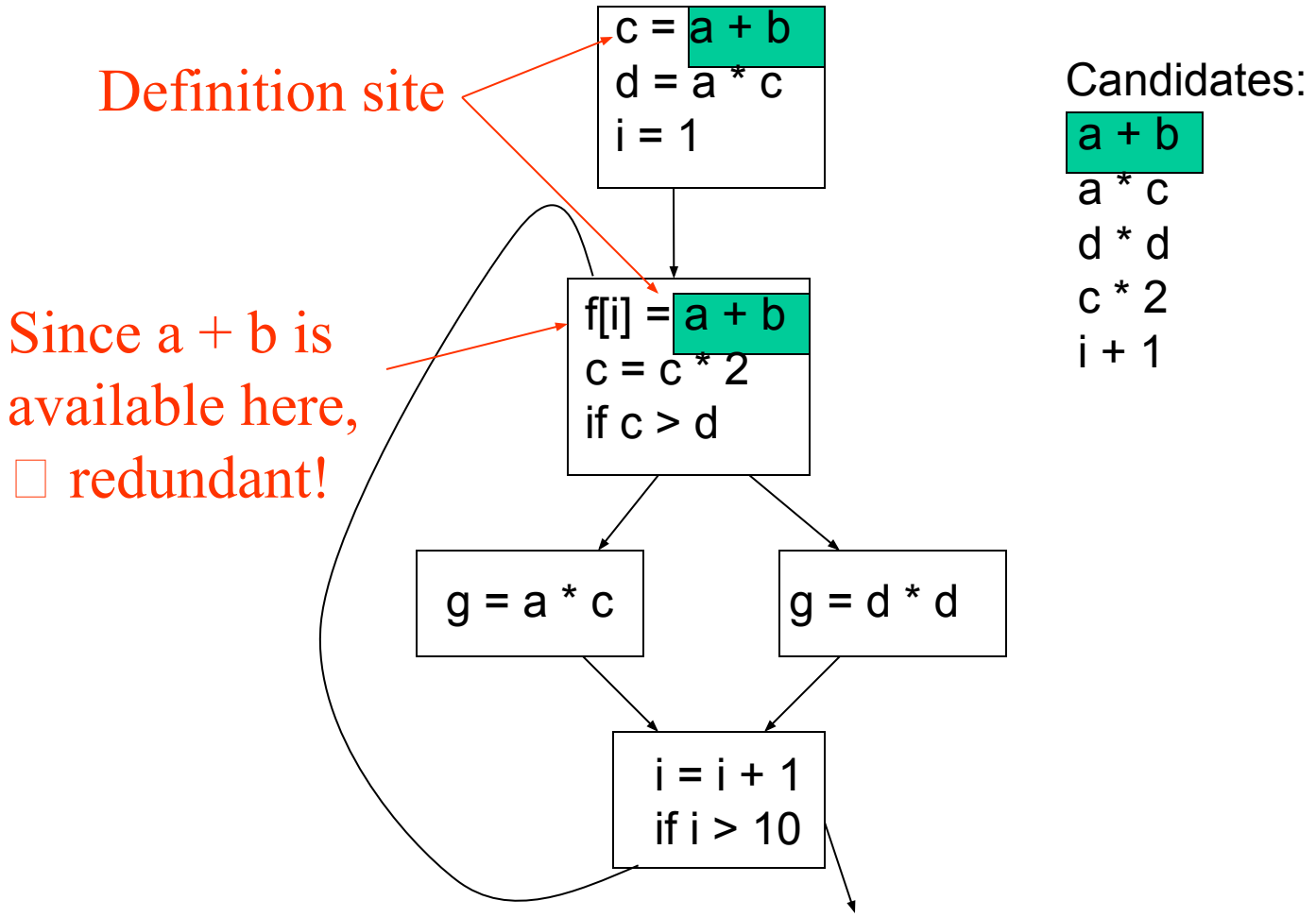
Redundant Expressions

An expression **x op y** is redundant at a point p if it has already been computed at some point(s) and no intervening operations redefine **x** or **y**.

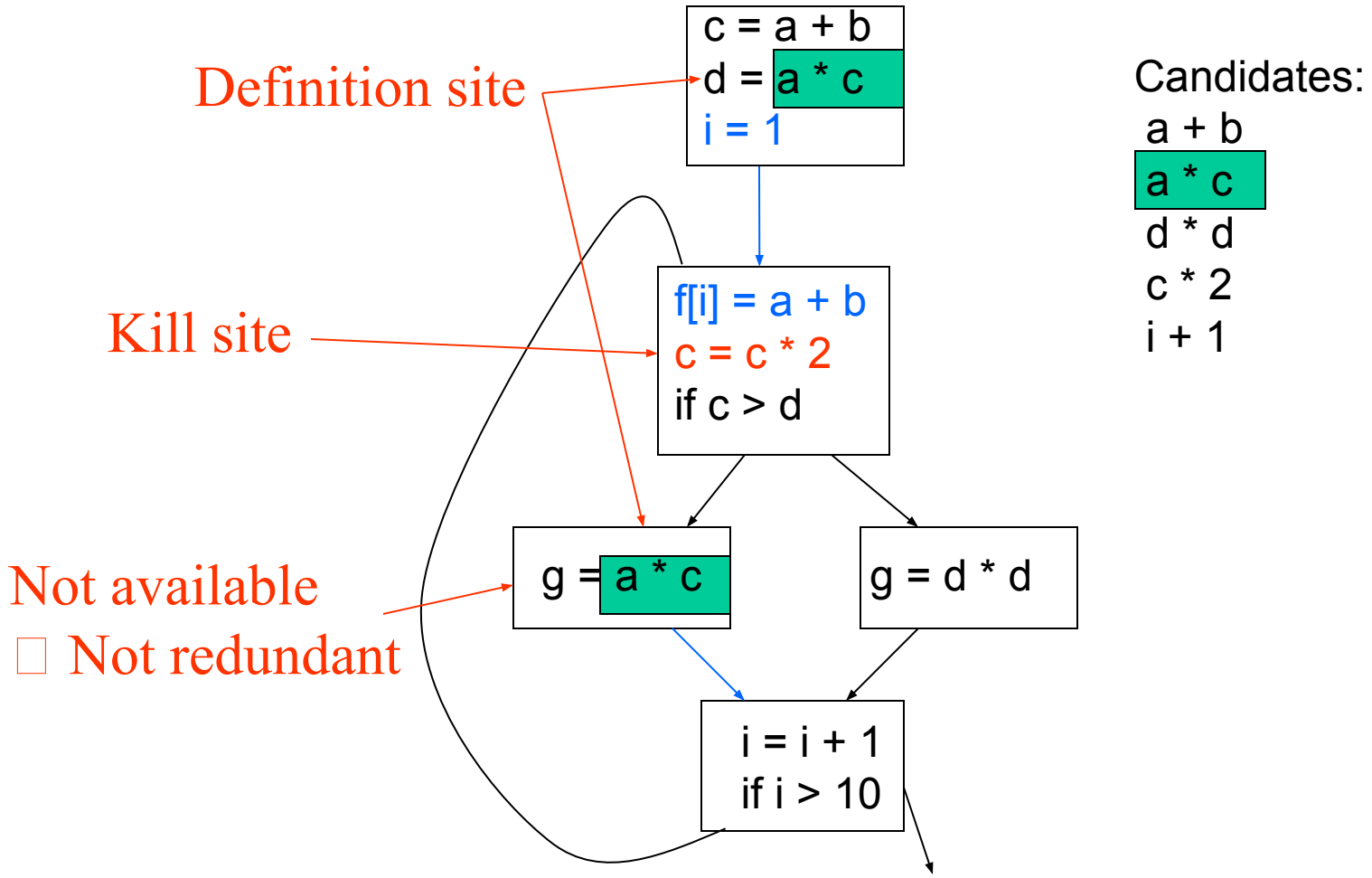
```
m = 2*y*z      t0 = 2*y       t0 = 2*y
        m = t0*z       m = t0*z
n = 3*y*z      t1 = 3*y       t1 = 3*y
        n = t1*z       n = t1*z
o = 2*y-z      t2 = 2*y
        o = t2-z       o = t0-z
```

redundant

Redundant Expressions



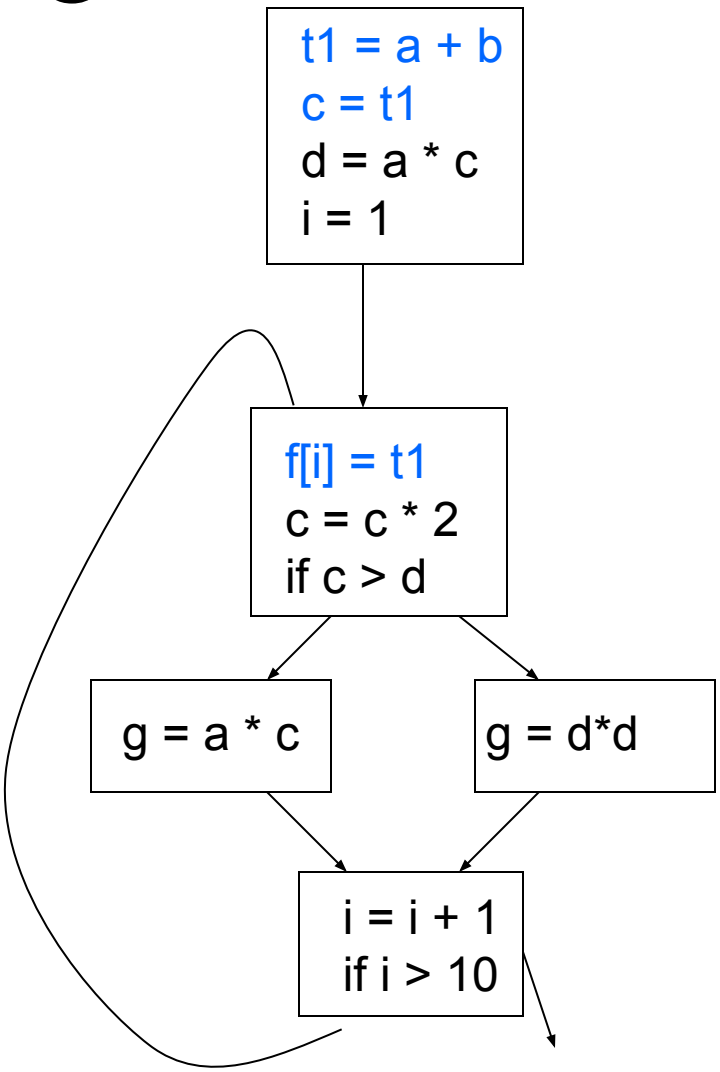
Redundant Expressions



Redundant Expressions

- An expression *e* is defined at some point *p* in the CFG if its value is computed at *p*. (definition site)
- An expression *e* is killed at point *p* in the CFG if one or more of its operands is defined at *p*. (kill site)
- An expression is ***available*** at point *p* in a CFG if every path leading to *p* contains a prior definition of *e* and *e* is not killed between that definition and *p*.

Removing Redundant Expressions



Candidates:

a + b

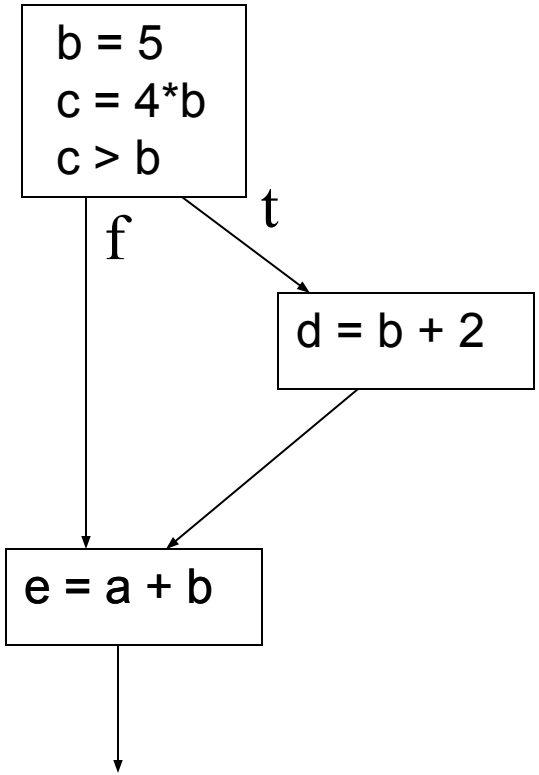
~~a * c~~

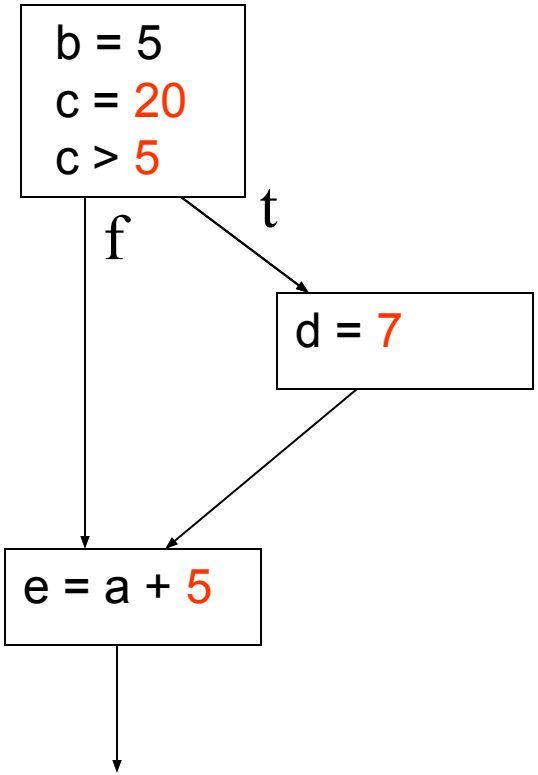
~~d * d~~

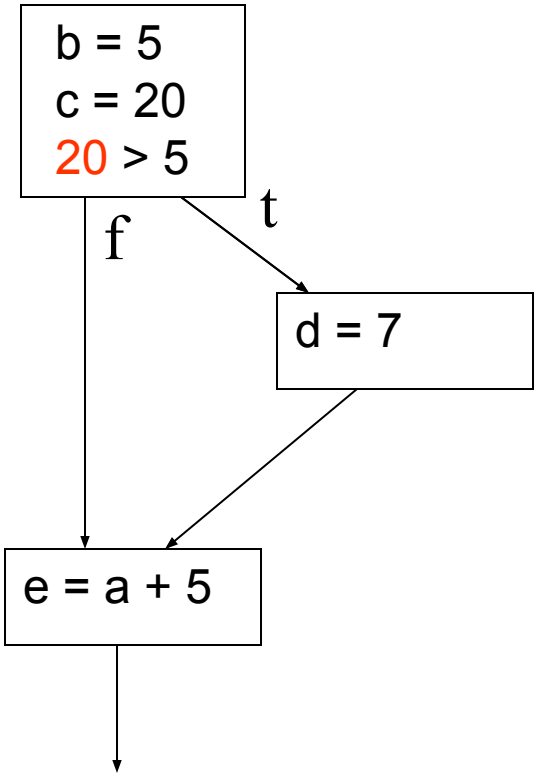
~~c * 2~~

~~i + 1~~

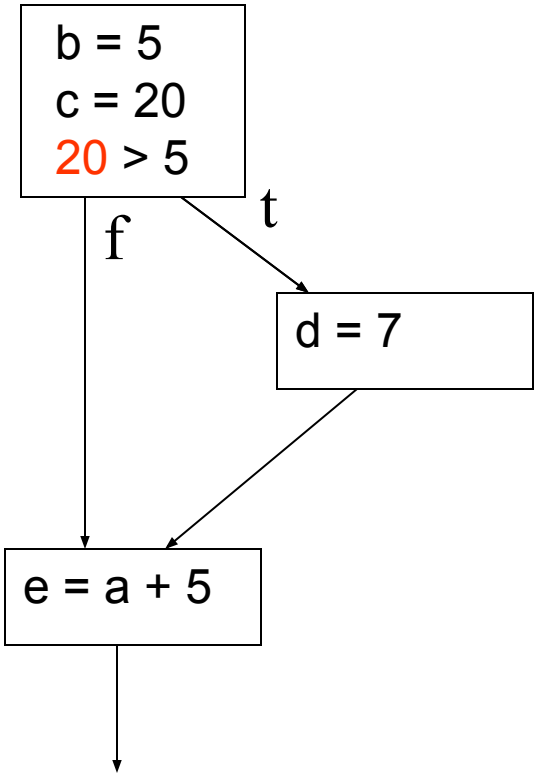
Constant Propagation

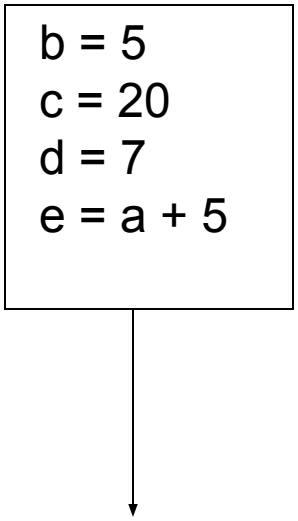




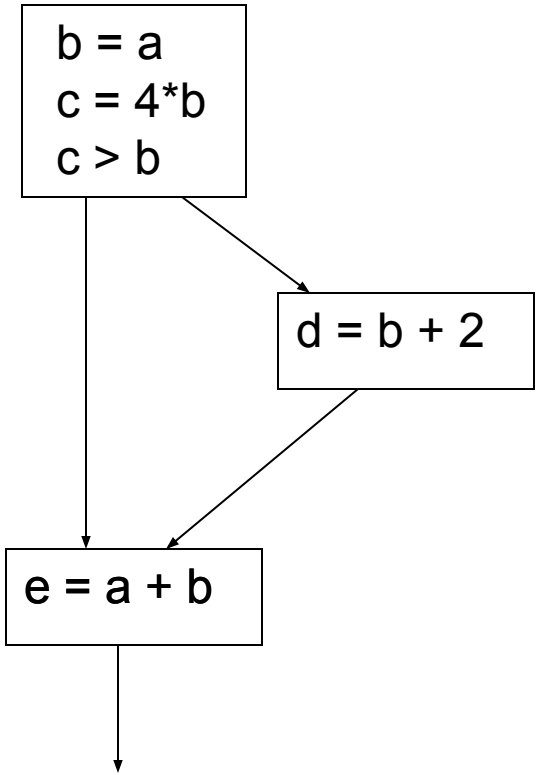


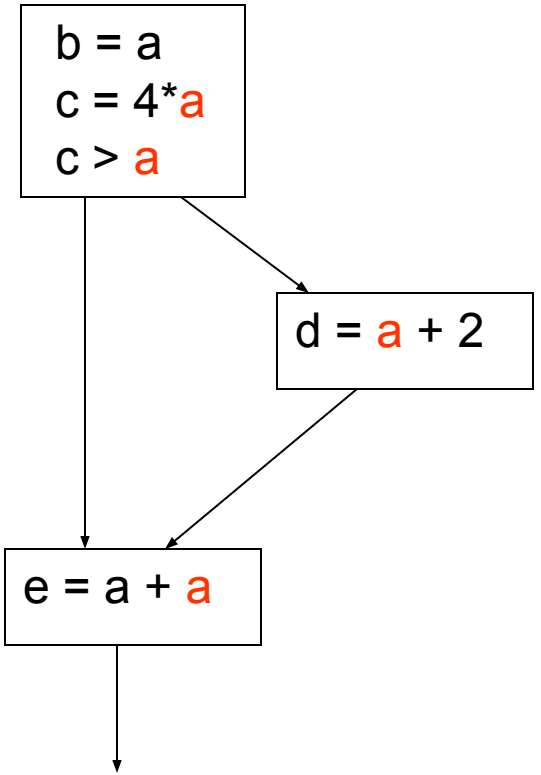
Constant Propagation





Copy Propagation





Simple Loop Optimizations: Code Motion

```
while (i <=  limit - 2)
```

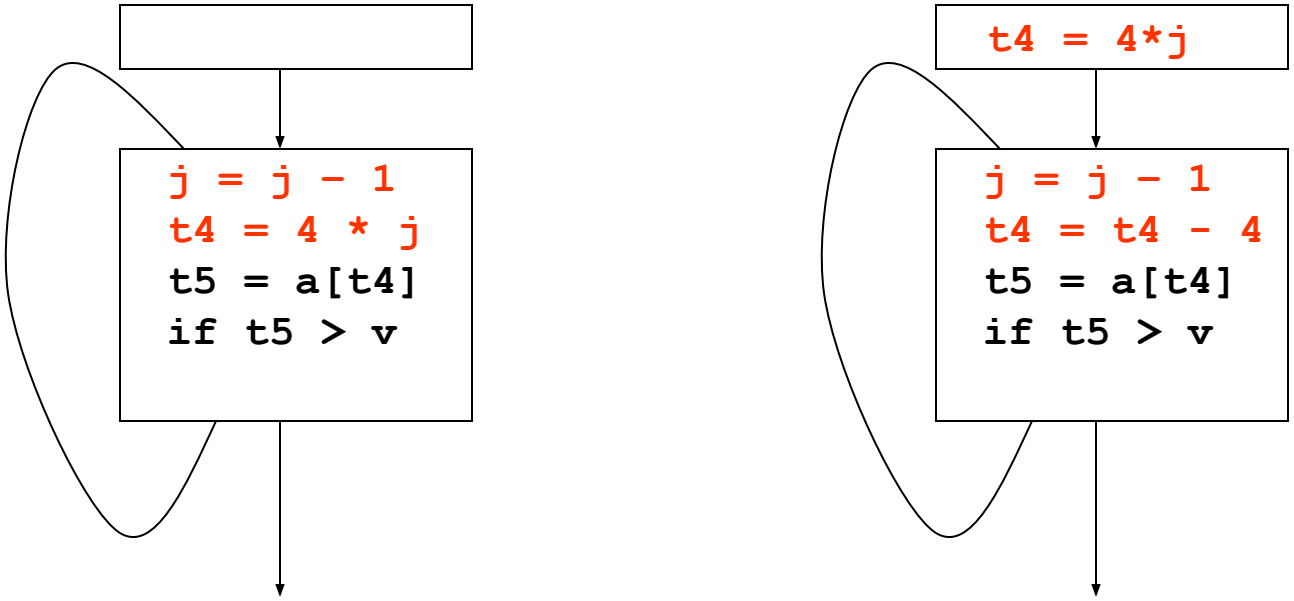
```
L1:
     t1 = limit - 2
     if (i > t1) goto L2
     body of loop
     goto L1
L2:
```

```
t := limit - 2
   while (i <= t)
```

```
     t1 = limit - 2
L1:
     if (i > t1) goto L2
     body of loop
     goto L1
L2:
```

Simple Loop Optimizations: Strength Reduction

- Induction Variables control loop iterations

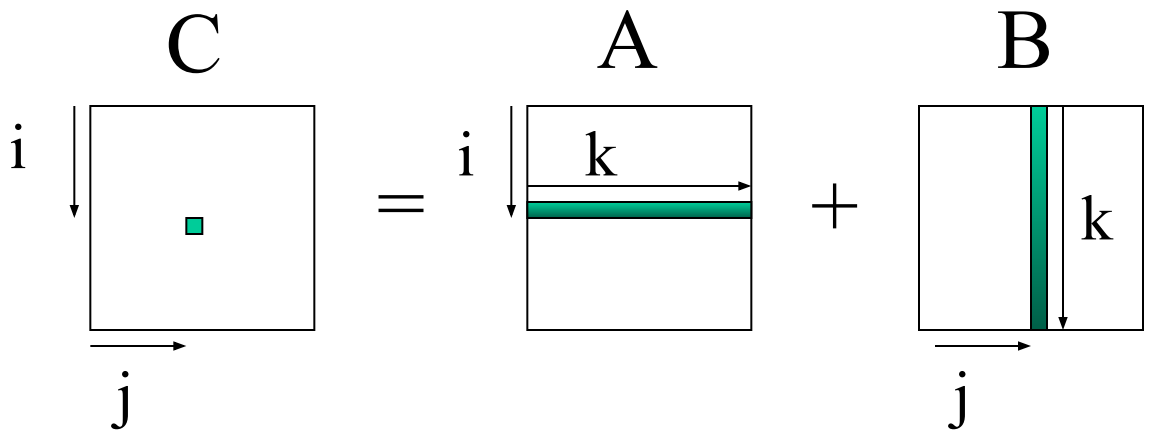


Simple Loop Optimizations

- Loop transformations are often used to expose other optimization opportunities:
  - Normalization
  - Loop Interchange
  - Loop Fusion
  - Loop Reversal
  - …

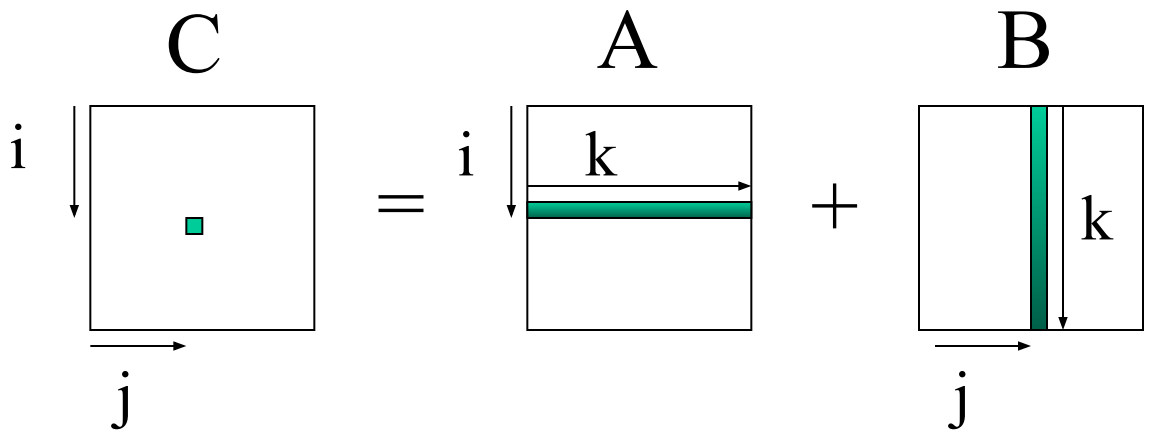
Consider Matrix Multiplication

```
for i = 1 to n do
  for j = 1 to n do
     for k = 1 to n do
        C[i,j] = C[i,j] + A[i,k] + B[k,j]
     end
  end
end
```



Memory Usage

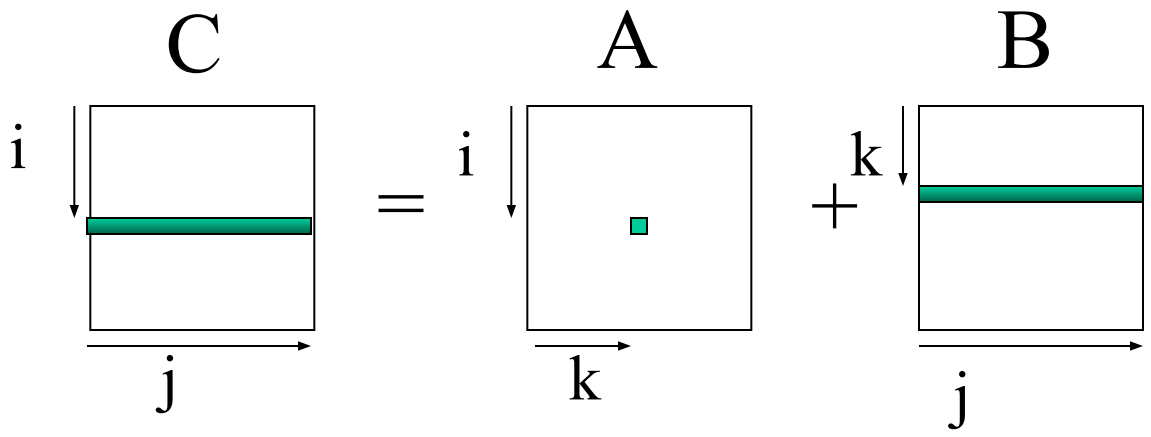
- **For A:** Elements are accessed across rows, spatial locality is exploited for cache (assuming row major storage)
- **For B:** Elements are accessed along columns, unless cache can hold all of B, cache will have problems.
- **For C:** Single element computed per loop – use register to hold



Matrix Multiplication Version 2

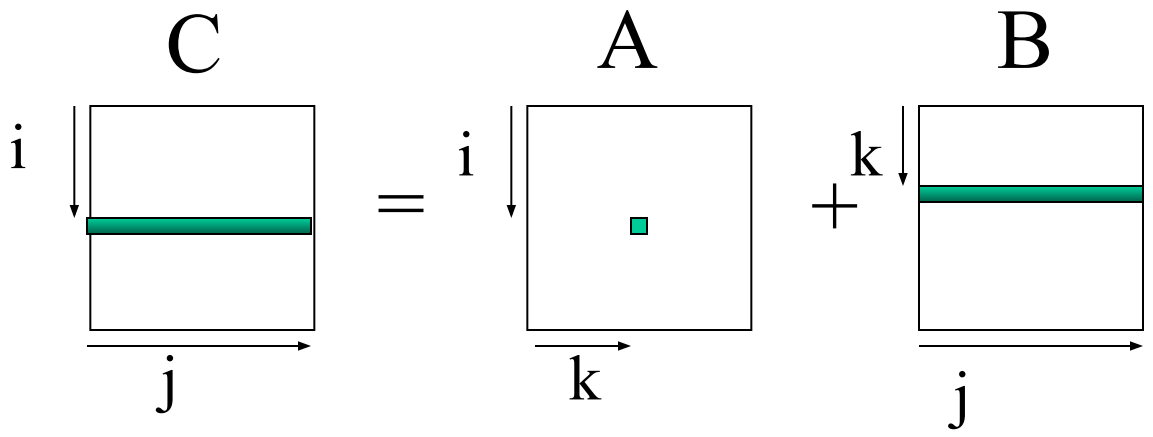
```
for i = 1 to n do
  for k = 1 to n do
     for j = 1 to n do
        C[i,j] = C[i,j] + A[i,k] + B[k,j]
     end
  end
end
```

loop interchange



Memory Usage

- **For A:** Single element loaded for loop body
- **For B:** Elements are accessed along rows to exploit spatial locality.
- **For C:** Extra loading/storing, but across rows



Simple Loop Optimizations

- How to determine safety?
  - Does the new multiply give the same answer?
  - Can be reversed??

    `for (I=1 to N) a[I] = a[I+1]` – can this loop be safely reversed?

Data Dependencies

- Flow Dependencies - write/read

  x := 4;

  y := x + 1
- Output Dependencies - write/write

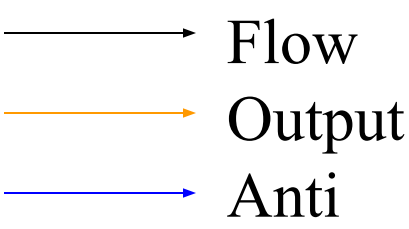
  x := 4;

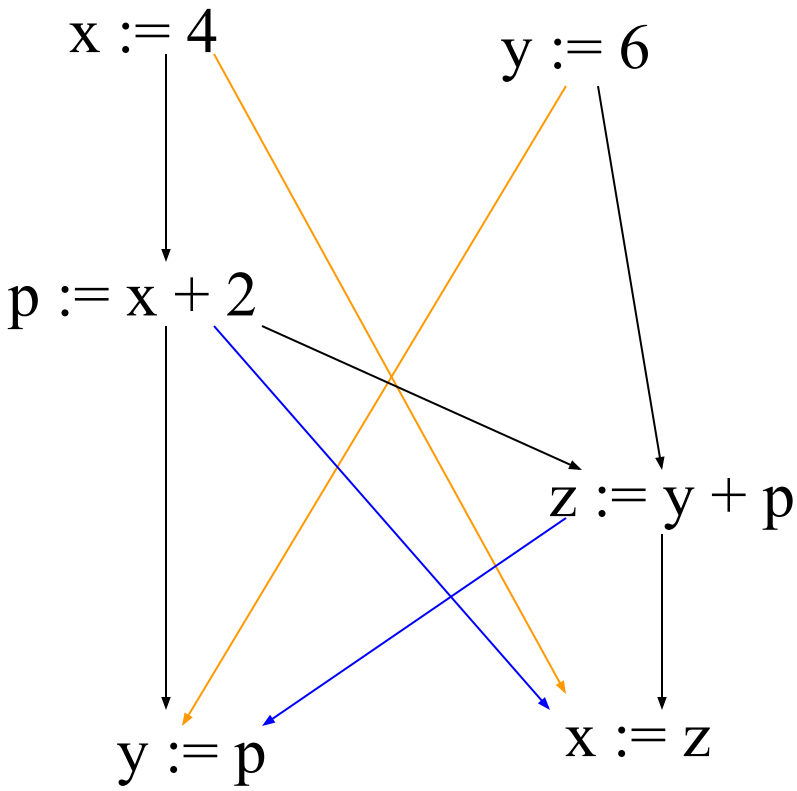
  x := y + 1;
- Antidependencies - read/write

  y := x + 1;

  x := 4;

```
x := 4
y := 6
p := x + 2
z := y + p
x := z
y := p
```





Global Data Flow Analysis

Collecting information about the way data is used in a program.

- Takes control flow into account
- HL control constructs
  - Simpler – syntax driven
  - Useful for data flow analysis of source code
- General control constructs – arbitrary branching

Information needed for optimizations such as: constant propagation, common sub-expressions, partial redundancy elimination …

Dataflow Analysis: Iterative Techniques

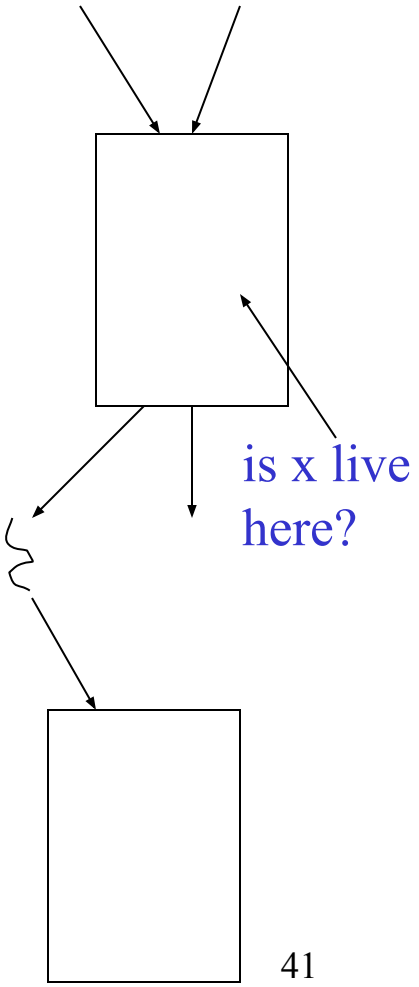
- First, compute local (block level) information.
- Iterate until no changes

```
while change do
     change = false
     for each basic block
           apply equations updating IN and OUT
           if either IN or OUT changes, set change
   to true
end
```

Live Variable Analysis

A variable **x** is *live* at a point p if there is some path from p where **x** is used before it is defined.

Want to determine for some variable x and point p whether the value of x *could* be used along some path starting at p.

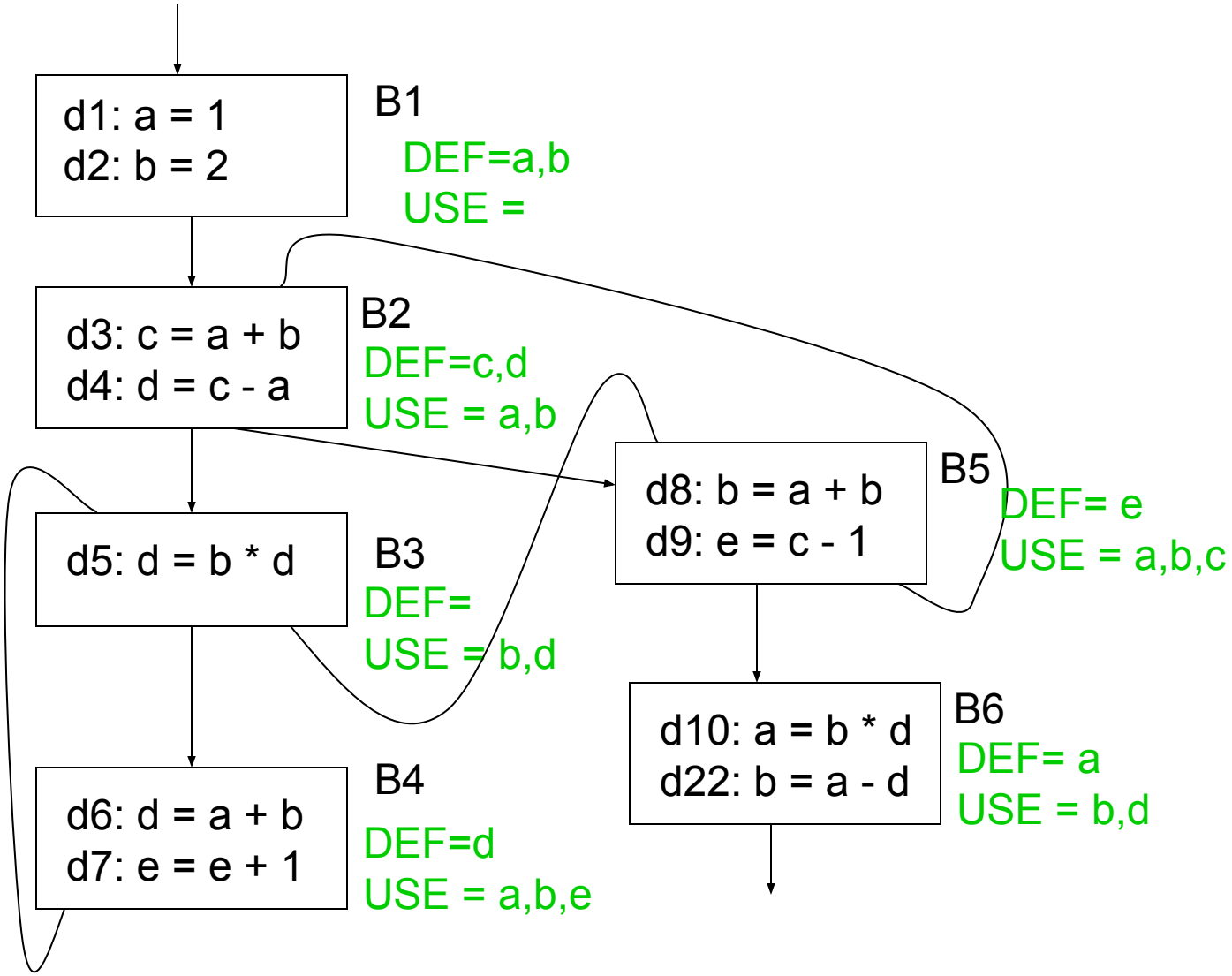


- Information flows backwards
- May – 'along some path starting at p'

Global Live Variable Analysis

Want to determine for some variable *x* and point *p* whether the value of *x* could be used along some path starting at *p*.

- DEF[B] - set of variables assigned values in B prior to any use of that variable
- USE[B] - set of variables used in B prior to any definition of that variable
- OUT[B] - variables live immediately after the block
  OUT[B] - $\cup$IN[S] for all S in succ(B)
- IN[B] - variables live immediately before the block
  IN[B] = USE[B] + (OUT[B] - DEF[B])


d1: a = 1
d2: b = 2
B1
DEF=a,b
USE =
d3: c = a + b
d4: d = c - a
B2
DEF=c,d
USE = a,b
d5: d = b * d
B3
DEF=
USE = b,d
d6: d = a + b
d7: e = e + 1
B4
DEF=d
USE = a,b,e
d8: b = a + b
d9: e = c - 1
B5
DEF= e
USE = a,b,c
d10: a = b * d
d22: b = a - d
B6
DEF= a
USE = b,d

Global Live Variable Analysis

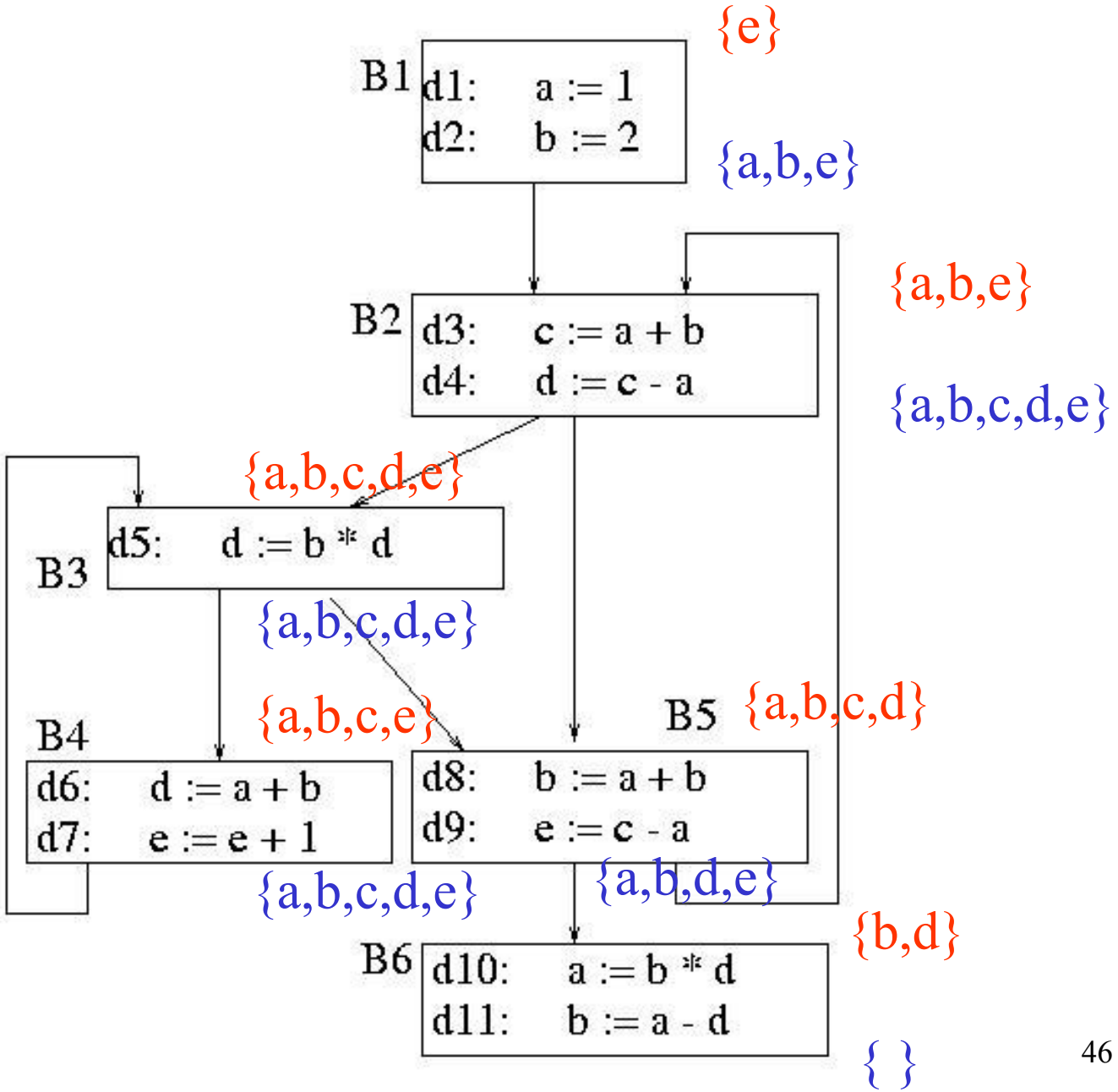
Want to determine for some variable *x* and point *p* whether the value of *x* could be used along some path starting at *p*.

- DEF[B] - set of variables assigned values in B prior to any use of that variable
- USE[B] - set of variables used in B prior to any definition of that variable
- OUT[B] - variables live immediately after the block
  OUT[B] - $\cup$ IN[S] for all S in succ(B)
- IN[B] - variables live immediately before the block
  IN[B] = USE[B] $\cup$ (OUT[B] - DEF[B])

	IN	OUT	IN	OUT	IN	OUT
B1	∅	a,b	∅	a,b	e	a,b,e
B2	a,b	a,b,c,d	a,b,e	a,b,c,d ,e	a,b,e	a,b,c,d,e
B3	a,b,c,d e	a,b,c,e	a,b,c,d,e	a,b,c,d,e	a,b,c,d,e	a,b,c,d,e
B4	a,b,c,e	a,b,c,d,e	a,b,c,e	a,b,c,d,e	a,b,c,e	a,b,c,d,e
B5	a,b,c,d	a,b,d	a,b,c,d	a,b,d,e	a,b,c,d	a,b,d,e
B6	b,d	∅	b,d	∅	b,d	∅

OUT[B] = ∪ IN[S] for all S in succ(B)
IN[B] = USE[B] + (OUT[B] - DEF[B])

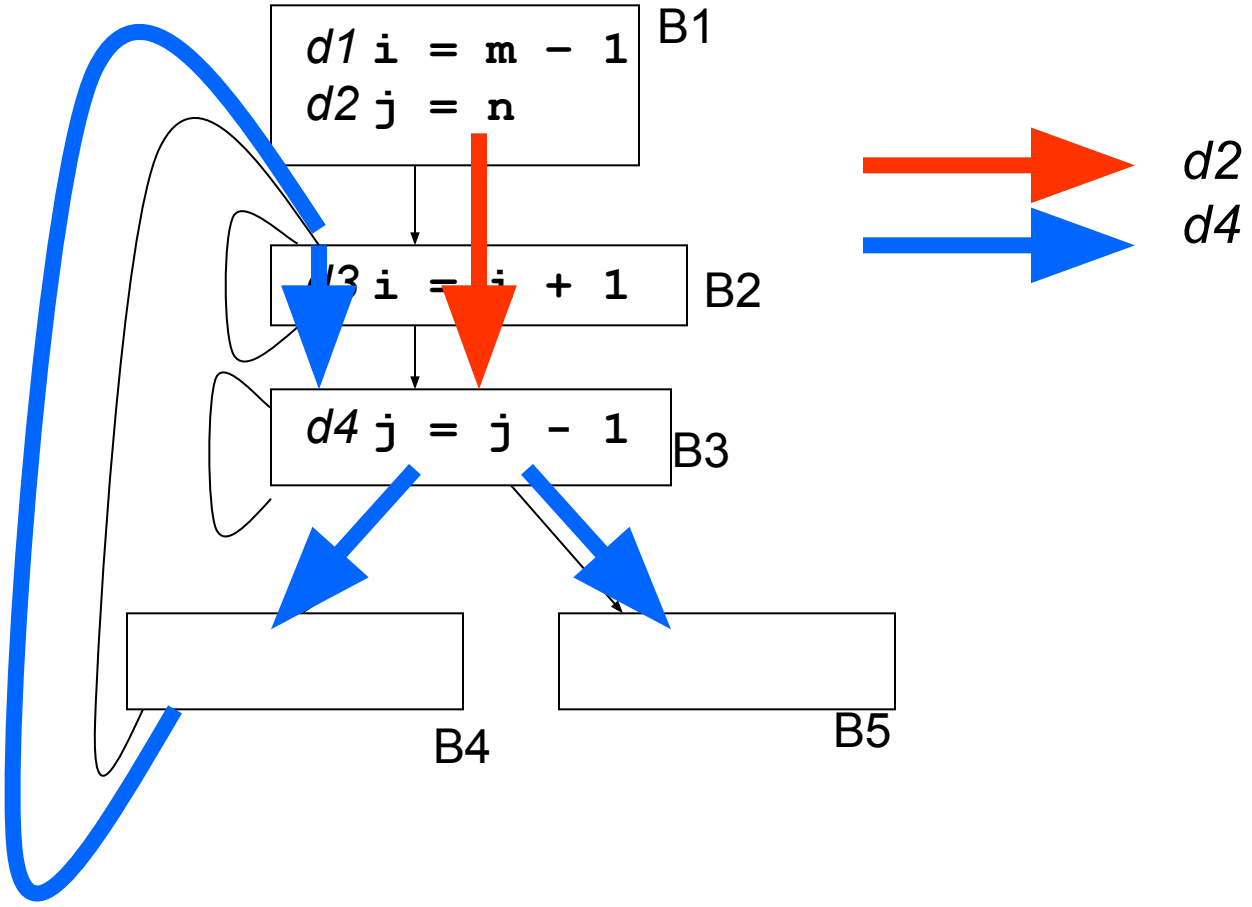
Block	DEF	USE
B1	{a,b}	{ }
B2	{c,d}	{a,b}
B3	{ }	{b,d}
B4	{d}	{a,b,e}
B5	{e}	{a,b,c}
B6	{a}	{b,d}


{e}
B1
d1: a := 1
d2: b := 2
{a,b,e}
{a,b,e}
B2
d3: c := a + b
d4: d := c - a
{a,b,c,d,e}
{a,b,c,d,e}
B3
d5: d := b * d
{a,b,c,d,e}
{a,b,c,e}
B4
d6: d := a + b
d7: e := e + 1
{a,b,c,d,e}
B5
{a,b,c,d}
d8: b := a + b
d9: e := c - a
{a,b,d,e}
{b,d}
B6
d10: a := b * d
d11: b := a - d
{ }

Dataflow Analysis Problem #2: Reachability

- A *definition of a variable x* is a statement that may assign a value to *x*.
- A definition may reach a program point *p* if there exists some path from the point immediately following the definition to *p* such that the assignment is not killed along that path.
- Concept: relationship between definitions and uses

What blocks do definitions d2 and d4 reach?



Reachability Analysis: Unstructured Input

1. Compute GEN and KILL at block—level
2. Compute IN[B] and OUT[B] for B

   IN[B] = U OUT[P] *where P is a predecessor of B*

   OUT[B] = GEN[B] U (IN[B] - KILL[B])
3. Repeat step 2 until there are no changes to OUT sets

Reachability Analysis: Step 1

For each block, compute local (block level) information = GEN/KILL sets

- GEN[B] = set of definitions generated by B
- KILL[B] = set of definitions that can not reach the end of B

This information does not take control flow between blocks into account.

Reasoning about Basic Blocks

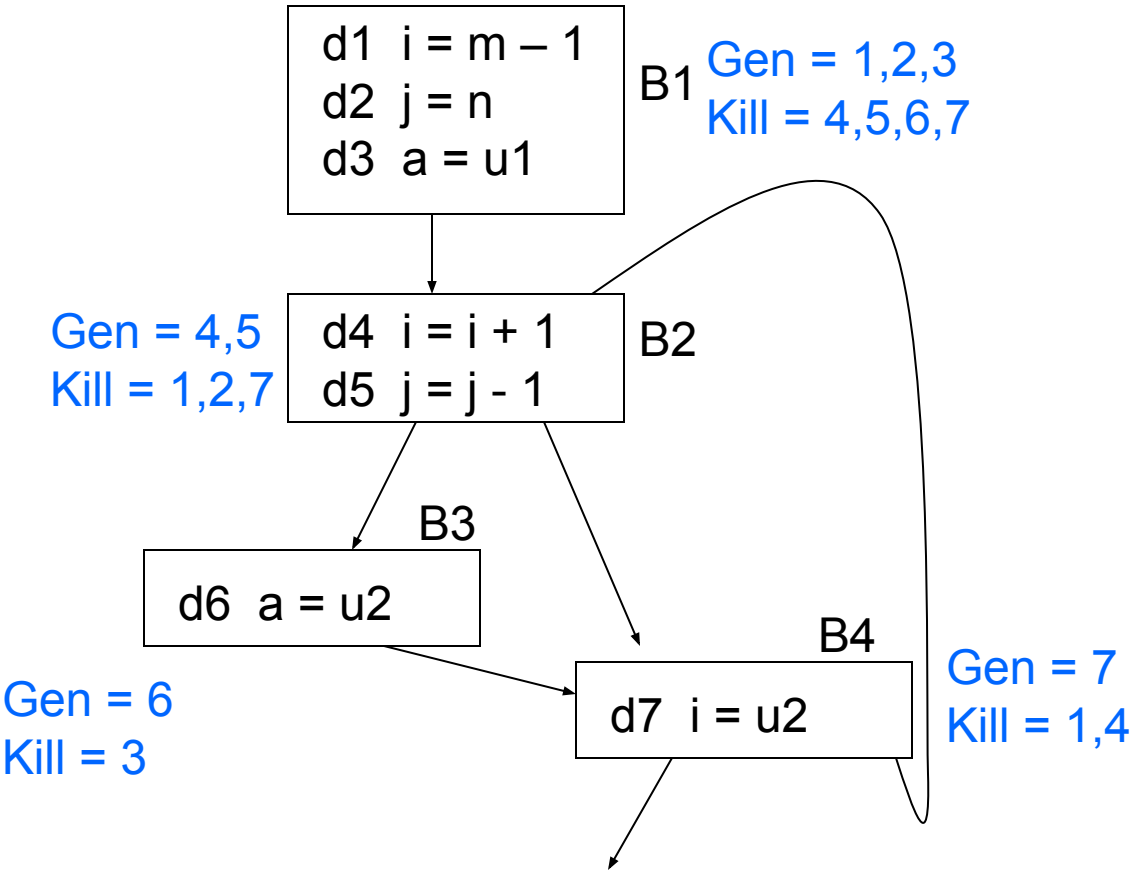
Effect of single statement: a = b + c

- Uses variables {b,c}
- **Kills all definitions of {a}**
- **Generates new definition (i.e. assigns a value) of {a}**

Local Analysis:

- Analyze the effect of each instruction
- Compose these effects to derive information about the entire block

Example


d1 i = m – 1
d2 j = n
d3 a = u1
B1
Gen = 1,2,3
Kill = 4,5,6,7
Gen = 4,5
Kill = 1,2,7
d4 i = i + 1
d5 j = j - 1
B2
B3
d6 a = u2
B4
d7 i = u2
Gen = 6
Kill = 3
Gen = 7
Kill = 1,4

Reachability Analysis: Step 2

Compute IN/OUT for each block in a forward direction. Start with IN[B] = ∅

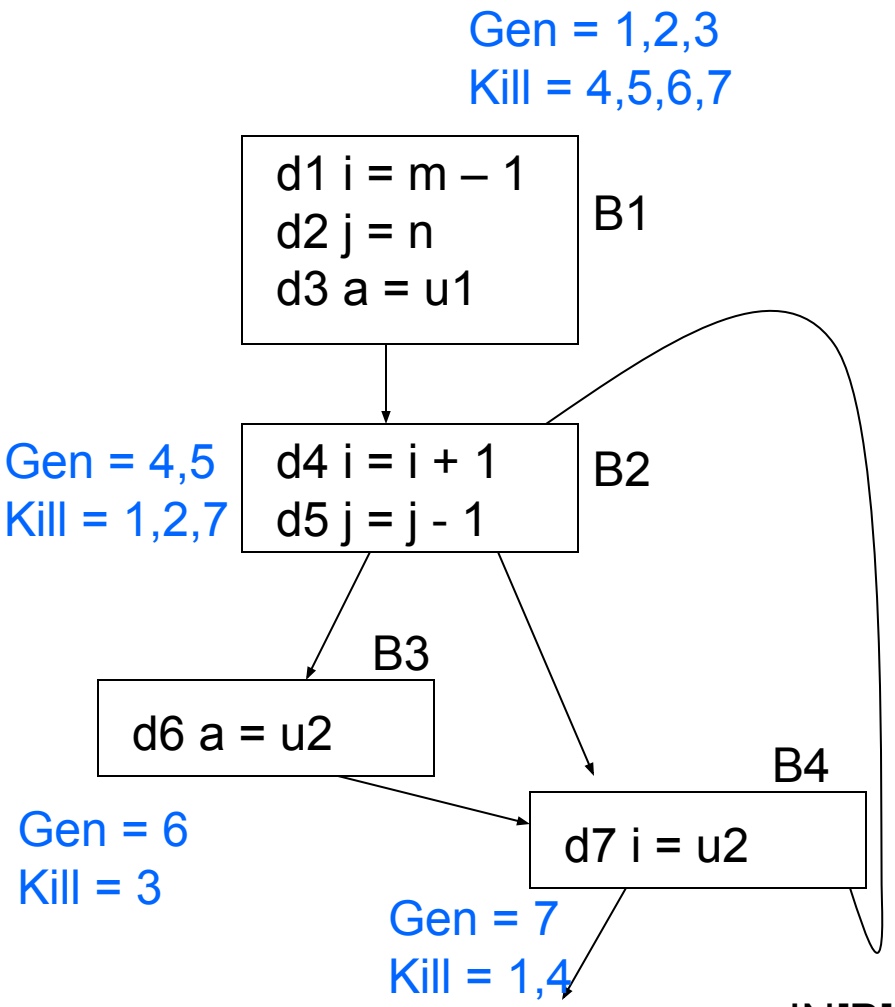
- IN[B] = *set of defns reaching the start of B*
  = ∪ (out[P]) for all predecessor blocks in the CFG
- OUT[B] = *set of defns reaching the end of B*
  = GEN[B] ∪ (IN[B] – KILL[B])

Keep computing IN/OUT sets until a fixed point is reached.

Reaching Definitions Algorithm

- Input: Flow graph with GEN and KILL for each block
- Output: in[B] and out[B] for each block.

```
For each block B do out[B] = gen[B], (true if in[B] = emptyset)
change := true;
while change do begin
        change := false;
        for each block B do begin
          in[B] := U out[P], where P is a predecessor of B;
          oldout = out[B];
          out[B] := gen[B] U  (in[B] - kill [B])
          if out[B] != oldout then change := true;
    end
end
```



	IN	OUT		
B1	∅	1,2,3		
B2	∅	4,5		
B3	∅	6		
B4	∅	7		

IN[B] = ∪(out[P]) for all predecessor blocks in the CFG

OUT[B] = GEN[B] ∪ (IN[B] – KILL[B])

	IN	OUT	IN	OUT		
B1	∅	1,2,3	∅	1,2,3		
B2	∅	4,5	OUT[1]+OUT[4] = 1,2,3,7	4,5 + (1,2,3,7 – 1,2,7) = 3,4,5		
B3	∅	6	OUT[2] = 3,4,5	6 + (3,4,5 – 3) = 4,5,6		
B4	∅	7	OUT[2]+OUT[3] = 3,4,5,6	7 + (3,4,5,6 – 1,4) = 3,5,6,7		

IN[B] = ∪(out[P]) for all predecessor blocks in the CFG

OUT[B] = GEN[B] + (IN[B] – KILL[B])

	IN	OUT	IN	OUT	IN	OUT
B1	∅	1,2,3	∅	1,2,3	∅	1,2,3
B2	∅	4,5	1,2,3,7	3,4,5	OUT[1] + OUT[4] = 1,2,3,5,6,7	4,5 + (1,2,3,5,6,7-1,2,7) = 3,4,5,6
B3	∅	6	3,4,5	4,5,6	OUT[2] = 3,4,5,6	6 + (3,4,5,6 – 3) = 4,5,6
B4	∅	7	3,4,5,6	3,5,6,7	OUT[2] + OUT[3] = 3,4,5,6	7+(3,4,5,6 – 1,4) = 3,5,6,7

IN[B] = ∪(out[P]) for all predecessor blocks in the CFG

OUT[B] = GEN[B] + (IN[B] – KILL[B])

Forward vs. Backward

Forward flow vs. Backward flow

Forward: Compute OUT for given IN,GEN,KILL

- Information propagates from the predecessors of a vertex.
- Examples: Reachability, available expressions, constant propagation

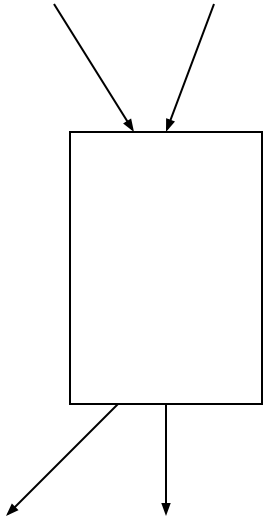
Backward: Compute IN for given OUT,GEN,KILL

- Information propagates from the successors of a vertex.
- Example: Live variable Analysis

Forward vs. Backward Equations

Forward vs. backward

- Forward:
  - IN[B] - process OUT[P] for all P in predecessors(B)
  - OUT[B] = local U (IN[B] – local)
- Backward:
  - OUT[B] - process IN[S] for all S in successor(B)
  - IN[B] = local U (OUT[B] – local)



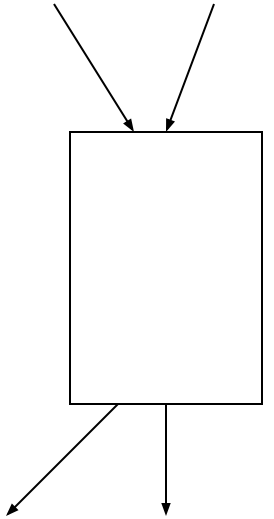
May vs. Must

May vs. Must

- Must – true on **all paths**
  - Ex: constant propagation – variable must provably hold appropriate constant on all paths in order to do a substitution
- May – true on **some path**
  - Ex: Live variable analysis – a variable is live if it could be used on some path; reachability – a definition reaches a point if it can reach it on some path

May vs. Must Equations

- May vs. Must
  - May – IN[B] = ∪(out[P]) for all P in pred(B)
  - Must – IN[B] = ∩(out[P]) for all P in pred(B)



- Reachability
  - IN[B] = ∪(out[P]) for all P in pred(B)
  - OUT[B] = GEN[B] + (IN[B] – KILL[B])
- Live Variable Analysis
  - OUT[B] = ∪(IN[S]) for all S in succ(B)
  - IN[B] = USE[B] ∪ (OUT[B] - DEF[B])
- Constant Propagation
  - IN[B] = ∩(out[P]) for all P in pred(B)
  - OUT[B] = DEF_CONST[B] ∪ (IN[B] – KILL_CONST[B])

Discussion

- Why does this work?
  - Finite set – can be represented as bit vectors
  - Theory of lattices
- Is this guaranteed to terminate?
  - Sets only grow and since finite in size …
- Can we find ways to reduce the number of iterations?

Choosing visit order for Dataflow Analysis

In forward flow analysis situations, if we visit the blocks in depth first order, we can reduce the number of iterations.

Suppose definition d follows block path 3 □ 5 □ 19 □ 35 □ 16 □ 23 □ 45 □ 4 □ 10 □ 17 where the block numbering corresponds to the preorder depth-first numbering.

Then we can compute the reach of this definition in 3 iterations of our algorithm.

3 □ 5 □ 19 □ 35 □ 16 □ 23 □ 45 □ 4 □ 10 □ 17



Runtime environemt

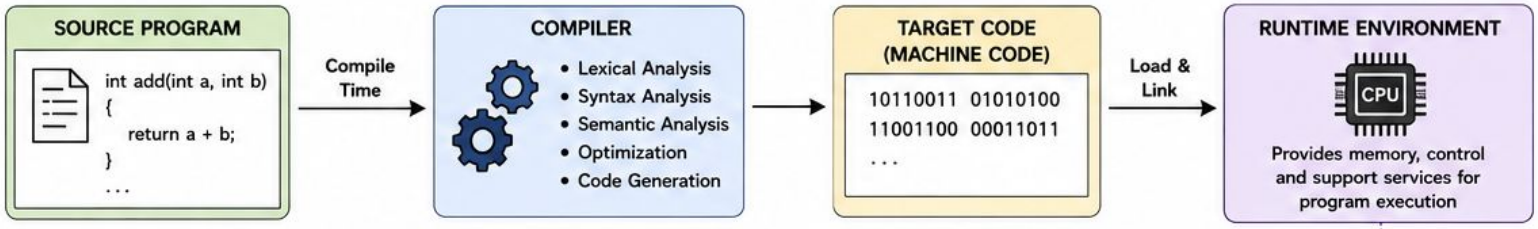
Kazy Noor e Alam Siddiquee, PhD
Professor, CSE, MIST

Run-Time Environments

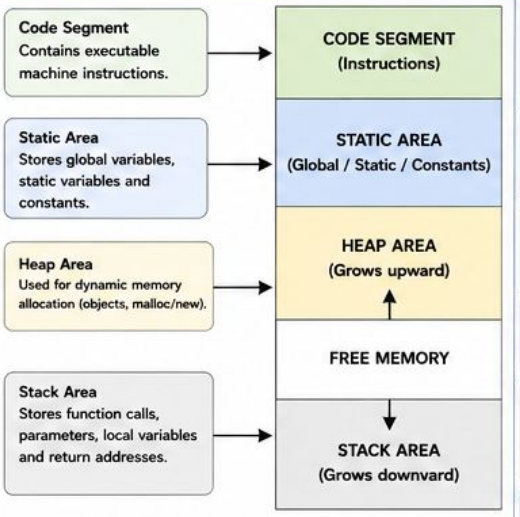
- How do we allocate the space for the generated target code and the data object of our source programs?
- The places of the data objects that can be determined at compile time will be *allocated statically*.
- But the places for the some of data objects will be *allocated at run-time*.
- The allocation of de-allocation of the data objects is managed by the *run-time support package*.
  - run-time support package is loaded together with the generate target code.
  - the structure of the run-time support package depends on the semantics of the programming language (especially the semantics of procedures in that language).
- Each execution of a procedure is called as *activation of that procedure*.

RUNTIME ENVIRONMENT – CONCEPTS AND STRUCTURE

DEFINITION: Runtime Environment is the collection of data structures, memory areas and mechanisms that support program execution after compilation.



1. MEMORY ORGANIZATION DURING EXECUTION



2. RUNTIME COMPONENTS AND SERVICES

Activation Records (Stack Frames)
Hold information for each function call: parameters, local variables, return address, control link (static link / dynamic link).

Procedure Call Mechanism
Handles call, return, parameter passing, return value, and control transfer.

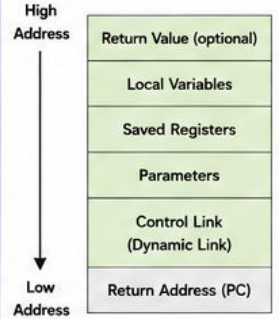
Scope and Access Management
Helps to access variables according to scope rules (local, non-local, global).

Dynamic Memory Management
Supports allocation and deallocation of memory at run time (heap management).

Exception Handling
Supports detection of run-time errors, stack unwinding and transfer of control to handlers.

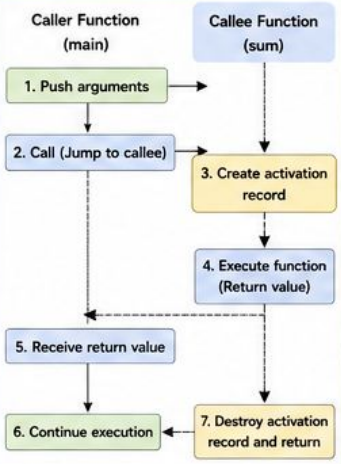
I/O and System Services
Provides interface to OS for input/output operations and other system services.

3. ACTIVATION RECORD (STACK FRAME)



Created on function call and removed on return.
Each call has its own activation record.

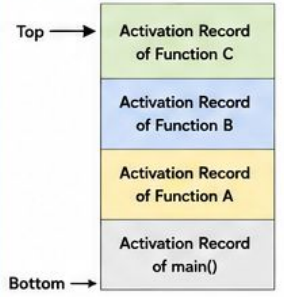
4. FUNCTION CALL SEQUENCE



6. KEY CONCEPTS SUMMARY

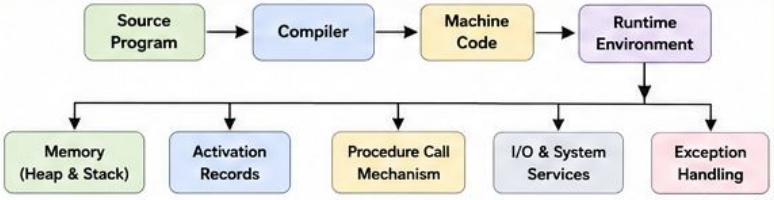
Memory Organization: Code, Static, Heap and Stack.
Activation Records: Hold parameters, locals, return address, control link.
Procedure Calls: Managed using call stack and control transfer.
Parameter Passing: By value, by reference, by result, etc.
Dynamic Memory: Allocated and freed at run time (heap).
Recursion Support: Each call gets its own activation record on stack.
Scope Management: Access to local and non-local variables.
Exception Handling: Run-time error detection and handling.

5. CALL STACK (DURING EXECUTION)



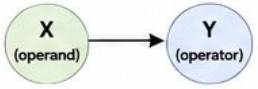
Records are pushed on call and popped on return.

OVERALL VIEW



DEPENDENCY (EDGE) RULE

If X is used to compute Y, there is a directed edge X → Y (operand → operator).



SUMMARY

- ✓ Runtime environment bridges the gap between compiled code and actual execution.
- ✓ It provides memory areas, activation records, and control mechanisms.
- ✓ It manages function calls, parameters, recursion, dynamic memory and exceptions.
- ✓ It ensures correct execution of the program in a structured and efficient way.

★ **The runtime environment provides the infrastructure required for the generated code to execute correctly.**

Procedure Activations

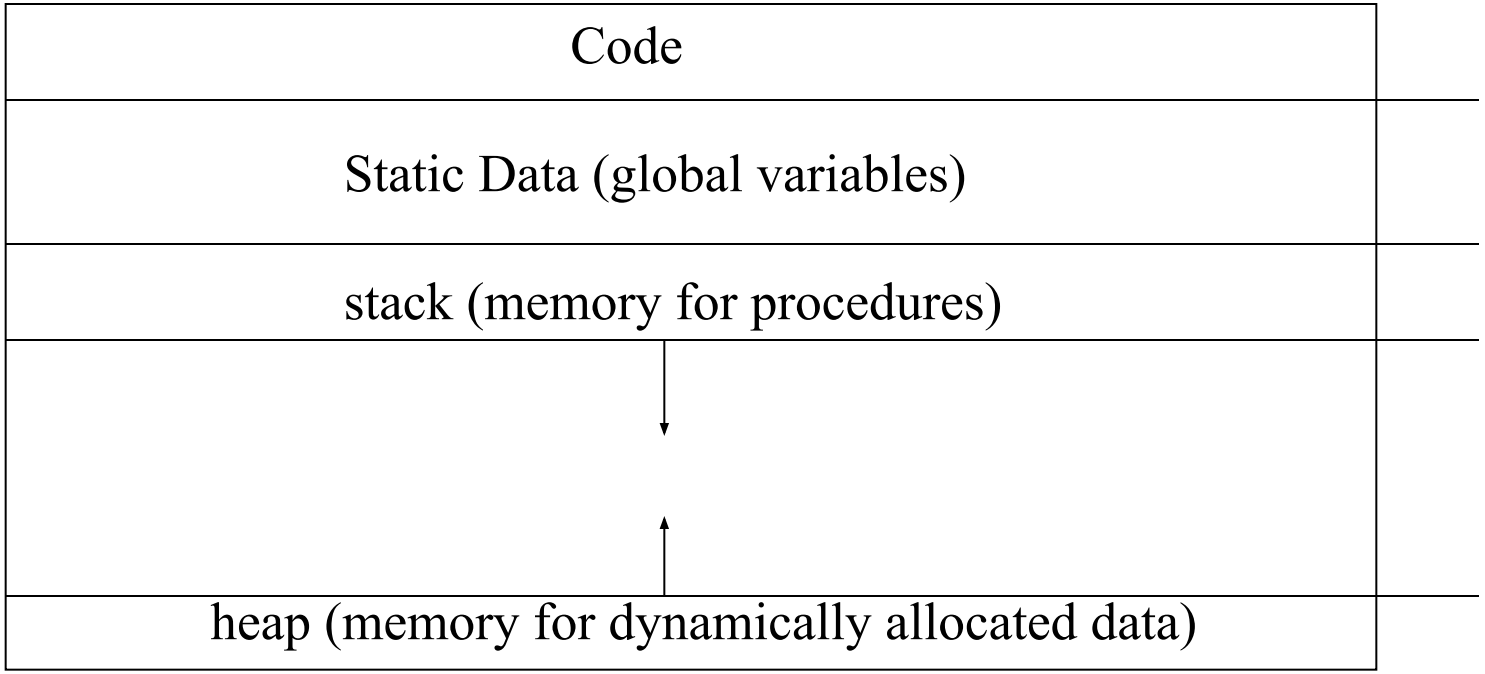
- An execution of a procedure starts at the beginning of the procedure body;
- When the procedure is completed, it returns the control to the point immediately after the place where that procedure is called.
- Each execution of a procedure is called as its *activation*.
- *Lifetime* of an activation of a procedure is the sequence of the steps between the first and the last steps in the execution of that procedure (including the other procedures called by that procedure).
- If a and b are procedure activations, then their lifetimes are either non-overlapping or are nested.
- If a procedure is recursive, a new activation can begin before an earlier activation of the same procedure has ended.

– Run time memory organization.

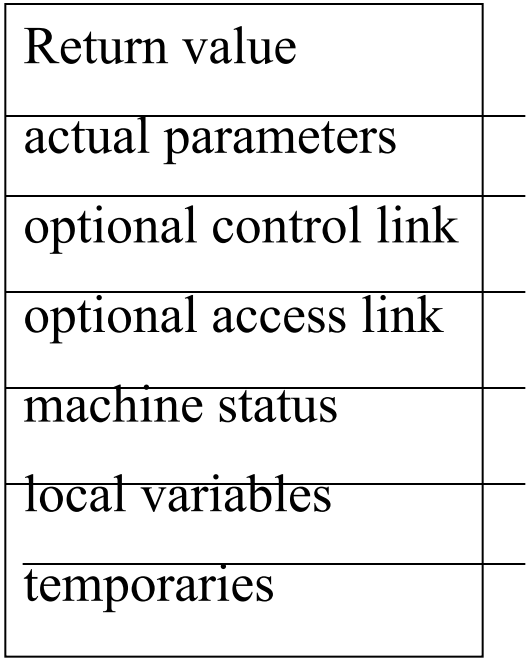
```
char   abc[1000];
char *foo() { char buf[50], *c; buf[0] = '\0'; c=malloc(50);
                    return( c );}
main() {char *c; c = foo();}
```

- We need to use memory to store:
  - code
  - static data (global variables)
  - dynamic data objects
    - » data that are used when executing a certain procedure.
    - » Dynamically allocated objects (malloc, free).

– Typical organization of run-time memory



- Activation Records:
  - also called frames
  - Information(memory) needed by a single execution of a procedure
  - A general activation record:



STATIC vs DYNAMIC MEMORY ALLOCATION

Differences, Examples and Applications

STATIC MEMORY ALLOCATION

Memory is allocated at compile time (before program execution) and its size is fixed throughout the program.

DYNAMIC MEMORY ALLOCATION

Memory is allocated at run time (during program execution) from the heap and size can change as needed.

EXAMPLE (C)

```c
// Static allocation
int a[5] = {1,2,3,4,5};
int b;

// Stack allocation
void func()
{
   int x = 10;  // stored on stack
}

// Global (static) allocation
static int count = 0;
int g = 100;          // global
```

Static Memory Allocation	Comparison	Dynamic Memory Allocation
• At compile time.	When Allocated	• At run time during program execution.
• Fixed and must be known in advance.	Size	• Can be changed as needed.
• Stack (for local/static variables) or Data segment (for globals).	Memory Area	• Heap.
• Exists until the end of the scope or program.	Lifetime	• Exists until explicitly deallocated.
• Faster allocation and deallocation.	Speed	• Slower than static allocation.
• Less flexible.	Flexibility	• More flexible.
• Stack overflow if memory is insufficient.	Risk of Failure	• May fail if heap memory is insufficient.
• Automatic (when scope ends).	Deallocation	• Manual (using free/delete).
• No fragmentation.	Fragmentation	• May cause fragmentation.
• Low overhead.	Overhead	• Higher overhead.
• `int a[10];`, `float x;`, `static int count;`	Example	• `int *p = (int*)malloc(n * sizeof(int));`

EXAMPLE (C)

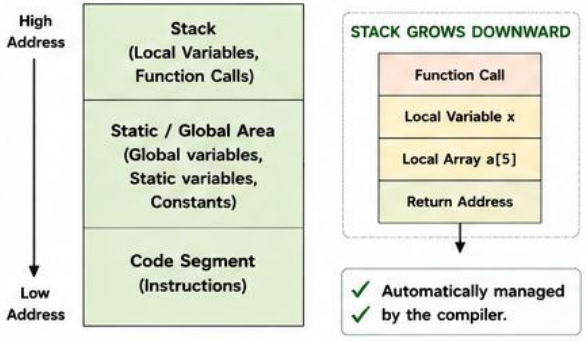
```c
// Dynamic allocation
int *p;
int n = 5;

p = (int*)malloc(n * sizeof(int));
if(p == NULL)
{
    printf("Memory not allocated");
    exit(1);
}

for(int i=0; i<n; i++)
    p[i] = i+1;

free(p);    // deallocate
```

MEMORY LAYOUT (STATIC)



EXAMPLE (C++)

Static Allocation

```cpp
#include <iostream>
using namespace std;

int main()
{
   int arr[5] = {10,20,30,40,50};
   for(int i=0; i<5; i++)
       cout << arr[i] << " ";

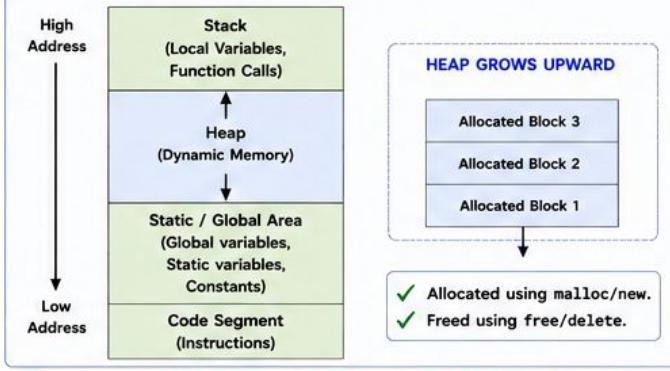
   return 0;
}
```

Dynamic Allocation

```cpp
#include <iostream>
using namespace std;

int main()
{
   int n;
   cin >> n;
   int *arr = new int[n];
   for(int i=0; i<n; i++)
      cin >> arr[i];
   for(int i=0; i<n; i++)
       cout << arr[i] << " ";
   delete[] arr;
   return 0;
}
```

MEMORY LAYOUT (DYNAMIC)



APPLICATIONS OF STATIC ALLOCATION

- When size is fixed and known in advance.
- Small data structures (arrays, structures, primitives).
- Local variables in functions.
- Global constants and configuration values.
- Embedded systems where memory is limited and predictable.
- When performance (speed) is critical.

KEY TAKEAWAY

Use Static Allocation When:

- ✓ Size is known in advance
- ✓ Better performance is needed
- ✓ Simplicity and safety are priority
- ✓ Memory is small and predictable

Use Dynamic Allocation When:

- ✓ Size is not known in advance
- ✓ Large or variable data is required
- ✓ Program needs more flexibility
- ✓ Efficient memory usage is needed

APPLICATIONS OF DYNAMIC ALLOCATION

- When size is not known until runtime.
- Large data structures (linked lists, trees, graphs).
- Data that grows or shrinks during execution.
- Implementing containers (vector, list, map in STL).
- Memory intensive applications.
- When flexibility is required.

Static allocation is simple, fast and safe but inflexible. Dynamic allocation is flexible and powerful but requires careful management to avoid memory leaks and fragmentation.

- Storage Allocation Strategies
  - static allocation lays out storage for all data objects at compile time.
    - Restrictions:
      - » size of object must be known and alignment requirements must be known at compile time.
      - » No recursion.
      - » No dynamic data structure
  - Stack allocation manages the run time storage as a stack
    - The activation record is pushed on as a function is entered.
    - The activation record is popped off as a function exits.
    - Restrictions:
      - » values of locals cannot be retained when an activation ends.
      - » A called activation cannot outlive a caller.

- Heap allocation -- allocates and deallocates stroage as needed at runtime from a data area known as heap.
  - Most flexible: no longer requires the activation of procedures to be LIFO.
  - Most inefficient: need true dynamic memory management.

- Note: static allocation too restricted, heap allocation too inefficient. Most current compiler/language/processor uses the stack allocation scheme.

Activation Tree

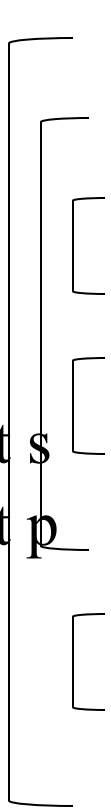
- We can use a tree (called **activation tree**) to show the way control enters and leaves activations.

- In an activation tree:
  - Each node represents an activation of a procedure.
  - The root represents the activation of the main program.
  - The node a is a parent of the node b iff the control flows from a to b.
  - The node a is left to to the node b iff the lifetime of a occurs before the lifetime of b.

Activation Tree (cont.)

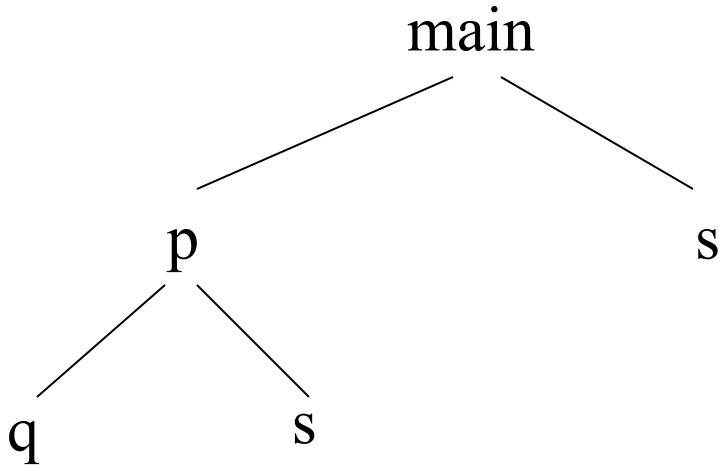
```
program main;
  procedure s;
    begin ... end;
  procedure p;
    procedure q;
      begin ... end;
    begin q; s; end;
  begin p; s; end;
```

enter main
enter p
enter q
exit q
enter s
exit s
exit p
enter s
exit s
exit main



A Nested Structure

Activation Tree (cont.)



Control Stack

- The flow of the control in a program corresponds to a depth-first traversal of the activation tree that:
  - starts at the root,
  - visits a node before its children, and
  - recursively visits children at each node an a left-to-right order.
- A stack (called **control stack**) can be used to keep track of live procedure activations.
  - An activation record is pushed onto the control stack as the activation starts.
  - That activation record is popped when that activation ends.
- When node n is at the top of the control stack, the stack contains the nodes along the path from n to the root.

Variable Scopes

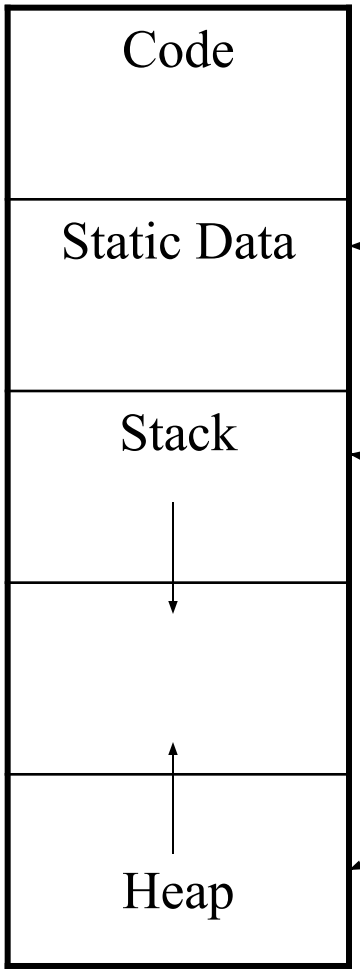
- The same variable name can be used in the different parts of the program.
- The scope rules of the language determine which declaration of a name applies when the name appears in the program.
- An occurrence of a variable (a name) is:
  - **local**: If that occurrence is in the same procedure in which that name is declared.
  - **non-local**: Otherwise (ie. it is declared outside of that procedure)

```
procedure p;
  var b:real;
  procedure p;
    var a: integer;
    begin a := 1;  b := 2; end;
  begin ... end;
```

a is local

b is non-local

Run-Time Storage Organization



Memory locations for code are determined at compile time.

Locations of static data can also be determined at compile time.

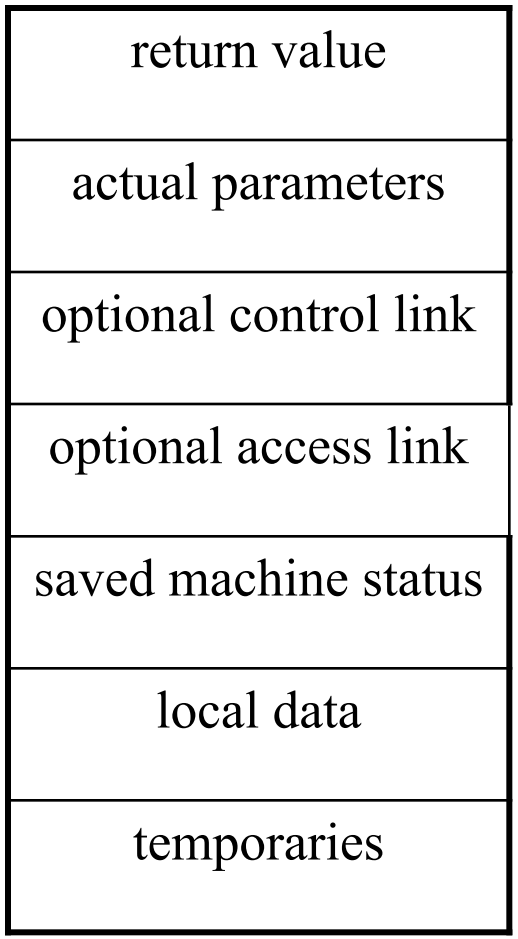
Data objects allocated at run-time. (Activation Records)

Other dynamically allocated data objects at run-time. (For example, malloc area in C).

Activation Records

- Information needed by a single execution of a procedure is managed using a contiguous block of storage called **activation record**.
- An activation record is allocated when a procedure is entered, and it is de-allocated when that procedure exited.
- Size of each field can be determined at compile time (Although actual location of the activation record is determined at run-time).
  - Except that if the procedure has a local variable and its size depends on a parameter, its size is determined at the run time.

Activation Records (cont.)



The returned value of the called procedure is returned in this field to the calling procedure. In practice, we may use a machine register for the return value.

The field for actual parameters is used by the calling procedure to supply parameters to the called procedure.

The optional control link points to the activation record of the caller.

The optional access link is used to refer to nonlocal data held in other activation records.

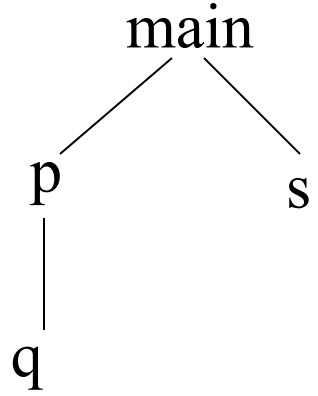
The field for saved machine status holds information about the state of the machine before the procedure is called.

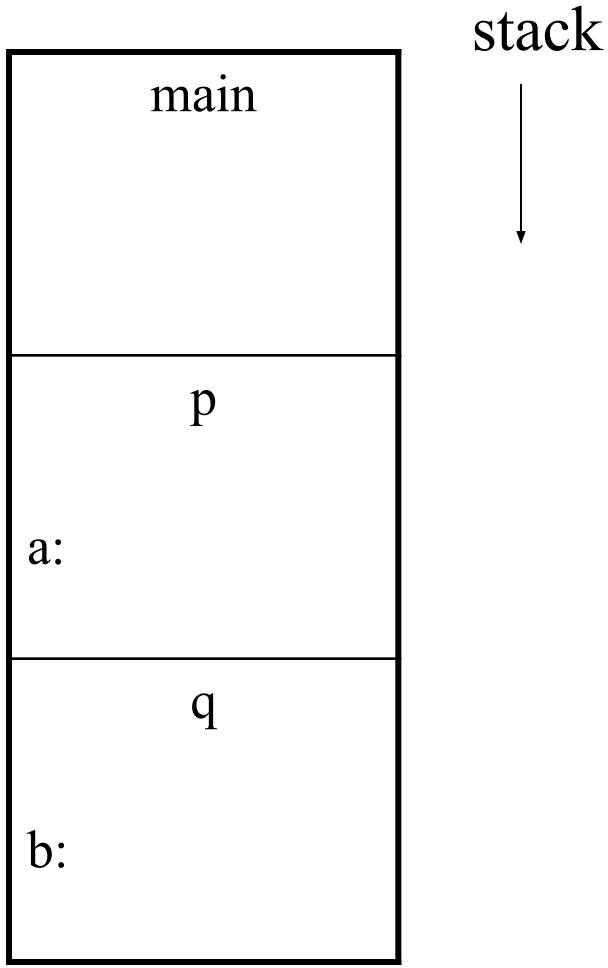
The field of local data holds data that local to an execution of a procedure..

Temporay variables is stored in the field of temporaries.

Activation Records (Ex1)

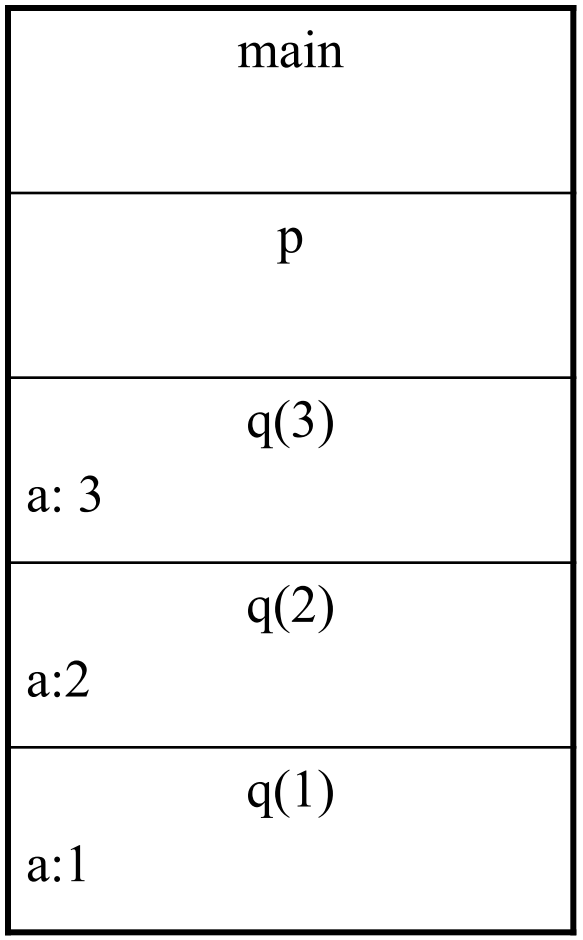
```
program main;
  procedure p;
    var a:real;
    procedure q;
      var
b:integer;
      begin ...
end;
    begin q; end;
  procedure s;
    var c:integer;
    begin ... end;
  begin p; s; end;
```





Activation Records for Recursive Procedures

```
program main;
  procedure p;
    function
q(a:integer):integer;
      begin
        if (a=1) then q:=1;
        else q:=a+q(a-1);
      end;
    begin q(3); end;
  begin p; end;
```

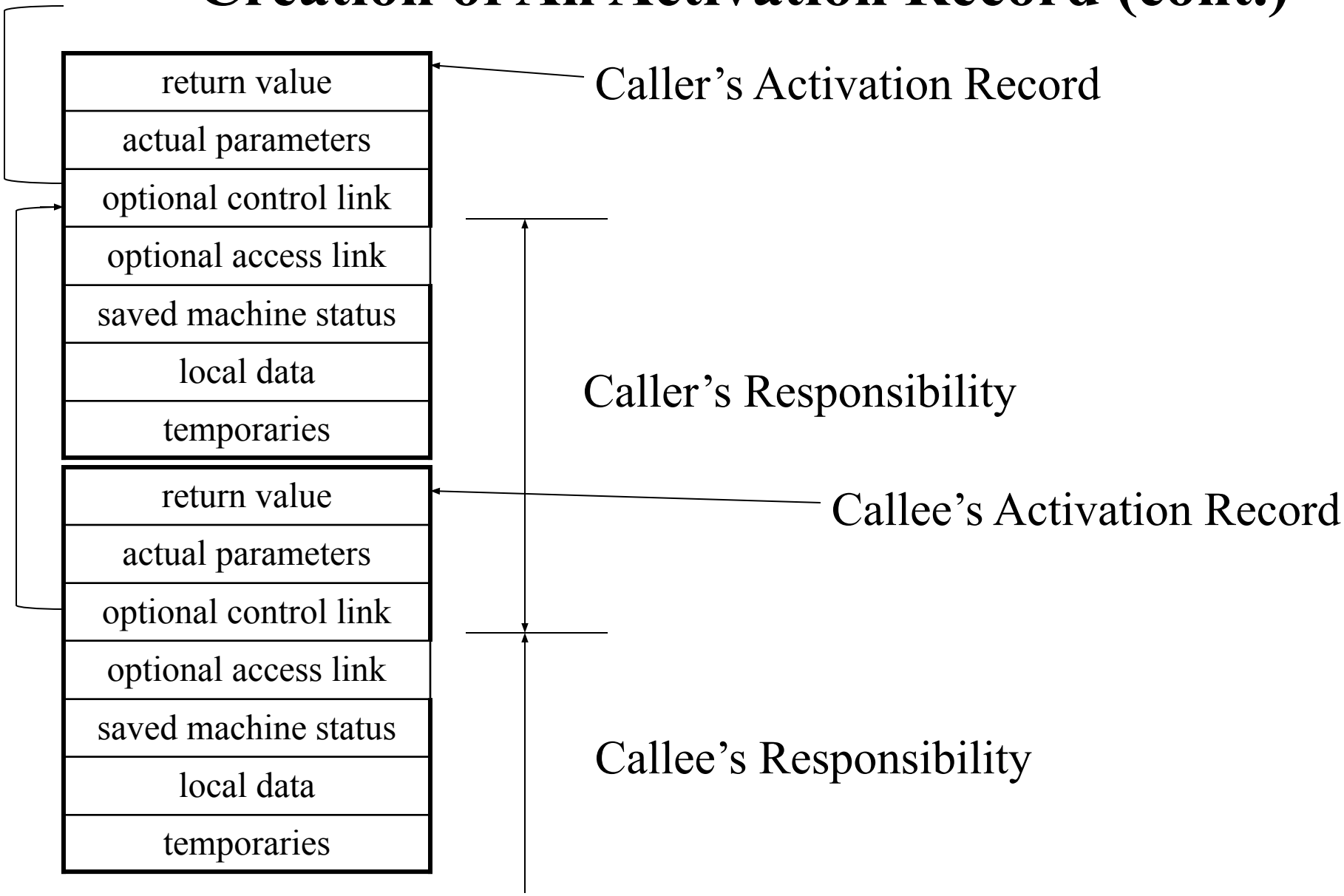


Creation of An Activation Record

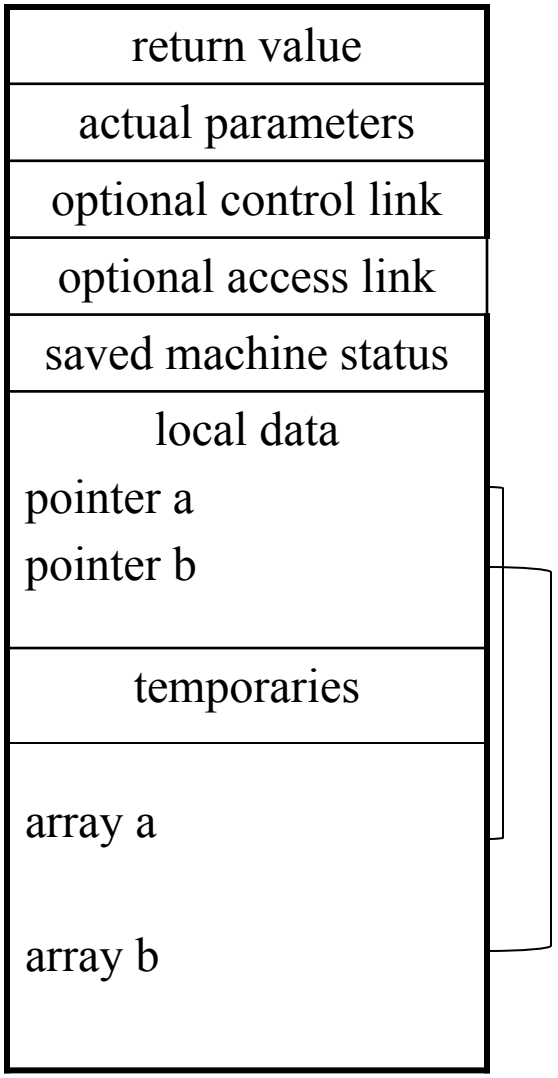
- Who allocates an activation record of a procedure?
  - Some part of the activation record of a procedure is created by that procedure immediately after that procedure is entered.
  - Some part is created by the caller of that procedure before that procedure is entered.

- Who deallocates?
  - Callee de-allocates the part allocated by Callee.
  - Caller de-allocates the part allocated by Caller.

Creation of An Activation Record (cont.)



Variable Length Data



Variable length data is allocated after temporaries, and there is a link to from local data to that array.

Dangling Reference

- Whenever a storage is de-allocated, the problem of dangling references may accur.

```
main () {
  int *p;
  p = dangle();
}
int *dangle*() {
  int i=2;
  return &i;
}
```

DANGLING REFERENCE

A reference to an object that no longer exists.

Accessing a dangling reference leads to **UNDEFINED BEHAVIOR** (crashes, incorrect results, security vulnerabilities).

IN SIMPLE WORDS

A pointer or reference still points to a memory location whose object has already been destroyed or deallocated.

MEMORY VIEW



1 EXAMPLE 1: MEMORY DEALLOCATION

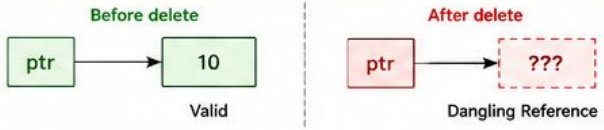
```cpp
#include <iostream>
using namespace std;

int main() {
   int *ptr = new int(10);              // allocate memory
   cout << "Value: " << *ptr << endl;   // OK
   delete ptr;                          // memory released
   cout << "Value: " << *ptr << endl;   // Dangling reference!

   return 0;
}
```

What happens?

1. Memory is allocated for an integer (10).
2. After delete, memory is returned to the system.
3. ptr still holds the old address.
4. Accessing *ptr is undefined behavior.



2 EXAMPLE 2: RETURNING ADDRESS OF LOCAL VARIABLE

```cpp
#include <iostream>
using namespace std;

int* fun() {
   int x = 5;              // local variable
   return &x;              // returning address of x
}

int main() {
   int *p = fun();         // p gets address of x
   cout << "Value: " << *p << endl;  // Dangling!
   return 0;
}
```

What happens?

1. x is created inside fun().
2. Its address is returned.
3. After fun() ends, x is destroyed.
4. p now points to destroyed variable → **Dangling** reference.



3 EXAMPLE 3: OBJECT GOING OUT OF SCOPE (REFERENCE)

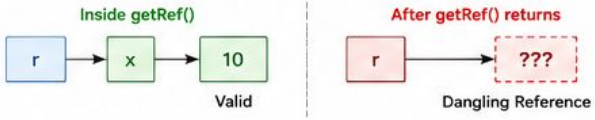
```cpp
#include <iostream>
using namespace std;

int& getRef() {
   int x = 10;
   return x;      // returning reference to local variable
}

int main() {
   int &r = getRef();      // r refers to destroyed x
   cout << r << endl;      // Dangling Reference!
}
```

What happens?

1. x is local to getRef().
2. Reference r binds to x.
3. When getRef() ends, x is destroyed.
4. r becomes a dangling reference.



COMMON CAUSES

- Deallocating memory (delete or free).
- Returning address/reference of a local variable.
- Object goes out of scope while reference/pointer still exists.
- Removing entries in data structures (e.g., symbol table) while other nodes still refer to them (Compiler Design).

EFFECTS

- Program crash
- Unpredictable behavior
- Incorrect results
- Security vulnerabilities

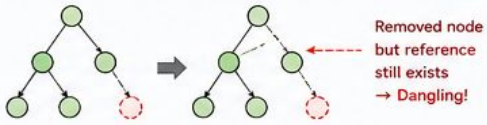
HOW TO AVOID DANGLING REFERENCES

Set pointer to NULL / nullptr after delete	`delete ptr;` `ptr = nullptr;  // now safe`
Use smart pointers (Modern C++)	`std::shared_ptr<int> p =` `  std::make_shared<int>(10);`
Do not return address / reference of local variables	`Return by value, or allocate on heap.`
Manage lifetime carefully in data structures (e.g., AST, DAG, Symbol Table)	Ensure referenced objects remain valid while in use.

IN COMPILER DESIGN

Dangling references may occur in:

- Symbol Tables
- Abstract Syntax Trees (ASTs)
- DAG nodes
- Intermediate Representations

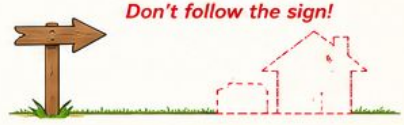


BEST PRACTICES

- Always initialize pointers.
- Always set pointer to nullptr after delete.
- Prefer smart pointers over raw pointers.
- Avoid returning references/pointers to local objects.
- Ensure proper lifetime management.

SUMMARY

A dangling reference is like a signpost pointing to a house that no longer exists.



Access to Nonlocal Names

- Scope rules of a language determine the treatment of references to nonlocal names.
- Scope Rules:
  - **Lexical Scope (Static Scope)**
    - Determines the declaration that applies to a name by examining the program text alone at compile-time.
    - Most-closely nested rule is used.
    - Pascal, C, ..
  - **Dynamic Scope**
    - Determines the declaration that applies to a name at run-time.
    - Lisp, APL, ...

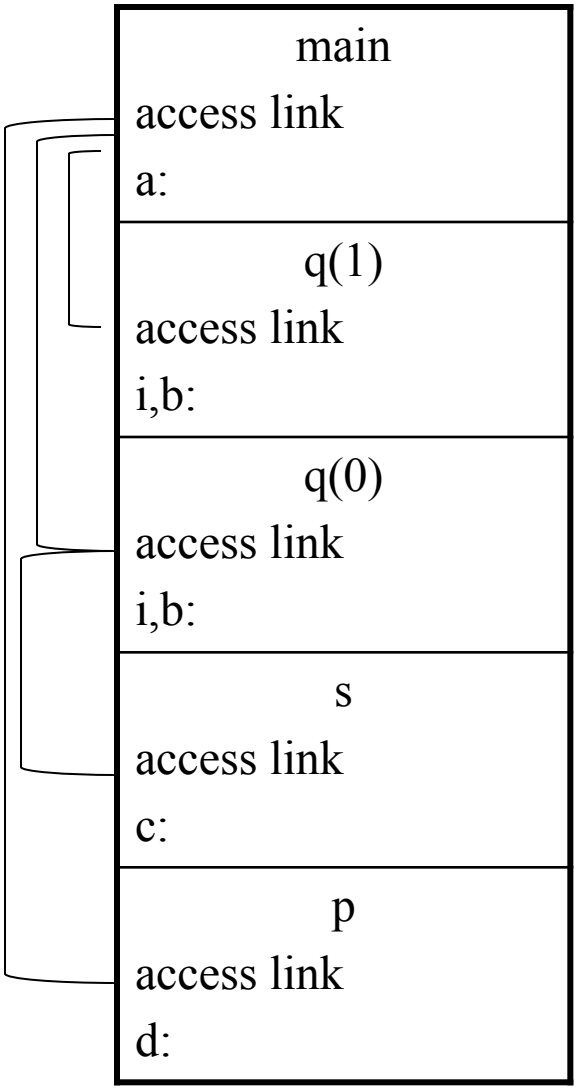
Lexical Scope

- The scope of a declaration in a block-structured language is given by the *mostly closed rule*.
- Each procedure (block) will have its own activation record.
  - procedure
  - begin-end blocks
    - (treated same as procedure without creating most part of its activation record)
- A procedure may access to a nonlocal name using:
  - access links in activation records, or
  - displays (an efficient way to access to nonlocal names)

Access Links

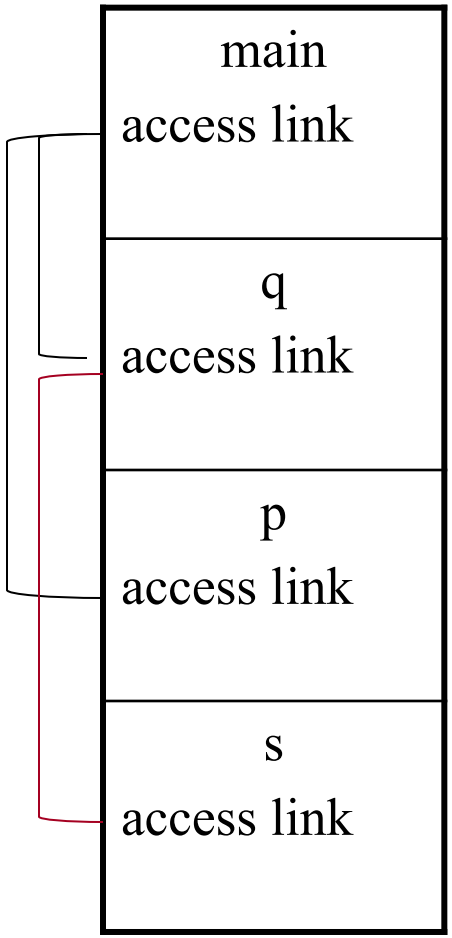
```
program main;
  var a:int;
  procedure p;
    var d:int;
    begin a:=1; end;
  procedure q(i:int);
    var b:int;
    procedure s;
      var c:int;
      begin p; end;
    begin
      if (i<>0) then q(i-1)
      else s;
    end;
  begin q(1); end;
```

Access Links



Procedure Parameters

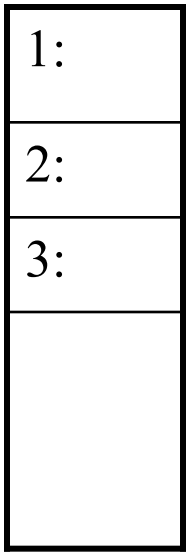
```
program main;
  procedure p(procedure a);
    begin a; end;
  procedure q;
    procedure s;
      begin ... end;
    begin p(s) end;
  begin q; end;
```



Access links must be passed with procedure parameters.

Displays

- An array of pointers to activation records can be used to access activation records.
- This array is called as displays.
- For each level, there will be an array entry.



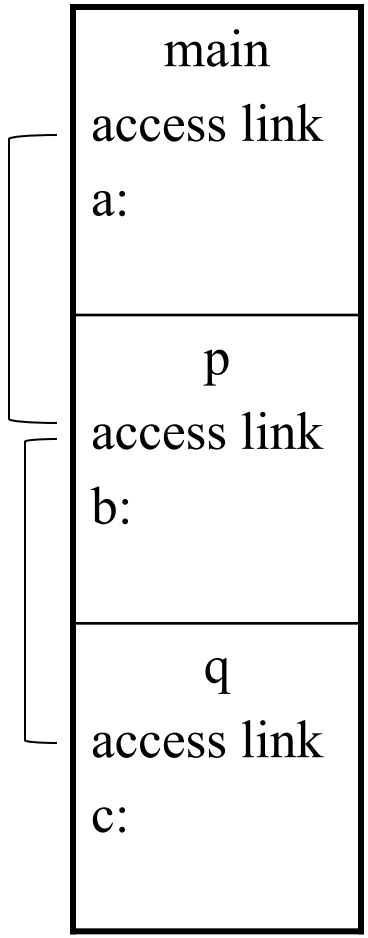
Current activation record at level 1

Current activation record at level 2

Current activation record at level 3

Accessing Nonlocal Variables

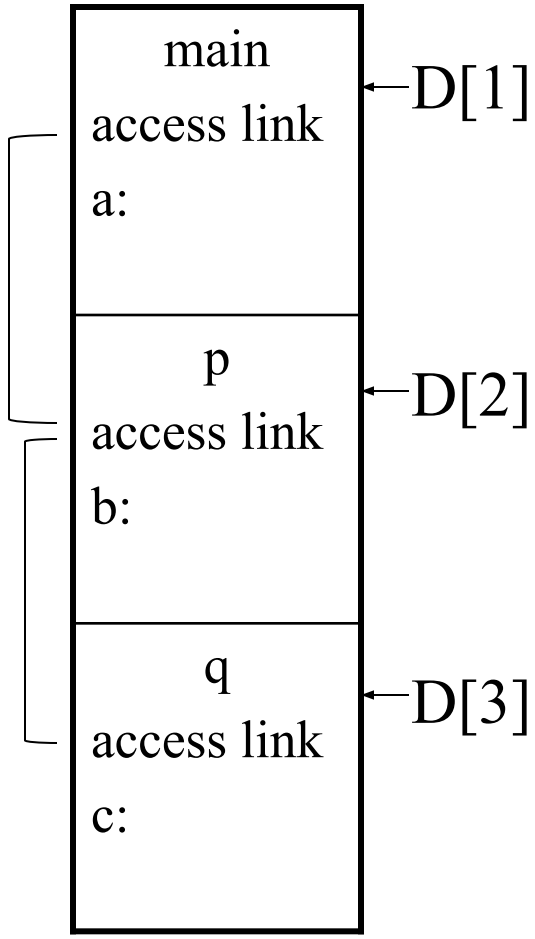
```
program main;
  var a:int;
  procedure p;
    var b:int;
    begin q; end;
  procedure q();
    var c:int;
    begin
      c:=a+b;
    end;
  begin p; end;
```




```
addrC := offsetC(currAR)
t := *currAR
addrB := offsetB(t)
t := *t
addrA := offsetA(t)
ADD addrA,addrB,addrC
```

Accessing Nonlocal Variables using Display

```
program main;
  var a:int;
  procedure p;
    var b:int;
    begin q; end;
  procedure q();
    var c:int;
    begin
      c:=a+b;
    end;
  begin p; end;
```



addrC := offsetC(D[3])
addrB := offsetB(D[2])
addrA := offsetA(D[1])
ADD addrA,addrB,addrC

Parameter Passing Methods

- Call-by-value
- Call-by-reference
- Call-by-name (used by Algol)

Idea:
Different ways of passing arguments control whether the function works on a copy, the original, or even the expression itself.

PARAMETER PASSING TECHNIQUES

How arguments are passed to functions

Value / Data flow
Reference / Link
Memory location

1 CALL-BY-VALUE

A copy of the actual argument's value is passed to the function. Changes made to the parameter do NOT affect the actual argument.

Example (C++)

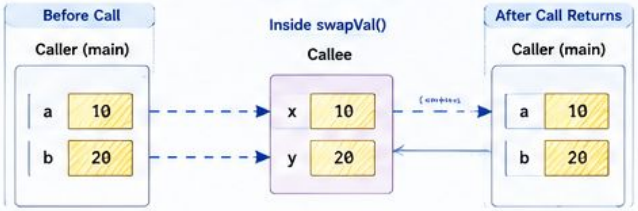
```
void swapVal(int x, int y) {
   int temp = x;
   x = y;
   y = temp;
}

int main() {
   int a = 10, b = 20;
   swapVal(a, b);
   // a = 10, b = 20 (unchanged)
}
```

Execution

1. a and b are copied to x and y.
2. swapVal() works on its local copies.
3. Original a and b remain unchanged.

Memory / Stack Illustration



Key Point
Safe and simple, but modifications inside the function DO NOT affect the actual arguments.

2 CALL-BY-REFERENCE

The address (reference) of the actual argument is passed to the function. The parameter becomes an alias of the actual argument. Changes affect the original.

Example (C++)

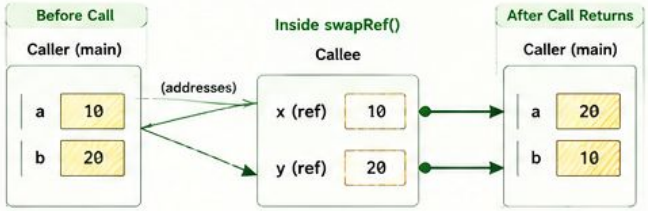
```
void swapRef(int &x, int &y) {
   int temp = x;
   x = y;
   y = temp;
}

int main() {
   int a = 10, b = 20;
   swapRef(a, b);
   // a = 20, b = 10 (swapped)
}
```

Execution

1. Addresses of a and b are passed to x and y.
2. x and y refer to the same memory as a and b.
3. Changes through x and y modify a and b.

Memory / Stack Illustration



Key Point
Efficient and allows modification of actual arguments from inside the function.

3 CALL-BY-NAME

The actual argument's expression (not its value) is passed. The expression is re-evaluated every time the parameter is used in tha function.

Example (C / Macro-like)

```
#define SQR(x) ((x) * (x))

int main() {
   int i = 5;
   int r = SQR(i++);
   // Expands to (i++) * (i++)
   // r = 5 * 6 = 30, i = 7
}
```

Execution

1. The expression i++ is substituted for x.
2. Each use of x re-evaluates i++.
3. Side effects may occur multiple times.

Evaluation Steps

SQR(i++) → (i++) * (i++)

1. i = 5
2. Left (i++) → uses 5, then i becomes 6
3. Right (i++) → uses 6, then i becomes 7
4. Result = 5 * 6 = 30

Timeline

Step	Expression Used	i's Value
Start	–	5
1	Left i++	(value 5) → 6
2	Right i++	(value 6) → 7
End	–	7

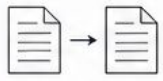
Key Point
Powerful but dangerous. The expression is evaluated every time; side effects may occur multiple times.

COMPARISON SUMMARY

Aspect	Call-by-Value	Call-by-Reference	Call-by-Name
What is passed?	Copy of value	Address (reference)	Expression (text)
Can modify actual argument?	No	Yes	Yes (via re-evaluation)
Evaluation frequency	Once (before call)	Once (address passed)	Every time parameter is used
Overhead	May copy large data	Low	Potentially high
Safety	Safe	Safe (if used carefully)	Risky (side effects)
Typical Use	General purpose	When modification is needed	Macros / Special cases
Common in	C, C++, Java, Python	C++ (&), Java (object ref)	Macros (C), Thunks (Algol)

ANALOGIES

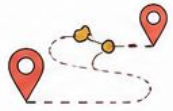
Call-by-Value
Like giving a photocopy of a document.



Call-by-Reference
Like giving the address of your house.



Call-by-Name
Like giving directions on how to reach your house, used every time.



NOTES

- Call-by-Value is default in most programming languages.
- Call-by-Reference provides efficiency and ability to modify caller's data.
- Call-by-Name is mostly a theoretical concept; implemented via macros or special mechanisms.



SOME OTHER ISSUES

- Symbol Tables hold scope information.
- Dynamic Scope:
  - uses control link (similar to access links)
- Dynamic Storage Allocation (Heap Management)
  - different allocation techniques

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

A. Identify the problem with this grammar.
B. Rewrite it into an equivalent grammar suitable for Recursive Descent Parsing.
C. Explain why the transformed grammar works

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T' \mid \varepsilon$

$F \rightarrow (E) \mid id$